Operating Systems (OS) - తెలుగు గైడ్ (SSE Fundamentals)

> ఈ document చదివిన తర్వాత Operating Systems మళ్ళీ జీవితంలో మర్చిపోలేవు. OS అస్సలు తెలియని వ్యక్తిని — ZERO నుండి — Senior Software Engineer (SSE) interview crack చేసే స్థాయికి తీసుకెళ్లడమే లక్ష్యం. ప్రతి concept కి ఒక vivid real-life analogy, ఎందుకు ఉంది / ఎప్పుడు matter అవుతుంది, ఒక ASCII diagram, trade-offs, gotchas, మరియు interview దృష్టి (అడిగే ప్రశ్న + crisp సమాధానం) — అన్నీ ఉంటాయి. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."
>
> **నీ background:** నీకు JavaScript / React / Node.js / MongoDB (MERN) తెలుసు, కానీ CS degree లేదు, OS అస్సలు చదవలేదు. అందుకే ఇక్కడ ప్రతి OS concept ని వీలైనంత వరకు నీ **web/Node.js knowledge** తో connect చేస్తాను (ఉదా: "Node.js event loop vs OS threads", "libuv thread pool", "MongoDB connection = ఒక process/socket"). దీంతో కొత్త concept కూడా already తెలిసిన దానితో ముడిపడి బుర్రలో నిలిచిపోతుంది.
>
> **లక్ష్యం:** Programming వచ్చిన కానీ OS fundamentals తెలియని engineer ని absolute basics నుండి SSE interview-ready వరకు తీసుకెళ్లడం. Best-teacher style — intuition first, formalism తర్వాత. ఇది CS-fundamentals set లో ఒక భాగం: `DSA_00_Foundations_Telugu.md` , `OOPS_Telugu.md` , `HLD_Telugu.md` , `LLD_Telugu.md` , `SystemDesign_Go_Telugu.md` లతో కలిపి చదువు.

విషయ సూచిక (Table of Contents)

Part 1 — Basics (పునాదులు)

1. OS అంటే ఏమిటి, ఎందుకు (role, kernel vs user mode, system calls, OS types)
2. Process (program vs process, PCB, process states diagram, fork/exec)
3. Threads (thread vs process, user vs kernel threads, multithreading, thread pools; Node.js single-thread connection)
4. Context Switching (mechanism, cost, ఎప్పుడు జరుగుతుంది)

Part 2 — Scheduling (CPU ని ఎవరికి ఇవ్వాలి)

5. CPU Scheduling basics (criteria: throughput / turnaround / waiting / response, preemptive vs non-preemptive)
6. Scheduling Algorithms (FCFS, SJF, SRTF, Priority, Round Robin, MLFQ) — ప్రతిదానికి Gantt chart + waiting/turnaround calculation

Part 3 — Concurrency (ఒకేసారి చాలా పనులు — safe గా)

7. Race Conditions & Critical Section (problem + 4 requirements)
8. Synchronization (mutex, binary/counting semaphore, monitors, condition variables)
9. Classic Problems (Producer-Consumer, Readers-Writers, Dining Philosophers) — solutions తో

10. Deadlocks (4 Coffman conditions, prevention, avoidance / Banker's algorithm with example, detection, recovery)

Part 4 — Memory (RAM ని ఎలా పంచాలి)

11. Memory Management (logical vs physical, MMU, contiguous allocation, fragmentation internal/external)
12. Paging (pages/frames, page table, TLB, multi-level page table)
13. Segmentation (vs paging)
14. Virtual Memory & Demand Paging (page faults, replacement: FIFO/LRU/Optimal/Clock with traces, Belady's anomaly, thrashing, working set)

Part 5 — Storage & I/O

15. File Systems (files, directories, allocation, inodes)
16. Disk Scheduling (FCFS / SSTF / SCAN / C-SCAN with examples)
17. I/O & Interrupts (polling vs interrupt, DMA)

Part 6 — Reference (interview కి చివరి మెరుగు)

18. IPC — Inter-Process Communication (pipes, shared memory, message passing, sockets)
19. Interview Q&A + Memory Tips table + Common Mistakes

Part 1 — Basics (పునాదులు)

> OS ని అర్థం చేసుకోవాలంటే మొదట ఒక్క ప్రశ్న అడుక్కో: **"నా code run అయ్యేటప్పుడు, hardware (CPU, RAM, disk) ని ఎవరు manage చేస్తున్నారు?"** సమాధానం — Operating System. ఈ Part లో OS అంటే ఏమిటి, program ఎలా ఒక living process అవుతుంది, ఒక process లోపల threads ఎలా పని చేస్తాయి, మరియు CPU ఒక పని నుండి ఇంకో పనికి ఎలా switch అవుతుంది — ఈ నాలుగు పునాది concepts నేర్చుకుంటాం. ఇవి strong గా ఉంటే మిగతా OS అంతా సులభం.

---

1. OS అంటే ఏమిటి, ఎందుకు

వివరణ

Operating System (OS) అంటే — నీ computer లో hardware (CPU, RAM, disk, network card, keyboard) కి మరియు నీ applications (Chrome, VS Code, నీ Node.js server) కి **మధ్యలో ఉండే software layer.** ఇది hardware ని directly ఎవ్వరూ ముట్టుకోకుండా, అందరికీ *fair* గా, *safe* గా, *efficient* గా పంచిపెట్టే **manager.**

ఒక్క వాక్యంలో: **OS = నీ computer యొక్క resource manager + hardware ని దాచే abstraction layer.**

OS చేసే మూడు ముఖ్య పనులు:

1. **Resource management** — CPU ని ఏ program కి ఎంత సేపు ఇవ్వాలి, RAM ని ఎవరికి ఎంత ఇవ్వాలి, disk read/write లని ఎలా order చేయాలి — ఇవన్నీ OS నిర్ణయిస్తుంది.
2. **Abstraction (hardware ని దాచడం)** — నువ్వు `fs.writeFile("a.txt", data)` అని రాస్తే, disk లో ఏ physical sector కి, ఏ voltage తో, ఏ controller command తో రాయాలో నీకు తెలియనవసరం లేదు. OS ఆ complexity ని దాచి, నీకు simple "file" అనే idea ఇస్తుంది.
3. **Protection & isolation** — ఒక program ఇంకో program యొక్క memory ని చదవకూడదు / corrupt చేయకూడదు. Chrome crash అయితే నీ Node server crash కాకూడదు. ఈ గోడలు OS కడుతుంది.

నీ MERN connection: నువ్వు `node server.js` అని run చేసినప్పుడు — Node.js runtime కూడా ఒక program మాత్రమే. అది CPU ని directly control చేయదు; అది OS ని system calls ద్వారా అడుగుతుంది ("ఈ port మీద socket తెరువు", "ఈ file చదువు", "ఈ memory ఇవ్వు"). అంటే **నీ Express server కి OS ఒక silent landlord** — CPU time, RAM, network sockets అన్నీ OS అద్దెకి ఇస్తుంది.

Real-life Scenario

> **OS = ఒక పెద్ద apartment building యొక్క manager (watchman + caretaker).**
>
> Building = నీ computer hardware. Tenants (అద్దెదారులు) = నీ applications (Chrome, VS Code, Node server). ఒక్కో tenant కి నేరుగా building యొక్క main electricity board, water pump, main gate కి access ఇస్తే — గందరగోళం, ప్రమాదం (ఒకరు water మొత్తం వాడేస్తారు, ఇంకొకరు పక్కవాడి flat తలుపు తీస్తారు).
>
> అందుకే **manager (OS)** ఉంటాడు:
>
> - నీకు water (RAM) కావాలంటే manager ని అడుగు — అతను fair గా share చేస్తాడు.
> - నువ్వు పక్క flat (ఇంకో process memory) లోకి తొంగి చూడలేవు — manager గోడ కట్టాడు.
> - Common lift (CPU) ని ఎవరు ముందు వాడాలి అనేది manager schedule చేస్తాడు.
>
> నీకు building wiring internals తెలియనవసరం లేదు — "నీళ్ళు కావాలి" అంటే చాలు. అదే **abstraction.** ఈ manager లేకపోతే ప్రతి tenant నేరుగా hardware తో కుస్తీ పట్టాలి. **OS ఉండటం వల్లే నువ్వు `fs.readFile` రాసి happy గా ఉన్నావు.**

Kernel vs User Mode — OS యొక్క గుండె

CPU లో ప్రతి instruction రెండు modes లో ఏదో ఒకదాంట్లో run అవుతుంది. ఇది hardware-level protection.

```
┌────────────────────────────────────────────┐
│              USER MODE (limited)              │
│  Chrome | VS Code | node server.js | your JS │
│  - directly hardware ముట్టుకోలేదు            │
│  - dangerous instructions run చేయలేదు            │
└────────────────────────────────────────────┘
                    │  system call (trap)
                    │  "OS, ఈ పని చేయవా?"
                    ▼
┌────────────────────────────────────────────┐
│            KERNEL MODE (full power)           │
│  OS Kernel — CPU/RAM/disk/network అన్నీ        │
│  control చేసే privileged code                 │
└────────────────────────────────────────────┘
```

- **User mode** — నీ application code ఇక్కడ run అవుతుంది. CPU ఇక్కడ కొన్ని "privileged" instructions (disk కి directly write, other process memory access, CPU ని halt) execute చేయనివ్వదు. తప్పు చేస్తే CPU వెంటనే kernel కి control ఇస్తుంది (exception).
- **Kernel mode** (supervisor mode) — OS kernel ఇక్కడ run అవుతుంది. అన్నీ instructions అనుమతి. Hardware మొత్తం access.

ఎందుకు ఈ రెండు modes? ఒక్క mode మాత్రమే ఉంటే, ఏ buggy program అయినా OS ని / మిగతా programs ని corrupt చేయగలదు. రెండు modes = **గోడ.** నీ code తప్పు చేసినా OS బతికి ఉంటుంది. (నీ Node process crash అయినా OS + మిగతా apps బతికే ఉంటాయి — ఇదే isolation.)

System Calls — user నుండి kernel కి "door"

User mode code కి hardware directly access లేదు కదా. మరి file ఎలా చదువుతుంది? **System call** ద్వారా. System call = user program kernel ని "ఈ privileged పని నా తరఫున చేయవా?" అని అడిగే **official request.** ఇది ఒక controlled door — CPU mode ని user → kernel కి switch చేస్తుంది (దీన్ని *trap* / *mode switch* అంటారు).

నీ high-level code	లోపల జరిగే system call (Linux)	OS ఏం చేస్తుంది
`fs.readFile("a.txt")`	`open()` , `read()` , `close()`	disk నుండి bytes చదివి ఇస్తుంది
`require('net')` server, `.listen(3000)`	`socket()` , `bind()` , `listen()` , `accept()`	network port ని program కి కేటాయిస్తుంది
`console.log("hi")`	`write()` (fd 1 = stdout)	terminal కి bytes రాస్తుంది
`child_process.spawn()`	`fork()` , `execve()`	కొత్త process సృష్టిస్తుంది
`new Array(1e6)` (memory పెరిగితే)	`brk()` / `mmap()`	process కి ఇంకా RAM ఇస్తుంది

కీలక insight: నీ MERN app లో నువ్వు రాసే ప్రతి I/O line చివరికి ఒక system call అవుతుంది. System call **ఖరీదైనది** (mode switch cost ఉంది) — అందుకే Node.js I/O ని batch చేసి, non-blocking గా చేస్తుంది. "System call ఖరీదు" అనే idea Node performance కి foundation.

OS Types — ఎన్ని రకాలు, ఎందుకు

రకం	ఏంటి	ఎక్కడ	ఉదాహరణ
Batch OS	Jobs ని గుంపుగా (batch) పోగుచేసి, ఒకదాని తర్వాత ఒకటి, human interaction లేకుండా run చేస్తుంది	పాత mainframes, payroll runs	పాత IBM systems
Time-sharing (multitasking)	చాలా users/programs CPU ని **వేగంగా turn-by-turn** పంచుకుంటారు — అందరికీ "నాకే machine" అనిపిస్తుంది	Desktops, servers	Linux, Windows, macOS
Real-time OS (RTOS)	పని **deadline లోపల ఖచ్చితంగా** పూర్తవ్వాలి — timing guarantee ముఖ్యం	Car airbags, pacemakers, robotics	VxWorks, FreeRTOS
Distributed OS	చాలా machines కలిసి ఒక్క system లా కనిపిస్తాయి	Clusters, cloud	(concept; k8s దీనికి దగ్గర)
Embedded OS	చిన్న device లో, తక్కువ resources తో	Router, smart TV, IoT	Embedded Linux

SSE కి ముఖ్యమైనవి: Time-sharing (నీ servers ఇవే — Linux) మరియు RTOS concept (hard vs soft real-time తేడా). RTOS లో "correct కానీ late = wrong" — ఇది normal server OS కి వేరు (server లో late అయినా OK, correctness ముఖ్యం).

Key Points

- **OS = hardware ↔ apps మధ్య manager:** resource management + abstraction + protection.
- **Kernel mode = full power, user mode = limited.** ఈ గోడ వల్లే ఒక buggy app మొత్తం system ని కూల్చదు.
- **System call = user code kernel ని పని అడిగే controlled door** (trap / mode switch). File I/O, network, process creation, memory — అన్నీ system calls.
- **System call ఖరీదైనది** (mode switch cost) — అందుకే performance-sensitive code వాటిని తగ్గిస్తుంది.
- **Node.js కూడా ఒక user-mode program:** నీ `fs`, `net`, `http` calls అన్నీ చివరికి OS system calls.
- OS types: batch, time-sharing (నీ servers), real-time (deadline guarantee), distributed, embedded.

Interview దృష్టి

Q1: Kernel mode vs user mode ఎందుకు అవసరం?

> Protection కోసం. User mode లో privileged instructions (hardware access, other process memory, CPU halt) block అవుతాయి. దీనివల్ల ఒక buggy/malicious application OS ని లేదా మిగతా processes ని corrupt చేయలేదు. Privileged పని కావాలంటే system call ద్వారా kernel ని అడగాలి — ఆ door దగ్గర OS validate చేస్తుంది.

Q2: System call అంటే ఏమిటి, function call కి ఎలా వేరు?

> Function call = అదే process లో, అదే user mode లో ఒక code నుండి ఇంకో code కి jump. System call = user mode నుండి kernel mode కి controlled switch (trap), OS privileged పని చేసి తిరిగి control ఇస్తుంది. System call cost ఎక్కువ (mode switch + context save). ఉదా: `read()`, `write()`, `fork()`, `mmap()`.

Q3: `console.log` లోపల ఏం జరుగుతుంది (OS దృష్టిలో)?

> V8/Node string ని format చేసి, `write()` system call ద్వారా file descriptor 1 (stdout) కి bytes పంపుతుంది. అది trap అయి kernel mode లోకి వెళ్ళి, terminal device / pipe కి bytes రాస్తుంది. అంటే ఒక్క `console.log` కూడా ఒక system call — అందుకే tight loop లో log చేస్తే performance పడుతుంది.

Q4: OS లేకపోతే ఏమవుతుంది?

> ప్రతి program నేరుగా hardware ని manage చేయాల్సి వస్తుంది — CPU scheduling, memory allocation, device drivers అన్నీ program లోనే. Multitasking, isolation, security ఉండవు; ఒక program crash మొత్తం machine ని కూల్చవచ్చు. (Embedded లో కొన్నిసార్లు "bare-metal" — OS లేకుండా — రాస్తారు, కానీ అప్పుడు programmer యే mini-OS రాయాలి.)

2. Process (program vs process, PCB, states, fork/exec)

వివరణ

ఇక్కడ ఒక చాలా ముఖ్యమైన తేడా అర్థం చేసుకో – **program vs process:**

- **Program** = disk లో పడి ఉన్న నిర్జీవ file. నీ `server.js` file, లేదా `/usr/bin/node` binary. ఇది కేవలం instructions (recipe). RAM లో ఏమీ లేదు, run అవ్వట్లేదు.
- **Process** = ఆ program ని RAM లోకి load చేసి, CPU మీద run చేస్తున్న **సజీవ instance.** దీనికి own memory, own state, own resources ఉంటాయి.

ఒక్క వాక్యంలో: **Program = recipe (కాగితం); Process = ఆ recipe ప్రకారం వంట చేస్తున్న actual cooking session (గ్యాస్ వెలుగుతోంది).**

కీలక insight: **ఒకే program → చాలా processes.** ఉదా: `node server.js` ని రెండు terminals లో run చేస్తే → రెండు వేరే processes (వేరే PID, వేరే memory), కానీ అదే program. అలాగే Chrome లో ప్రతి tab ఒక వేరే process (అందుకే ఒక tab crash అయినా మిగతావి బతుకుతాయి).

Real-life Scenario

> **Program vs Process = recipe పుస్తకం vs వంట session.**
>
> నీ దగ్గర "బిర్యానీ recipe" (program) ఒక్కటే పుస్తకంలో ఉంది. కానీ నువ్వు, నీ friend – ఇద్దరూ ఒకేసారి రెండు వేరే kitchens లో అదే recipe తో బిర్యానీ వండుతున్నారు. అవి **రెండు వేరే cooking sessions (processes):** వేరే kitchen (memory), వేరే గ్యాస్ stage (CPU register state), వేరే సగం అయిన పని ("ఇప్పుడు మసాలా వేస్తున్నా" vs "ఇప్పుడు అన్నం కలుపుతున్నా" – program counter).
>
> Recipe పుస్తకం (program) ఒక్కటే, కానీ దాన్ని ప్రాణం పోసి run చేస్తున్న sessions (processes) ఎన్నయినా. ఒకరి kitchen లో పొగ వచ్చినా (crash), రెండో kitchen ప్రశాంతంగా ఉంటుంది. **ఇదే process isolation.**

ఒక Process యొక్క Memory Layout

RAM లో ఒక process ఇలా అమర్చబడుతుంది (address చిన్నది → పెద్దది):

```
  High address
 ┌────────────────────────┐
 │         STACK          │  ← function calls, local variables
 │           │            │    (కిందికి పెరుగుతుంది ↓)
 │           ▼            │
 │                        │
 │      (free space)      │
 │                        │
 │           ▲            │
 │           │            │  ← malloc/new, objects
 │          HEAP          │    (పైకి పెరుగుతుంది ↑)
 ├────────────────────────┤
 │   BSS / DATA           │  ← global & static variables
 ├────────────────────────┤
 │   TEXT (code)          │  ← program instructions (read-only)
```

```
└──────────────────────┘
 Low address
```

- **Text** — program యొక్క machine code (read-only, అందుకే code accidentally overwrite అవ్వదు).
- **Data / BSS** — global, static variables (initialized / uninitialized).
- **Heap** — runtime లో dynamically allocate అయ్యే memory (JS లో నీ objects, arrays ఇక్కడే; V8 heap ఇదే). పైకి పెరుగుతుంది.
- **Stack** — function call frames, local variables, return addresses. కిందికి పెరుగుతుంది. చాలా deep recursion → stack overflow (JS లో `RangeError: Maximum call stack size exceeded` — ఇదే stack పరిమితి).

Heap ↔ Stack ఒకదానివైపు ఒకటి పెరుగుతాయి; మధ్యలో free space అయిపోతే memory అయిపోయినట్టు.

PCB — Process Control Block (process యొక్క "Aadhaar card")

OS ప్రతి process గురించి ఒక record పెట్టుకుంటుంది — దాన్ని **PCB (Process Control Block)** అంటారు. Process ని pause చేసి తర్వాత తిరిగి కొనసాగించాలంటే, దాని *మొత్తం state* ఎక్కడో save అవ్వాలి కదా — అదే PCB.

PCB లో ఏం ఉంటుంది:

Field	ఏంటి
PID	Process ID — unique number (నీ `process.pid` in Node)
Process state	running / ready / waiting / ...
Program counter (PC)	తర్వాత execute చేయాల్సిన instruction address
CPU registers	pause అయినప్పుడు registers లో ఉన్న values
Memory info	page table, base/limit registers (దీని memory ఎక్కడ)
Open files	file descriptor table (ఏ files/sockets తెరిచి ఉన్నాయి)
Scheduling info	priority, ఎంత CPU వాడింది
Parent PID (PPID)	ఎవరు దీన్ని create చేశారు

PCB = process యొక్క పూర్తి "snapshot సేవ్ చేసే చోటు." Context switch (Topic 4) అంటే — ఒక process PCB లో state save చేసి, ఇంకో process PCB నుండి state load చేయడం.

Process States — ఒక process యొక్క జీవితం

ఒక process ఎప్పుడూ ఏదో ఒక state లో ఉంటుంది:

```
                 ┌─────────┐
    create       │         │    admit
  ─────────────→ │   NEW   │ ──────────→┐
                 └─────────┘            │
                                        ▼
┌──────────────────────────────────────→┌─────────┐
│              (scheduler dispatch)     │  READY  │←─────────┐
```

```
   |                ┌─────────────────────────┐ ┌──────────┐        |
   |                ▼                                      |        |
   |                                                       |        |   I/O done /
   |         ┌──────────┐       time-slice over            |        |   event occurs
   |         │ RUNNING  │──────────────────────────────────┘        |
   |         └──────────┘       (interrupt)                         |
   |               |                                                |
   |   needs I/O|  (e.g. DB read, file, network)                    |
   |               ▼                                                |
   |         ┌──────────┐                                           |
   |         │ WAITING  │───────────────────────────────────────────┘
   |         │(blocked) │
   |         └──────────┘
   |
   |   exit()
   └────────────────▶ ┌─────────────┐
                      │ TERMINATED  │
                      └─────────────┘
```

State	అర్థం	నీ Node ఉదాహరణ
New	process create అవుతోంది, ఇంకా ready కాలేదు	`spawn()` మొదలైన క్షణం
Ready	run అవ్వడానికి పూర్తిగా సిద్ధం, కానీ CPU కోసం queue లో ఎదురుచూస్తోంది	CPU కోసం wait
Running	ప్రస్తుతం CPU మీద execute అవుతోంది (ఒక core కి ఒకసారి ఒకటే)	నీ JS actually run అవుతోంది
Waiting/Blocked	ఒక event (I/O పూర్తి, lock) కోసం ఆగింది; CPU వద్దు	`await db.find()` — DB reply కోసం wait
Terminated	పని అయిపోయింది / kill అయ్యింది	`process.exit()`

కీలక తేడా — Ready vs Waiting: Ready = "CPU ఇస్తే వెంటనే run చేస్తా" (పని ఉంది, CPU లేదు). Waiting = "CPU ఇచ్చినా ఇప్పుడు run చేయలేను, నా I/O పూర్తవ్వాలి." ఈ తేడా interviews లో తరచూ అడుగుతారు.

Node connection: నీ async code (`await fetch` , `await db.query`) I/O కోసం wait చేసేటప్పుడు — OS దృష్టిలో ఆ process **Waiting/Blocked** కావచ్చు, కానీ Node event loop ఆ సమయంలో అదే thread మీద *వేరే* callbacks run చేస్తుంది. అందుకే Node "non-blocking" — ఒక request DB కోసం ఆగినా, thread ఖాళీగా కూర్చోకుండా ఇంకో request handle చేస్తుంది.

fork() మరియు exec() — కొత్త process ఎలా పుడుతుంది

Unix/Linux లో కొత్త process create చేయడం రెండు steps:

1. `fork()` — ప్రస్తుత process ని **నకలు (clone)** చేస్తుంది. Parent process యొక్క exact copy — అదే code, అదే memory (copy) — తయారవుతుంది (child). `fork()` ఒకసారి పిలిస్తే **రెండుసార్లు return అవుతుంది:** parent కి child యొక్క PID, child కి 0.

```
parent process (PID 100)
      |  fork()
      ├──────────────▶ child process (PID 101) — parent యొక్క clone
      |                     |
```

```
    fork() returns 101    fork() returns 0
    (child PID)           (నేను child అని తెలుసు)
```

2. `exec()` (execve) — ప్రస్తుత process యొక్క memory ని పూర్తిగా **కొత్త program తో replace** చేస్తుంది. PID అదే ఉంటుంది, కానీ లోపల code మొత్తం మారిపోతుంది.

కలిపి fork + exec = "కొత్త program run చేయడం": shell లో నువ్వు `ls` type చేస్తే — shell `fork()` చేసి (తన clone తయారుచేసి), ఆ child లో `exec("ls")` చేస్తుంది (clone ని `ls` program గా మార్చుతుంది). Parent (shell) `wait()` తో child పూర్తయ్యేదాకా ఆగుతుంది.

Node connection: `child_process.spawn('node', ['worker.js'])` లోపల ఇదే fork+exec జరుగుతుంది. Node `cluster` module కూడా అనేక worker processes ని ఇలా create చేసి, ఒకే port మీద CPU cores అన్నిటినీ వాడుతుంది (single Node process ఒక్క core మాత్రమే వాడుతుంది కాబట్టి).

> **Copy-on-Write (COW) — smart optimization:** `fork()` parent memory మొత్తం వెంటనే copy చేయదు (ఖరీదు!). Parent, child మొదట అదే physical pages share చేస్తారు (read-only గా mark). ఎవరైనా write చేసినప్పుడే ఆ page copy అవుతుంది. అందుకే fork వేగం — "నిజంగా మార్చే memory మాత్రమే copy."

Key Points

- **Program = disk లో నిర్జీవ file; Process = RAM లో run అవుతున్న సజీవ instance.** ఒకే program → అనేక processes.
- Process memory layout: **Text (code) / Data / Heap (↑, objects) / Stack (↓, calls).** Deep recursion → stack overflow.
- **PCB = process యొక్క పూర్తి state record** (PID, PC, registers, memory info, open files). Context switch = PCB save/load.
- **5 states:** New → Ready → Running → (Waiting) → Terminated. **Ready = CPU కావాలి; Waiting = I/O కావాలి** — ఈ తేడా గుర్తుంచుకో.
- **fork() = process clone (2 returns); exec() = program replace.** కలిపి కొత్త program run. **Copy-on-Write** వల్ల fork వేగం.
- Node: `cluster` / `child_process` లోపల fork+exec; multiple processes తో multi-core వాడతారు.

Interview దృష్టి

Q1: Program మరియు process మధ్య తేడా?

> Program = disk మీద నిల్వ ఉన్న passive instructions (file). Process = ఆ program యొక్క active execution instance — own memory (text/data/heap/stack), own registers, own state. ఒక program నుండి అనేక processes create చేయవచ్చు. Program నిర్జీవం, process సజీవం.

Q2: PCB అంటే ఏమిటి, ఎందుకు అవసరం?

> Process Control Block — OS ప్రతి process గురించి ఉంచే data structure. PID, state, program counter, CPU registers, memory info (page table), open files, scheduling priority ఉంటాయి. Context switch సమయంలో process యొక్క state ని save/restore చేయడానికి ఇది అవసరం — PCB లేకపోతే pause చేసిన process ని తిరిగి కొనసాగించలేం.

Q3: fork() ఎన్నిసార్లు return అవుతుంది?

> ఒకసారి పిలిస్తే రెండుసార్లు return అవుతుంది — ఒకటి parent లో (child యొక్క PID తో), ఒకటి child లో (0 తో). ఈ return value చూసి "నేను parent నా child నా" అని code నిర్ణయించుకుంటుంది. Error అయితే -1 return (child create కాలేదు).

Q4: Ready state మరియు Waiting state తేడా?

> Ready = process run అవ్వడానికి పూర్తిగా సిద్ధం, కేవలం CPU కోసం queue లో wait చేస్తోంది (CPU ఇస్తే వెంటనే run). Waiting/Blocked = process ఒక event (I/O completion, lock release) కోసం ఆగింది; ఇప్పుడు CPU ఇచ్చినా వాడలేదు. I/O పూర్తయ్యాక Waiting → Ready కి మారుతుంది.

Q5: Zombie మరియు Orphan process అంటే ఏమిటి?

> **Zombie** = child process పూర్తయ్యింది కానీ parent ఇంకా `wait()` చేసి దాని exit status చదవలేదు — PCB entry మిగిలిపోయి "zombie" గా ఉంటుంది. **Orphan** = parent ముందే చనిపోయి, child ఇంకా బతికి ఉంది; అప్పుడు దాన్ని `init` / `systemd` (PID 1) దత్తత తీసుకుని cleanup చేస్తుంది. Zombie లు ఎక్కువైతే PID table నిండిపోతుంది — production లో process leak గా కనిపిస్తుంది.

3. Threads (thread vs process, user vs kernel threads, multithreading, thread pools)

వివరణ

Thread = ఒక process లోపల ఒక **స్వతంత్ర execution path** (ఒక "పని దారం"). ఒక process కి కనీసం ఒక thread (main thread) ఉంటుంది; అంతకంటే ఎక్కువ ఉంటే — **multithreading.**

కీలక idea: **ఒక process లోని threads అన్నీ అదే memory (heap, code, data, open files) share చేస్తాయి;** కానీ ఒక్కో thread కి **own stack, own registers, own program counter** ఉంటాయి. అంటే అన్నీ ఒకే ఇంట్లో ఉంటాయి (shared heap), కానీ ఒక్కొక్కరికీ own desk (stack).

ఒక్క వాక్యంలో: **Process = ఇల్లు (own memory); Thread = ఆ ఇంట్లో పని చేసే మనిషి. ఒకే ఇంట్లో అనేక మనుషులు (threads) common వస్తువులు (heap) వాడతారు, కానీ ఒక్కొక్కరికీ own notebook (stack).**

ఎందుకు threads? ఒకే process లో అనేక పనులు *ఏకకాలంలో (parallel/concurrent)* చేయడానికి. ఉదా: ఒక video app లో — ఒక thread video decode చేస్తుంది, ఒకటి audio, ఒకటి UI. Process కంటే thread create/switch చేయడం **చాలా చౌక** (memory share చేస్తారు కాబట్టి).

Real-life Scenario

> **Process = restaurant kitchen; Threads = అందులో cooks.**
>
> ఒక kitchen (process) లో common పరికరాలు ఉన్నాయి — గ్యాస్, fridge, మసాలా shelf (ఇవి shared heap). ఇప్పుడు ఒక్క cook (single thread) బదులు నలుగురు cooks (threads) పెట్టావు:
>
> - నలుగురూ **అదే fridge, అదే మసాలా shelf** వాడతారు (shared memory — వేగం, కానీ ఒకేసారి ఇద్దరు అదే గిన్నె పట్టుకుంటే గొడవ = race condition!).
> - ఒక్కొక్కరికీ **own cutting board, own recipe card** (own stack + registers).
>
> నలుగురు cooks ఒకేసారి పని చేస్తే వంట వేగం (parallelism). కానీ shared fridge దగ్గర discipline (synchronization) లేకపోతే — ఒకరు తీసిన గిన్నె ఇంకొకరు వాడేస్తారు, గందరగోళం. **ఇదే multithreading యొక్క power మరియు danger.** పక్క kitchen (వేరే process) మీ fridge ముట్టుకోలేదు — isolation.

Thread vs Process — పూర్తి పోలిక

అంశం	Process	Thread
Memory	own separate memory space	process యొక్క memory **share** చేస్తుంది (heap, code, data)
సొంతం ఏమిటి	మొత్తం address space	own stack, registers, PC మాత్రమే
Create cost	ఎక్కువ (memory copy, PCB)	తక్కువ (memory share)
Context switch cost	ఎక్కువ (page table/TLB flush)	తక్కువ (అదే memory)
Communication	IPC అవసరం (pipe, socket, shared mem)	నేరుగా shared memory (సులభం కానీ ప్రమాదకరం)
Isolation / safety	ఎక్కువ (ఒకటి crash → మిగతావి safe)	తక్కువ (ఒక thread crash → మొత్తం process crash)
ఉదాహరణ	Chrome tab, `node worker.js`	ఒక process లోని parallel పనులు

Trade-off crux: Threads వేగం + easy sharing, కానీ safety తక్కువ (ఒక thread bug మొత్తం process ని కూల్చవచ్చు; shared memory వల్ల race conditions). Processes safe + isolated, కానీ heavy + communication కష్టం. **"Chrome ఎందుకు tab కి process, thread కాదు?"** → ఒక tab crash మిగతా tabs ని కూల్చకూడదు (isolation).

User-level vs Kernel-level Threads

Threads ని ఎవరు manage చేస్తారు అనేదాన్ని బట్టి రెండు రకాలు:

```
USER-LEVEL threads                KERNEL-LEVEL threads

┌──────────────────┐              ┌──────────────────┐
│  T1  T2  T3       │  user        │  T1  T2  T3       │
│   \  |  /         │  space       │   |   |   |       │
│  thread library   │              │   |   |   |       │
└────────┬─────────┘              └───┴───┴───┴──────┘
```

```
        │ (kernel కి 1 thread     │    │    │
        ▼  మాత్రమే కనిపిస్తుంది)    ▼    ▼    ▼
   ┌──────────────┐             ┌──────────────┐
   │ Kernel: 1 KT  │             │ Kernel: 3 KTs │
   └──────────────┘             └──────────────┘
```

అంశం	User-level threads	Kernel-level threads
ఎవరు manage	user-space library (kernel కి తెలియదు)	OS kernel directly
Switch వేగం	చాలా వేగం (kernel involve కాదు)	నెమ్మది (mode switch)
Blocking problem	ఒక thread blocking system call చేస్తే **మొత్తం process block** (kernel కి ఒక్కటే కనిపిస్తుంది)	ఒక thread block అయితే మిగతావి run అవుతాయి
Multi-core	ఒక process ఒక core మాత్రమే వాడగలదు (kernel కి 1 thread)	నిజమైన parallelism (అనేక cores)

Mapping models: Many-to-One (అన్ని user threads → 1 kernel thread; blocking problem), One-to-One (ప్రతి user thread → 1 kernel thread; నేటి Linux/Windows ఇదే), Many-to-Many (m user → n kernel; flexible కానీ complex).

Node.js — "single-threaded" నిజంగా ఏంటి? (నీకు అత్యంత ముఖ్యం)

ఇది తప్పకుండా అర్థం చేసుకో, interview లో guaranteed:

Node.js యొక్క JavaScript execution single-threaded. అంటే నీ JS code మొత్తం **ఒకే main thread** (event loop thread) మీద run అవుతుంది. ఒకేసారి ఒక్క JS callback మాత్రమే execute అవుతుంది. అందుకే నీ JS లో "lock" అవసరం లేదు (రెండు callbacks ఒకేసారి run కావు కాబట్టి race condition ఉండదు... JS variables మీద).

మరి Node వేలాది concurrent requests ఎలా handle చేస్తుంది single thread తో?

```
  నీ JS (single main thread)          libuv (C++, background)

  ┌─────────────────────┐            ┌─────────────────────────┐
  │   Event Loop          │          │  Thread Pool (default 4)│
  │  (JS callbacks         │  offload │   ┌──┐ ┌──┐ ┌──┐ ┌──┐    │
  │   ఇక్కడ run అవుతాయి)    │─────────►│   │T1│ │T2│ │T3│ │T4│    │
  │                       │ file I/O │   └──┘ └──┘ └──┘ └──┘    │
  │  fetch/db → non-       │ crypto   │  (blocking పనులు ఇక్కడ)  │
  │  blocking              │ DNS      └─────────────────────────┘
  └─────────────────────┘
     callback ready అయ్యాక తిరిగి event loop కి
```

- **Network I/O** (HTTP, DB queries, sockets) — OS యొక్క non-blocking mechanisms (epoll/kqueue) వాడతాయి; thread అవసరం లేదు. Node "ఈ socket ready అయ్యాక చెప్పు" అని OS కి చెప్పి, తను ఇంకో పని చూసుకుంటుంది.
- **File I/O, DNS, crypto (`bcrypt`), zlib** — వీటికి OS non-blocking API సరిగా లేదు, కాబట్టి libuv యొక్క **thread pool** (default **4 threads**, `UV_THREADPOOL_SIZE` తో మార్చవచ్చు) వాడతాయి. ఈ background threads లో పని జరిగి, పూర్తయ్యాక callback ని main thread కి తిరిగి ఇస్తాయి.

అంటే Node "single-threaded" అనేది JS execution వరకే. లోపల libuv కి threads ఉన్నాయి. నీ event loop thread ఒక్కటే నీ JS run చేస్తుంది, కానీ heavy I/O ని background threads / OS కి offload చేస్తుంది.

Gotcha — CPU-bound work: నీ JS లో `for` loop తో 5 seconds heavy computation (image processing, big JSON parse, password hashing sync గా) చేస్తే — main thread block అవుతుంది, ఆ 5 seconds లో వేరే ఏ request handle అవ్వదు (event loop stuck). Solution: `worker_threads` (నిజమైన OS threads, own V8 instance) లేదా `cluster` (multiple processes). **"Node CPU-heavy పనికి ఎందుకు మంచిది కాదు?"** — ఇదే సమాధానం.

Node tool	ఏంటి	ఎప్పుడు
Event loop (main thread)	నీ JS + non-blocking I/O	సాధారణ web requests (I/O-bound)
libuv thread pool	file/DNS/crypto background threads	Node internally వాడుతుంది
worker_threads	నిజమైన OS threads, message passing తో	CPU-heavy పని (parse, hash, image)
cluster / child_process	multiple Node **processes**	multi-core scaling, isolation

Thread Pool — ఎందుకు, ఎలా

ప్రతి task కి కొత్త thread create చేయడం ఖరీదు (create/destroy cost + వేలాది threads → memory + scheduling overhead). **Thread pool** = ముందే కొన్ని threads (ఉదా 4-8) create చేసి ఉంచుకుని, పని వచ్చినప్పుడు ఖాళీ thread కి ఇవ్వడం; పని అయ్యాక thread ని చంపకుండా pool కి తిరిగి ఇవ్వడం (reuse).

```
Tasks queue:  [t1][t2][t3][t4][t5][t6]...
                          │
                          ▼  ఖాళీ thread కి assign
Pool:  [T1: busy][T2: busy][T3: idle][T4: idle]
                                ▲
                         పని అయ్యాక reuse
```

లాభాలు: thread create/destroy cost లేదు, threads సంఖ్య control (unbounded threads → resource exhaustion ని ఆపుతుంది), latency తక్కువ. libuv thread pool, Java's `ExecutorService`, DB connection pools — అన్నీ ఇదే idea. **నీ MongoDB driver కూడా connection pool వాడుతుంది** (ప్రతి query కి కొత్త TCP connection తెరవదు — reuse).

Key Points

- **Thread = process లోపల ఒక execution path.** Threads **memory (heap/code/data) share** చేస్తాయి, కానీ **own stack + registers + PC.**
- **Thread vs Process:** thread = చౌక + easy sharing + తక్కువ safety; process = ఖరీదు + isolated + safe. Chrome tab = process (isolation కోసం).
- **User threads** = వేగం కానీ blocking system call మొత్తం process ని block చేస్తుంది; **kernel threads** = నిజ parallelism + independent blocking. నేటి systems One-to-One.
- **Node.js JS = single-threaded (event loop).** Network I/O = OS non-blocking (epoll); file/crypto/DNS = **libuv thread pool (default 4)**. CPU-heavy పని event loop ని block చేస్తుంది → `worker_threads` / `cluster` వాడు.

- **Thread pool = threads ని reuse** చేసి create cost + resource exhaustion ని తగ్గిస్తుంది. Connection pools కూడా ఇదే idea.

Interview దృష్టి

Q1: Thread మరియు process మధ్య ముఖ్య తేడా?

> Process కి own separate memory space; thread ఆ process యొక్క memory (heap, code, data, open files) share చేస్తుంది కానీ own stack/registers/PC ఉంటాయి. Thread create/switch చౌక (memory share), కానీ ఒక thread crash మొత్తం process ని కూల్చవచ్చు. Processes isolated కాబట్టి safe కానీ heavy, communication కి IPC అవసరం.

Q2: Node.js single-threaded అయితే వేలాది requests ఎలా handle చేస్తుంది?

> JS execution మాత్రమే single-threaded (event loop). Network I/O ని OS non-blocking mechanisms (epoll/kqueue) కి, file/crypto/DNS ని libuv thread pool (default 4 threads) కి offload చేస్తుంది. Main thread ఏది కోసం block అవ్వకుండా, callbacks ready అయ్యేకొద్దీ వాటిని process చేస్తుంది. అందుకే I/O-heavy workloads కి చాలా efficient.

Q3: Node.js లో CPU-heavy computation ఎందుకు problem, పరిష్కారం?

> ఇది event loop (single JS thread) ని block చేస్తుంది — ఆ సమయంలో వేరే callbacks/requests handle అవ్వవు, latency పెరుగుతుంది. పరిష్కారం: `worker_threads` (నిజమైన OS threads, own V8 heap, message passing) లేదా `cluster` / `child_process` (multiple processes, multi-core).

Q4: User-level vs kernel-level threads?

> User-level = thread library manage చేస్తుంది, kernel కి తెలియదు; switch వేగం కానీ ఒక thread యొక్క blocking system call మొత్తం process ని block చేస్తుంది + multi-core వాడలేదు. Kernel-level = OS manage చేస్తుంది; switch నెమ్మది కానీ నిజ parallelism + independent blocking. నేటి Linux/Windows One-to-One mapping వాడతాయి.

Q5: Thread pool ఎందుకు వాడతారు?

> ప్రతి task కి thread create/destroy చేయడం ఖరీదు, మరియు unbounded threads resource exhaustion కి దారితీస్తాయి. Thread pool ముందే fixed threads create చేసి reuse చేస్తుంది — create cost తగ్గుతుంది, concurrency bounded గా ఉంటుంది, latency మెరుగవుతుంది. DB connection pool కూడా ఇదే principle.

4. Context Switching (mechanism, cost)

వివరణ

CPU core ఒకేసారి **ఒక్క** thread/process ని మాత్రమే run చేయగలదు. అయినా నీ laptop లో 300 processes "ఏకకాలంలో" నడుస్తున్నట్టు కనిపిస్తాయి — ఎలా? OS వాటి మధ్య **చాలా వేగంగా turn-by-turn switch** చేస్తుంది (సెకనుకు వందల/వేల సార్లు). కంటికి అది "అన్నీ ఒకేసారి" అనిపిస్తుంది. ఈ switch నే **context switch** అంటారు.

Context switch = CPU ని ఒక process/thread నుండి తీసేసి ఇంకోదానికి ఇచ్చే process. దీనికి — ప్రస్తుత process యొక్క **state (context)** ని PCB లో **save** చేసి, తర్వాతి process యొక్క state ని PCB నుండి **load** చేయాలి. "Context" అంటే — CPU registers, program counter, stack pointer, memory mapping info — అంటే process ఎక్కడ ఆగిందో గుర్తుపెట్టే మొత్తం snapshot.

ఒక్క వాక్యంలో: **Context switch = "ఒక పని ఎక్కడ ఆపావో bookmark పెట్టి, ఇంకో పని bookmark దగ్గర నుండి కొనసాగించడం."**

Real-life Scenario

> **Context switch = నువ్వు రెండు books ఒకేసారి చదవడం.**
> నీ దగ్గర Book A, Book B ఉన్నాయి, కానీ రెండు చేతులు ఒకేసారి రెండూ చదవలేవు (single CPU core). నువ్వు Book A చదువుతూ page 50 దగ్గర ఆగావు — ఇప్పుడు Book B చదవాలి. ఏం చేస్తావు?
>
> 1. Book A లో page 50 దగ్గర **bookmark పెడతావు** ("నేను ఇక్కడ ఆగాను" — state save చేయడం).
> 2. Book B తీసి, దాని bookmark (page 80) దగ్గర తెరుస్తావు (state load).
> 3. కొంతసేపు Book B చదివి, మళ్ళీ Book A కి switch — దాని bookmark దగ్గర కొనసాగిస్తావు.
>
> ఇక్కడ **bookmark పెట్టడం/తీయడం లోనే కొంత సమయం** పోతుంది — ఆ సమయంలో నువ్వు actual reading (useful work) చేయట్లేదు. **అదే context switch overhead.** ఎక్కువసార్లు switch చేస్తే — bookmark పెట్టడంలోనే time అయిపోయి, actual చదవడం తగ్గుతుంది. అందుకే "too many switches = waste."

Context Switch లోపల ఏం జరుగుతుంది (mechanism)

```
   Process P1 running                       Process P2 runs
   ─────────────────                        ─────────────────
         │
         │   ◄── trigger (interrupt / syscall / time-slice over)
         ▼
    ┌───────────────────────────┐
    │ 1. P1 యొక్క state save     │  registers, PC, SP → P1 యొక్క PCB
    ├───────────────────────────┤
    │ 2. P1 ని Ready/Waiting కి  │  state update
    │    మార్పు                   │
    ├───────────────────────────┤
    │ 3. Scheduler ఎంపిక         │  తర్వాత ఎవరు? → P2
    ├───────────────────────────┤
    │ 4. P2 యొక్క state load     │  P2 యొక్క PCB → registers, PC, SP
    ├───────────────────────────┤
```

```
│ 5. Memory context switch  │  page table pointer మార్పు,
│    (process అయితే)          │  TLB flush
└───────────────────────────┘
      │
      ▼
P2 తను ఆగిన చోటు నుండి కొనసాగుతుంది
```

ఈ మొత్తం సమయంలో CPU useful work చేయట్లేదు — కేవలం "పని మార్పిడి." అందుకే context switch ఒక **pure overhead** (అవసరం, కానీ productive కాదు).

Context switch ఎప్పుడు జరుగుతుంది?

1. **Time-slice (quantum) అయిపోయినప్పుడు** — timer interrupt వచ్చి "ఈ process చాలు, తర్వాతివాడికి turn" అంటుంది (preemptive scheduling).
2. **I/O కోసం block అయినప్పుడు** — process `read()` /DB query చేసి wait లోకి వెళ్తే, CPU ఖాళీ; వేరే process కి ఇస్తారు.
3. **Higher-priority process వచ్చినప్పుడు** — ముఖ్యమైన పని వస్తే ప్రస్తుతదాన్ని preempt చేస్తారు.
4. **Interrupt వచ్చినప్పుడు** — hardware interrupt (keyboard, network packet) handle చేయడానికి.
5. **System call / voluntary yield** — process తనే CPU వదిలేస్తే.

Cost — ఎందుకు ఖరీదు

Context switch **overhead** రెండు రకాలు:

Cost రకం	ఏంటి
Direct cost	state save/load, scheduler run, memory mapping మార్పు — ఇవి direct CPU cycles (సాధారణంగా కొన్ని microseconds)
Indirect cost (దాచి ఉన్నది, ఎక్కువ బాధ)	**Cache & TLB pollution** — P1 వాడిన CPU cache/TLB entries P2 కి పనికిరావు; P2 మళ్ళీ cold cache నుండి మొదలవుతుంది → cache misses పెరిగి నెమ్మది

Thread switch vs Process switch: ఒకే process లోని threads switch అయితే — memory (page table) అదే, కాబట్టి **TLB flush అవసరం లేదు** → చౌక. వేరే process కి switch అయితే — page table మారుతుంది, **TLB flush** అవుతుంది → ఖరీదు. అందుకే threads "lightweight."

Trade-off — quantum size: Time-slice చిన్నదైతే → వేగంగా switch → responsive కానీ ఎక్కువ overhead (bookmark పెట్టడంలోనే time). పెద్దదైతే → తక్కువ overhead కానీ తక్కువ responsive (ఒక process ఎక్కువసేపు పట్టుకుంటుంది). OS ఈ balance ని quantum tuning తో చేస్తుంది (సాధారణంగా 10-100ms).

Node connection: Node event loop ఒకే thread లో context switch లేకుండా వేలాది I/O operations handle చేస్తుంది — ఇది thread-per-request model (ప్రతి request కి thread, ఎక్కువ context switches) కంటే ఒక పెద్ద లాభం. "ఎందుకు Node high-concurrency కి మంచిది?" — thread-per-connection model యొక్క context-switch + memory overhead ని తప్పించడం ఒక కారణం.

Key Points

- **Context switch = CPU ని ఒక process/thread నుండి ఇంకోదానికి మార్చడం** — పాత state ని PCB లో save, కొత్త state ని load.
- ఇది **pure overhead** — ఆ సమయంలో useful work జరగదు. అందుకే too many switches = performance drop.
- **ఎప్పుడు:** time-slice ముగింపు, I/O block, higher-priority process, interrupts, voluntary yield.
- **Cost:** direct (save/load) + indirect (**cache/TLB pollution** — ఇది ఎక్కువ బాధ).
- **Thread switch < Process switch cost** (అదే process → TLB flush అవసరం లేదు). అందుకే threads lightweight.
- Quantum చిన్నది → responsive కానీ overhead ఎక్కువ; పెద్దది → సర్దుకుంది కానీ తక్కువ responsive.

Interview దృష్టి

Q1: Context switch అంటే ఏమిటి, ఎందుకు overhead?

> ఒక process/thread నుండి ఇంకోదానికి CPU ని మార్చే process — పాత process యొక్క state (registers, PC, SP) ని PCB లో save చేసి, కొత్తదాని state ని load చేస్తుంది. ఈ సమయంలో CPU useful work చేయదు (pure overhead). పైగా cache/TLB pollution వల్ల కొత్త process cold cache నుండి మొదలై నెమ్మదిస్తుంది — ఇది indirect cost.

Q2: Thread switch process switch కంటే ఎందుకు చౌక?

> ఒకే process లోని threads అదే memory (page table) share చేస్తాయి, కాబట్టి switch సమయంలో page table మార్చాల్సిన అవసరం లేదు, **TLB flush అవదు.** వేరే process కి switch అయితే page table మారి TLB flush అవుతుంది, cache కూడా cold అవుతుంది — అందుకే process switch ఖరీదు.

Q3: Context switch ఎప్పుడు trigger అవుతుంది?

> Time-slice/quantum అయిపోయినప్పుడు (timer interrupt), process I/O కోసం block అయినప్పుడు, higher-priority process ready అయినప్పుడు (preemption), hardware interrupt వచ్చినప్పుడు, లేదా process voluntarily CPU వదిలినప్పుడు.

Q4: Quantum (time-slice) చిన్నగా పెడితే ఏమవుతుంది?

> System responsive అవుతుంది (ప్రతి process త్వరగా turn పొందుతుంది), కానీ context switches సంఖ్య పెరిగి overhead ఎక్కువవుతుంది — useful work తగ్గుతుంది. చాలా పెద్దగా పెడితే overhead తగ్గుతుంది కానీ responsiveness తగ్గుతుంది (ఒక process ఎక్కువసేపు CPU పట్టుకుంటుంది). OS ఈ trade-off ని balance చేసి quantum ఎంచుకుంటుంది.

Part 2 — Scheduling (CPU ని ఎవరికి ఇవ్వాలి)

> ఒక CPU core, కానీ run అవ్వడానికి ready గా ఉన్న processes చాలా. మరి తర్వాత ఎవరికి CPU ఇవ్వాలి? ఈ నిర్ణయమే **CPU scheduling.** ఇది OS యొక్క గుండె — ఇక్కడ తీసుకునే నిర్ణయమే system fast గా అనిపిస్తుందా, responsive గా ఉంటుందా అనేది నిర్ణయిస్తుంది. ఈ Part లో scheduling ఎలా measure చేయాలి (criteria), preemptive vs non-preemptive తేడా, మరియు 6 classic algorithms — ప్రతిదానికి Gantt chart + waiting/turnaround time లెక్క — నేర్చుకుంటాం.

---

5. CPU Scheduling Basics (criteria, preemptive vs non-preemptive)

వివరణ

CPU scheduling = ready queue లో ఉన్న processes లో **తర్వాత ఎవరికి CPU ఇవ్వాలి** అని OS (scheduler) తీసుకునే నిర్ణయం. దీన్ని చేసే OS భాగం **scheduler**, actual CPU handover చేసేది **dispatcher.**

ఎందుకు ముఖ్యం? ఒకే CPU core ని అనేక processes పంచుకుంటున్నాయి. ఏ order లో run చేస్తే — average లో అందరూ త్వరగా పూర్తవుతారు? System responsive గా ఉంటుంది? CPU ఖాళీగా (idle) ఉండదు? ఈ నిర్ణయం తప్పైతే — CPU ఉన్నా system slow గా అనిపిస్తుంది.

ముందు కొన్ని terms:

- **Burst time (BT)** — process కి CPU మీద ఎంత సమయం అవసరం (ఒక "CPU burst" length).
- **Arrival time (AT)** — process ready queue కి ఎప్పుడు వచ్చింది.
- **Completion time (CT)** — process ఎప్పుడు పూర్తయ్యింది.
- **CPU burst vs I/O burst** — processes CPU వాడటం (compute) మరియు I/O కోసం wait చేయడం మధ్య మారుతూ ఉంటాయి. **CPU-bound** = ఎక్కువ compute (video encode); **I/O-bound** = ఎక్కువ wait (నీ Node server — DB, network). Scheduler ఈ mix ని బట్టి decisions తీసుకుంటుంది.

Real-life Scenario

> **CPU scheduling = ఒక్క doctor ఉన్న clinic లో patients queue.**
>
> ఒక్క doctor (CPU), waiting room లో చాలామంది patients (processes). Receptionist (scheduler) తర్వాత ఎవరిని లోపలికి పంపాలో నిర్ణయిస్తుంది. వేర్వేరు policies:
>
> - **వచ్చిన order లో** (FCFS) — fair, కానీ ముందు వచ్చిన ఒక పెద్ద case (2 గంటలు) వెనుక ఉన్న 5 చిన్న cases (5 నిమిషాలు each) ని ఆపేస్తుంది.
> - **చిన్న cases ముందు** (SJF) — average waiting తగ్గుతుంది, కానీ పెద్ద case ఎప్పటికీ turn రాకపోవచ్చు (starvation).
> - **అందరికీ 10 నిమిషాల turn, తర్వాత మళ్ళీ queue** (Round Robin) — అందరూ కొంచెం కొంచెం చూడబడతారు, fair + responsive.
>
> **ఏ policy "best"?** అది ఏం optimize చేయాలనుకుంటున్నావో దాన్ని బట్టి — average wait తగ్గించాలా, అందరూ త్వరగా response పొందాలా, doctor ఖాళీగా ఉండకూడదా? **ఒకే "best" లేదు — trade-offs మాత్రమే.** ఇదే scheduling యొక్క అసలు పాఠం.

Scheduling Criteria — దేన్ని optimize చేయాలి

ఒక scheduling algorithm ఎంత మంచిదో measure చేసే metrics. కొన్నిటిని maximize, కొన్నిటిని minimize చేయాలి:

Criterion	అర్థం	దిశ	Formula
CPU Utilization	CPU ఎంత % busy గా ఉంది (idle కాకుండా)	**maximize**	busy time / total time
Throughput	unit time కి ఎన్ని processes పూర్తవుతున్నాయి	**maximize**	completed / time
Turnaround Time (TAT)	process submit నుండి పూర్తయ్యేదాకా మొత్తం సమయం	**minimize**	`CT − AT`
Waiting Time (WT)	ready queue లో ఎంతసేపు ఖాళీగా wait చేసింది	**minimize**	`TAT − BT`
Response Time (RT)	submit నుండి **మొదటిసారి** CPU పొందేదాకా సమయం	**minimize**	`first CPU time − AT`

రెండు కీలక formulas (గుర్తుంచుకో):

```
Turnaround Time (TAT) = Completion Time − Arrival Time
Waiting Time (WT)     = Turnaround Time − Burst Time
                      = TAT − BT
```

Response vs Turnaround తేడా (interview favorite): Response time = *మొదటి* response ఎప్పుడు (interactive systems కి ముఖ్యం — నీ click కి screen ఎప్పుడు స్పందిస్తుంది). Turnaround = *పూర్తిగా* అయ్యేదాకా (batch jobs కి ముఖ్యం). ఒక process త్వరగా మొదటి response ఇచ్చి, తర్వాత ఎక్కువసేపు run అవ్వవచ్చు — అప్పుడు RT తక్కువ, TAT ఎక్కువ.

Preemptive vs Non-preemptive

ఇది scheduling లో అతి ముఖ్యమైన విభజన:

అంశం	Non-preemptive	Preemptive
నిర్వచనం	ఒకసారి CPU ఇస్తే, process **తనే వదిలేదాకా** (పూర్తవ్వడం / I/O కి wait) CPU తీసుకోరు	OS ఏ క్షణంలోనైనా CPU ని **బలవంతంగా తీసుకుని** ఇంకోదానికి ఇవ్వగలదు
CPU ఎప్పుడు మారుతుంది	process voluntarily వదిలినప్పుడు మాత్రమే	time-slice, higher-priority arrival, interrupt — ఎప్పుడైనా
Context switches	తక్కువ	ఎక్కువ (overhead)
Responsiveness	తక్కువ (ఒక పెద్ద process అందరినీ ఆపగలదు)	ఎక్కువ (fair, quick response)
Starvation risk	తక్కువ (అందరూ turn పూర్తిగా పొందుతారు)	ఎక్కువ కావచ్చు (priority-based లో)
ఉదాహరణ algorithms	FCFS, SJF, non-preemptive Priority	Round Robin, SRTF, preemptive Priority, MLFQ
ఎక్కడ	పాత batch systems	నేటి desktops/servers (Linux, Windows)

Convoy effect (non-preemptive యొక్క పెద్ద problem): ఒక పెద్ద CPU-bound process ముందు ఉంటే, వెనుక ఉన్న అనేక చిన్న processes అన్నీ దాని వెనుక ఆగిపోతాయి — ఒక ట్రక్ వెనుక bikes ఆగినట్టు. దీంతో average waiting time పెరిగి, throughput పడుతుంది. Preemption ఇది తప్పిస్తుంది.

నేటి OS లు preemptive — ఎందుకంటే ఒక్క program మొత్తం system ని hang చేయకూడదు (నీ browser stuck అయినా OS UI స్పందించాలి). Timer interrupt వచ్చి scheduler ని క్రమం తప్పకుండా run చేస్తుంది.

Dispatcher మరియు Dispatch Latency

Dispatcher = scheduler ఎంచుకున్న process కి actually CPU ఇచ్చే module. దీని పనులు: context switch చేయడం, user mode కి switch చేయడం, program యొక్క సరైన instruction కి jump చేయడం. **Dispatch latency** = ఒక process ని ఆపి ఇంకోదాన్ని start చేయడానికి పట్టే సమయం — ఇది తక్కువగా ఉండాలి.

Key Points

- **Scheduler = తర్వాత ఎవరికి CPU అని నిర్ణయిస్తుంది; dispatcher = actual handover చేస్తుంది.**
- **గుర్తుంచుకోవాల్సిన formulas:** `TAT = CT − AT`, `WT = TAT − BT`.
- **5 criteria:** CPU utilization ↑, throughput ↑, turnaround ↓, waiting ↓, response ↓.
- **Response time = మొదటి response** (interactive కి); **Turnaround = పూర్తిగా అయ్యేదాకా** (batch కి).
- **Non-preemptive** = process తనే వదిలేదాకా CPU తీసుకోరు (convoy effect ప్రమాదం); **Preemptive** = OS ఏ క్షణంలోనైనా తీసుకోగలదు (responsive కానీ ఎక్కువ switches). నేటి OS లు preemptive.
- **CPU-bound** (compute-heavy) vs **I/O-bound** (wait-heavy, నీ Node server) — scheduler ఈ mix ని పరిగణిస్తుంది.

Interview దృష్టి

Q1: Turnaround time మరియు waiting time formulas?

> Turnaround Time = Completion Time − Arrival Time (submit నుండి పూర్తయ్యేదాకా మొత్తం సమయం). Waiting Time = Turnaround Time − Burst Time (అంటే ready queue లో CPU కోసం ఖాళీగా ఎంతసేపు ఆగింది). ఈ రెండూ minimize చేయాలి.

Q2: Preemptive మరియు non-preemptive scheduling తేడా?

> Non-preemptive లో process ఒకసారి CPU పొందితే, తనే వదిలేదాకా (పూర్తవ్వడం లేదా I/O block) CPU తీసుకోరు — తక్కువ context switches కానీ convoy effect. Preemptive లో OS timer interrupt / higher-priority arrival ద్వారా CPU ని ఏ క్షణంలోనైనా బలవంతంగా తీసుకోగలదు — ఎక్కువ responsive కానీ ఎక్కువ overhead. నేటి OS లు preemptive.

Q3: Response time మరియు turnaround time తేడా, ఏది ఎప్పుడు ముఖ్యం?

> Response time = submit నుండి process మొదటిసారి CPU పొందేదాకా (మొదటి స్పందన). Turnaround = submit నుండి పూర్తిగా అయ్యేదాకా. Interactive/UI systems కి response time ముఖ్యం (user త్వరగా feedback పొందాలి); batch/background jobs కి turnaround ముఖ్యం (పూర్తవ్వడం మాత్రమే లక్ష్యం).

Q4: Convoy effect అంటే ఏమిటి?

> Non-preemptive scheduling (ముఖ్యంగా FCFS) లో ఒక పెద్ద CPU-bound process ముందు ఉంటే, వెనుక ఉన్న అనేక చిన్న processes అన్నీ దాని వెనుక ఆగిపోతాయి — average waiting time పెరిగి throughput పడుతుంది. ఒక ట్రక్ వెనుక bikes ఆగినట్టు. Preemption లేదా చిన్నవి ముందు run చేయడం (SJF) దీన్ని తగ్గిస్తుంది.

6. Scheduling Algorithms (FCFS, SJF, SRTF, Priority, Round Robin, MLFQ)

వివరణ

ఇప్పుడు అసలు algorithms. ప్రతిదానికి — idea, ఒక worked Gantt chart, waiting/turnaround లెక్క, మరియు trade-off. **Gantt chart** = time line మీద ఏ process ఎప్పుడు run అయ్యిందో చూపే bar diagram. ఇవి interview లో handwritten గా వేయమంటారు — practice చెయ్యి.

గుర్తుంచుకో: **TAT = CT − AT**, **WT = TAT − BT**.

Real-life Scenario

> **Scheduling algorithms = supermarket లో billing counter policies.**
>
> ఒక్క billing counter (CPU), trolleys తో customers (processes) queue లో. ఏ policy తో pilustaru?
>
> - **FCFS** = వచ్చిన వరుసలో. ముందు 100 items ఉన్న family వెనుక, 2 items ఉన్నవాళ్ళు అందరూ ఆగుతారు (convoy).
> - **SJF/SRTF** = "10 items or less" express counter — చిన్నవాళ్ళు ముందు, average wait తగ్గుతుంది, కానీ పెద్ద trolley వాడు ఎప్పటికీ turn రాకపోవచ్చు (starvation).
> - **Priority** = VIP/pregnant/senior citizen ముందు — ముఖ్యమైనవాళ్ళు ముందు, కానీ సాధారణ వాళ్ళు starve అవ్వచ్చు (aging తో fix).
> - **Round Robin** = ప్రతి customer కి "5 items scan, తర్వాత queue చివరికి" — అందరూ కొద్దికొద్దిగా serve అవుతారు (fair, quick response).
> - **MLFQ** = smart cashier — చిన్నవాళ్ళని/express ని గమనించి fast lane కి, పెద్ద trolleys ని slow lane కి తనంతట తను సర్దుతాడు.
>
> **ఏదీ absolute best కాదు** — average wait తగ్గించాలా, అందరూ quick response పొందాలా, ముఖ్యమైనవి ముందా — దేన్ని optimize చేయాలో దాన్ని బట్టి policy. Trade-offs మాత్రమే.

6.1 FCFS — First Come First Serve

Idea: వచ్చిన order లో run చెయ్యి (queue లాగా). Non-preemptive. అత్యంత simple, fair.

ఉదాహరణ (అందరూ time 0 కి వచ్చారు అనుకో):

Process	Burst Time
P1	24
P2	3
P3	3

Gantt chart:

```
| P1                 | P2    | P3    |
0                   24      27      30
```

Process	CT	TAT (CT−AT)	WT (TAT−BT)
P1	24	24	0
P2	27	27	24
P3	30	30	27

Average WT = (0+24+27)/3 = 17. పెద్ద P1 ముందు రావడం వల్ల P2, P3 చాలాసేపు wait చేశాయి — **convoy effect.** ఒకవేళ order P2, P3, P1 అయితే average WT = (0+3+6)/3 = 3 మాత్రమే! **అదే processes, వేరే order → 17 vs 3.** ఇదే FCFS బలహీనత.

- **లాభం:** simple, fair, starvation లేదు.
- **నష్టం:** convoy effect, average waiting time ఎక్కువ, interactive systems కి పనికిరాదు.

6.2 SJF — Shortest Job First (non-preemptive)

Idea: ready ఉన్న వాటిలో **అతి చిన్న burst time** ఉన్నదాన్ని ముందు run చెయ్యి. Non-preemptive.

అదే 3 processes (P1=24, P2=3, P3=3, అందరూ t=0): చిన్నవి ముందు → P2, P3, P1.

Gantt chart:

```
| P2  | P3  | P1                  |
0     3     6                     30
```

Process	CT	TAT	WT
P2	3	3	0
P3	6	6	3
P1	30	30	6

Average WT = (6+0+3)/3 = 3. FCFS లో 17 → SJF లో 3! **SJF average waiting time ని minimize చేస్తుంది — ఇది provably optimal** (అన్ని processes ఒకేసారి ఉంటే).

- **లాభం:** minimum average waiting time (గణితంగా optimal).
- **నష్టం: burst time ముందే తెలియాలి** (practical గా impossible — అంచనా వేయాలి); పెద్ద jobs **starvation** (చిన్నవి వస్తూనే ఉంటే పెద్దది ఎప్పటికీ turn రాదు).

> **Burst time ఎలా అంచనా?** గత bursts నుండి **exponential averaging**: `τ(n+1) = α·t(n) + (1−α)·τ(n)`. ఇది real OS లు "guessing" చేసే విధానం.

6.3 SRTF — Shortest Remaining Time First (preemptive SJF)

Idea: SJF యొక్క preemptive version. ఏ క్షణంలోనైనా, **అతి తక్కువ remaining burst** ఉన్నదాన్ని run చెయ్యి. కొత్త process వచ్చి దాని burst < ప్రస్తుత remaining అయితే — preempt.

ఉదాహరణ (వేర్వేరు arrival times):

Process	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Trace: t=0 P1 start. t=1 P2(4) < P1 remaining(7) → P2 run. t=2 P3(9) > P2 rem(3) → P2 continue. t=3 P4(5) > P2 rem(2) → P2 continue. t=5 P2 done; ready: P1(7), P3(9), P4(5) → P4. t=10 P4 done; P1(7) < P3(9) → P1. t=17 P1 done; P3.

Gantt chart:

```
| P1 | P2        | P4        | P1            | P3            |
0    1           5           10              17              26
```

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Average WT = (9+0+15+2)/4 = 6.5. SRTF minimum average waiting ఇస్తుంది, కానీ ఎక్కువ context switches + starvation (పెద్ద P3 చివర్లో).

6.4 Priority Scheduling

Idea: ప్రతి process కి priority number. **అత్యధిక priority** ముందు (సాధారణంగా చిన్న number = high priority). Preemptive లేదా non-preemptive రెండూ ఉంటాయి.

ఉదాహరణ (అందరూ t=0; చిన్న number = high priority):

Process	BT	Priority
P1	10	3
P2	1	1
P3	2	4
P4	5	2

Order (priority ప్రకారం): P2(1), P4(2), P1(3), P3(4).

Gantt chart:

```
| P2 | P4      | P1                | P3   |
0    1         6                  16     18
```

Process	CT	TAT	WT
P2	1	1	0
P4	6	6	1
P1	16	16	6
P3	18	18	16

Average WT = (6+0+16+1)/4 = 5.75.

- **నష్టం: Starvation** — తక్కువ priority process ఎప్పటికీ turn రాకపోవచ్చు (high-priority వస్తూనే ఉంటే).
- **పరిష్కారం — Aging:** ఎక్కువసేపు wait చేసిన process యొక్క priority ని క్రమంగా పెంచడం. దీంతో ఏదో ఒకరోజు అది high priority అయి turn పొందుతుంది. **Starvation కి standard సమాధానం = aging.**

6.5 Round Robin (RR) — నేటి interactive systems గుండె

Idea: ప్రతి process కి fixed **time quantum** (ఉదా 4 units) ఇవ్వు. Quantum అయిపోతే — preempt చేసి ready queue **చివరికి** పంపు, తర్వాతివాడికి turn. Preemptive, circular.

ఉదాహరణ (అందరూ t=0, quantum = 4):

Process	BT
P1	5
P2	4
P3	2

Trace: P1 runs 4 (rem 1) → P2 runs 4 (done) → P3 runs 2 (done) → P1 runs 1 (done).

Gantt chart:

```
| P1    | P2    | P3  | P1|
0       4       8     10  11
```

Process	BT	CT	TAT	WT
P1	5	11	11	6
P2	4	8	8	4
P3	2	10	10	8

Average WT = (6+4+8)/3 = 6.

- **లాభం: fair** (అందరికీ turn), **మంచి response time** (interactive systems కి ideal), starvation లేదు.
- **Quantum tuning (కీలకం):**
  - Quantum **చాలా పెద్దది** → FCFS లా అవుతుంది (ప్రతి process ఒకేసారి పూర్తవుతుంది).
  - Quantum **చాలా చిన్నది** → ఎక్కువ context switches → overhead ఎక్కువ.
  - Rule of thumb: quantum ≈ 80% CPU bursts పూర్తయ్యేంత (సాధారణంగా 10-100ms).

Node/web connection: RR ఆలోచన load balancer లో కనిపిస్తుంది — requests ని servers కి round-robin గా పంచడం. అలాగే OS RR వల్లే నీ terminal లో heavy `npm build` నడుస్తున్నా, editor responsive గా ఉంటుంది.

6.6 MLFQ — Multi-Level Feedback Queue (అత్యంత practical)

Idea: అనేక queues, ఒక్కోదానికి వేరే priority + వేరే quantum. Process యొక్క **behavior ని బట్టి** queues మధ్య పైకి/కిందికి కదులుతుంది (feedback). నిజ OS (Windows, macOS, పాత Linux) దీన్ని పోలినవి వాడతాయి.

```
Q0 (high priority, quantum=8) ─── interactive, చిన్న bursts
     │ quantum వాడేస్తే కిందికి ↓
Q1 (medium, quantum=16)      ─── medium jobs
     │ quantum వాడేస్తే కిందికి ↓
Q2 (low priority, FCFS)      ─── CPU-heavy, long jobs
```

నియమాలు:

1. కొత్త process అత్యధిక priority queue (Q0) లో మొదలవుతుంది.
2. ఒక process తన quantum పూర్తిగా వాడేస్తే (CPU-bound) → **కిందికి demote** (తక్కువ priority).
3. Quantum అయ్యేలోపు I/O కి yield చేస్తే (interactive/I/O-bound) → అదే level లో ఉంటుంది (reward).
4. **Aging:** అప్పుడప్పుడు అన్నిటినీ Q0 కి తిరిగి తెస్తారు (starvation నివారణ).

అద్భుతం ఏంటంటే: MLFQ **burst time ముందే తెలియకుండానే** SJF లాంటి behavior సాధిస్తుంది — process ని గమనించి, చిన్నవి/interactive వాటికి high priority ఇస్తుంది, పెద్దవి కిందికి పంపుతుంది. అంటే "learning by watching."

- **లాభం:** interactive jobs కి quick response + CPU-heavy jobs కి fair share; burst time అంచనా అవసరం లేదు.
- **నష్టం:** tuning complex (ఎన్ని queues, quantums, aging rate); gaming (process ఉద్దేశపూర్వకంగా I/O చేసి high priority నిలుపుకోవడం).

అన్నిటి పోలిక — ఒక్క table

Algorithm	Preemptive?	ఎంపిక ప్రాతిపదిక	ప్రధాన లాభం	ప్రధాన సమస్య
FCFS	No	arrival order	simple, fair	convoy effect, high avg WT
SJF	No	shortest burst	min avg WT (optimal)	burst తెలియాలి, starvation
SRTF	Yes	shortest remaining	min avg WT	ఎక్కువ switches, starvation
Priority	రెండూ	priority number	ముఖ్యమైనవి ముందు	starvation → aging
Round Robin	Yes	fixed quantum, circular	fair, good response	quantum tuning
MLFQ	Yes	behavior-based feedback	SJF లా, తెలియకుండా	tuning complex

Key Points

- **FCFS** = simple కానీ convoy effect. **SJF/SRTF** = min avg waiting (optimal) కానీ burst తెలియాలి + starvation. **Priority** = starvation → **aging** పరిష్కారం.
- **Round Robin** = fair + best response (interactive systems); **quantum** tuning కీలకం (పెద్దది→FCFS, చిన్నది→overhead).
- **MLFQ** = నిజ systems వాడేది; behavior గమనించి SJF-లాంటి effect ని burst తెలియకుండానే సాధిస్తుంది.
- **Starvation కి universal పరిష్కారం = aging.**
- Gantt chart + `TAT = CT−AT`, `WT = TAT−BT` — interview లో handwritten calculation practice చెయ్యి.

Interview దృష్టి

Q1: ఏ scheduling algorithm minimum average waiting time ఇస్తుంది, ఎందుకు practical కాదు?

> SJF (non-preemptive) / SRTF (preemptive) — ఇవి provably minimum average waiting time ఇస్తాయి. కానీ practical కాదు ఎందుకంటే process యొక్క burst time ముందే తెలియదు; దాన్ని past behavior నుండి exponential averaging తో అంచనా వేయాలి. పైగా long jobs starvation కి గురవుతాయి.

Q2: Round Robin లో quantum చాలా చిన్నగా / పెద్దగా పెడితే?

> చాలా చిన్నది → ప్రతి quantum తర్వాత context switch, overhead ఎక్కువ, useful work తగ్గుతుంది. చాలా పెద్దది → RR essentially FCFS అవుతుంది (ప్రతి process ఒకేసారి పూర్తవుతుంది), response time పెరుగుతుంది. సరైన quantum: చాలావరకు CPU bursts (~80%) quantum లోపల పూర్తయ్యేంత.

Q3: Starvation అంటే ఏమిటి, ఎలా పరిష్కరిస్తారు?

> కొన్ని processes (ఉదా low-priority లేదా long jobs) ఎప్పటికీ CPU పొందకపోవడం — ఎందుకంటే higher-priority/shorter jobs వస్తూనే ఉంటాయి. పరిష్కారం **aging**: ఎక్కువసేపు wait చేసిన process యొక్క priority ని క్రమంగా పెంచడం, తద్వారా చివరికి అది turn పొందుతుంది.

Q4: నిజమైన OS ఏ algorithm వాడుతుంది?

> చాలావరకు MLFQ (Multi-Level Feedback Queue) యొక్క variants — interactive/I/O-bound jobs కి high priority + quick response, CPU-bound jobs కి కిందికి demote చేసి fair share. ఇది burst time ముందే తెలియకుండానే process behavior గమనించి SJF-లాంటి ఫలితం సాధిస్తుంది; aging తో starvation నివారిస్తుంది. (Linux నేడు CFS — Completely Fair Scheduler — red-black tree ఆధారితం.)

Q5: FCFS లో convoy effect ఎలా వస్తుంది?

> ఒక పెద్ద CPU-bound process ముందు వస్తే, non-preemptive FCFS దాన్ని పూర్తయ్యేదాకా run చేస్తుంది; వెనుక ఉన్న చిన్న processes అన్నీ ఆగిపోయి average waiting time విపరీతంగా పెరుగుతుంది. ఉదా P1=24, P2=3, P3=3 తో avg WT=17; order మార్చితే 3. Preemption లేదా SJF దీన్ని తగ్గిస్తుంది.

Part 3 — Concurrency (ఒకేసారి చాలా పనులు — safe గా)

> Threads/processes ఒకేసారి పనిచేస్తే వేగం — కానీ అవి **shared data** ని ముట్టుకుంటే ప్రమాదం. ఇద్దరు ఒకేసారి అదే bank balance ని update చేస్తే? ఈ Part concurrency యొక్క గుండె: race conditions ఎలా పుడతాయి, critical section ని ఎలా కాపాడాలి, synchronization tools (mutex, semaphore, monitor), classic problems, మరియు అన్నిటికీ రాజు — **deadlock.** ఇది OS interviews లో అత్యధికంగా అడిగే Part.

---

7. Race Conditions & Critical Section

వివరణ

Race condition = ఇద్దరు (లేదా ఎక్కువ) threads/processes **shared data ని ఒకేసారి access** చేసి, కనీసం ఒకరు write చేస్తున్నప్పుడు — final result **ఏ order లో run అయ్యాయో దాన్ని బట్టి** మారిపోవడం. అంటే output "ఎవరు గెలిచారు" అనే race మీద ఆధారపడుతుంది → అందుకే "race condition." ఇది bug — కానీ intermittent (అప్పుడప్పుడు మాత్రమే కనిపిస్తుంది), అందుకే కనిపెట్టడం కష్టం.

అసలు problem: మనం "ఒక్క line" అనుకునే operation కూడా hardware level లో **అనేక steps** — అవి మధ్యలో interrupt అవ్వచ్చు.

`count = count + 1` నిజంగా 3 steps:

```
1. LOAD  count → register    (memory నుండి చదువు)
2. ADD   register + 1         (1 కలుపు)
3. STORE register → count     (memory కి తిరిగి రాయి)
```

ఈ 3 steps మధ్యలో context switch జరిగితే — విపత్తు.

Real-life Scenario

> **Race condition = joint bank account, ఇద్దరూ ఒకేసారి ₹1000 withdraw.**
>
> Account balance ₹1000. భార్య ATM లో, భర్త mobile app లో — **ఒకేసారి** ₹1000 withdraw చేస్తున్నారు. రెండు transactions ఇలా జరుగుతాయి:
>
> - భార్య app: balance చదువుతుంది → ₹1000. "1000 ≥ 1000, OK."
> - **అదే క్షణంలో** భర్త app: balance చదువుతుంది → ఇంకా ₹1000 (భార్య ఇంకా deduct చేయలేదు). "1000 ≥ 1000, OK."
> - భార్య: 1000 − 1000 = 0 రాస్తుంది.
> - భర్త: తను చదివిన 1000 − 1000 = 0 రాస్తుంది.
>
> **ఫలితం:** ₹2000 బయటికి వచ్చాయి, balance ₹0! Bank ₹1000 నష్టపోయింది. ఎందుకు? ఇద్దరూ **update అవ్వకముందు** balance చదివారు (stale read). ఇదే race condition — "check చేయడం" మరియు "act చేయడం" మధ్య ఇంకొకరు జొరబడ్డారు.
>
> **పరిష్కారం:** ఒకరు account ముట్టుకున్నప్పుడు **తలుపు వేసుకోవాలి** (lock) — రెండోవాడు తలుపు తీసేదాకా ఆగాలి. అదే **mutual exclusion.**

Critical Section — problem యొక్క గుండె

Critical Section (CS) = code లో ఆ భాగం, ఇక్కడ **shared resource ని access/modify** చేస్తారు. పై ఉదాహరణలో `balance = balance - amount` — ఇది critical section. నియమం: **ఒకేసారి ఒక్క thread మాత్రమే** critical section లో ఉండాలి (mutual exclusion).

ప్రతి thread యొక్క code నిర్మాణం:

```
do {
    [ ENTRY SECTION ]       ← "నేను లోపలికి వెళ్తా" — permission తీసుకో (lock)
        CRITICAL SECTION    ← shared data ని ముట్టుకునే code
    [ EXIT SECTION ]        ← "అయ్యింది" — permission వదులు (unlock)
        REMAINDER SECTION   ← మిగతా (shared కాని) code
} while (true);
```

Critical Section సమస్యకు 3 అవసరాలు (interview core)

ఏ correct solution అయినా ఈ మూడు షరతులు తీర్చాలి:

#	అవసరం	అర్థం
1	**Mutual Exclusion**	ఒకేసారి **ఒక్క** process మాత్రమే critical section లో ఉండాలి
2	**Progress**	CS ఖాళీగా ఉండి, కొందరు లోపలికి వెళ్లాలనుకుంటే — ఎవరో ఒకరు వెళ్ళగలగాలి; నిర్ణయం **అనవసరంగా వాయిదా పడకూడదు** (CS లో లేని process ఎవరినీ ఆపకూడదు)
3	**Bounded Waiting**	ఒక process CS కోసం అడిగాక, ఇతరులు లోపలికి వెళ్ళే సంఖ్యకు **ఒక పరిమితి** ఉండాలి — అది శాశ్వతంగా wait చేయకూడదు (no starvation)

(కొన్ని పుస్తకాలు 4వదిగా **"no assumptions about speed/number of CPUs"** చేర్చుతాయి.)

గుర్తుంచుకోండి: Mutual Exclusion + Progress + Bounded Waiting. ఏ synchronization solution ని అయినా ఈ 3 తో measure చెయ్యి.

సొంతంగా solve చేయడం ఎందుకు కష్టం?

మొదటి ఆలోచన: ఒక `flag` variable పెడదాం — "నేను లోపల ఉన్నా" అని. కానీ `flag` చూడటం + set చేయటం కూడా రెండు వేర్వేరు steps → వాటి మధ్య race! అందుకే software-only solutions (Peterson's algorithm లాంటివి) చాలా చాతుర్యంతో రాయాలి, మరియు modern CPUs లో reordering వల్ల నమ్మలేం. **పరిష్కారం:** hardware **atomic instructions** (Test-and-Set, Compare-and-Swap) — ఇవి "చదవడం + రాయడం" ని ఒకే uninterruptible step గా చేస్తాయి.

```
Test-and-Set (atomic — మధ్యలో ఎవరూ జొరబడలేరు):
boolean TestAndSet(boolean *lock) {
    old = *lock;        ┐ ఇవి రెండూ
    *lock = true;       ┘ ఒకే atomic operation
    return old;         // పాత value తిరిగిస్తుంది
}
// lock తీసుకోవడం:  while (TestAndSet(&lock)) ; // busy wait
```

CAS (Compare-And-Swap) — n, expected, new: memory value == expected అయితేనే new రాయి, atomic గా. ఇదే lock-free programming, JS atomics (`Atomics.compareExchange`), database optimistic locking కి ఆధారం.

Node connection: నీ single-threaded JS లో ఈ race conditions **variables మీద ఉండవు** (రెండు callbacks ఒకేసారి run కావు). కానీ **logical race conditions** ఉంటాయి — ఉదా `await` boundary దగ్గర: `const u = await getUser(); u.balance -= 100; await save(u);` — ఈ మధ్యలో అదే user కోసం ఇంకో request run అయితే lost update! ఇది exact బ్యాంక్ ఉదాహరణ. అందుకే DB level లో atomic updates (`$inc`), transactions, లేదా optimistic locking (version field) వాడతారు.

Key Points

- **Race condition = shared data ని concurrent access; result timing/order మీద ఆధారపడుతుంది** — intermittent bug.
- కారణం: "ఒక్క line" (`count++`) కూడా hardware లో load/modify/store — మధ్యలో interrupt అవ్వచ్చు.
- **Critical Section = shared resource ని ముట్టుకునే code.** నియమం: ఒకేసారి ఒక్కరే.
- **3 అవసరాలు: Mutual Exclusion + Progress + Bounded Waiting.** ప్రతి solution ని వీటితో పరీక్షించు.

- Software-only solutions కష్టం/నమ్మలేం; **hardware atomic instructions (Test-and-Set, CAS)** పునాది.
- Node: JS variables మీద race లేదు (single-thread), కానీ `await` **boundaries దగ్గర logical races** ఉంటాయి → DB atomic ops / transactions / optimistic locking.

Interview దృష్టి

Q1: Race condition అంటే ఏమిటి, ఒక ఉదాహరణ?

> అనేక threads shared data ని ఒకేసారి access చేసి, కనీసం ఒకరు modify చేస్తున్నప్పుడు final result execution order మీద ఆధారపడటం. ఉదా: ఇద్దరు threads `count++` (load-add-store) చేస్తే, ఒకరు load చేసి store చేయకముందే రెండోవాడు load చేస్తే — ఒక increment పోతుంది (lost update). Result nondeterministic కాబట్టి bug intermittent.

Q2: Critical section problem యొక్క 3 అవసరాలు?

> (1) **Mutual Exclusion** — ఒకేసారి ఒక్క process మాత్రమే CS లో. (2) **Progress** — CS ఖాళీగా ఉంటే, వెళ్లాలనుకునే వాటిలో ఎవరో ఒకరు వెళ్ళగలగాలి, CS బయట ఉన్నవారు ఆపకూడదు. (3) **Bounded Waiting** — ఒక process అడిగాక, ముందు వెళ్ళే వాటికి పరిమితి ఉండాలి (starvation లేదు).

Q3: `count++` atomic కాదా? ఎందుకు?

> కాదు. అది hardware లో మూడు steps: memory నుండి load, register లో add, memory కి store. ఈ steps మధ్య context switch జరిగితే రెండు threads ఒకే పాత value చదివి, ఒక increment పోతుంది. Atomic చేయాలంటే hardware instruction (Test-and-Set/CAS) లేదా lock అవసరం.

Q4: Node.js single-threaded అయితే race conditions ఉంటాయా?

> JS variables మీద data races ఉండవు (రెండు callbacks ఒకేసారి run కావు). కానీ `await` boundaries దగ్గర logical race conditions ఉంటాయి — ఒక async function await చేస్తున్నప్పుడు అదే shared state ని ఇంకో request మార్చవచ్చు (read-modify-write మధ్య interleaving → lost update). పరిష్కారం: DB atomic operations (`$inc`), transactions, లేదా version-based optimistic locking.

8. Synchronization (mutex, semaphore, monitors, condition variables)

వివరణ

Topic 7 లో problem చూశాం (race, critical section). ఇప్పుడు **tools** — వీటితో critical section ని కాపాడతాం. వీటిని **synchronization primitives** అంటారు.

Real-life Scenario

> **Synchronization = public toilet తలుపు తాళం.**
>
> ఒక్క toilet (shared resource), చాలామంది వాడాలి. తాళం లేకపోతే — ఇద్దరు ఒకేసారి తలుపు తీసి, గందరగోళం (race condition). **Mutex = ఆ తలుపు తాళం** — లోపలికి వెళ్ళినవాడు తాళం వేసుకుంటాడు (`lock`); బయటివాళ్ళు "occupied" చూసి ఆగుతారు; అతను బయటికి వచ్చి తాళం తీస్తేనే (`unlock`) తర్వాతివాడు వెళ్తాడు. **తాళం వేసినవాడే తీయాలి** (ownership).
>
> ఇప్పుడు toilet ఒకటి కాదు, **5 ఉన్నాయనుకో** (restaurant restrooms). ఒక్క తాళం కాదు — ఒక **counter (5)** ఉంది: లోపలికి ఒకరు వెళ్తే counter తగ్గుతుంది (`wait` : 5→4→...), బయటికి వస్తే పెరుగుతుంది (`signal`). Counter 0 అయ్యాక వచ్చినవాళ్ళు queue లో wait. **ఇదే counting semaphore** — N identical resources ని manage చేస్తుంది (= నీ DB connection pool). Mutex = ఒక్క తాళం; semaphore = N permits counter.

Mutex (MUTual EXclusion lock) — తలుపు తాళం

Mutex = ఒకేసారి **ఒక్క** thread మాత్రమే critical section లోకి వెళ్ళేలా చేసే lock. ఇది "own చేసేవాడు మాత్రమే వదలగలడు" (ownership ఉంది).

```
mutex.lock();        ← తలుపు తీసి, తాళం వేసుకో (ఇంకెవరూ రాలేరు)
    // critical section — safe గా shared data వాడు
mutex.unlock();      ← తాళం తీసి బయటికి రా (తర్వాతివాడు రావచ్చు)
```

Thread A lock తీసుకుంటే, B `lock()` దగ్గర **block** అవుతుంది (ఆగిపోతుంది), A `unlock()` చేసేదాకా. అప్పుడు B కొనసాగుతుంది. **Mutex కి ownership ఉంది — lock తీసుకున్న thread మాత్రమే unlock చేయాలి.**

Semaphore — permits ఉన్న counter

Semaphore = ఒక **integer counter** + రెండు atomic operations (`wait` / `P` / `acquire` మరియు `signal` / `V` / `release`). ఇది "N permits" ఉన్న box లాంటిది.

```
wait(S)   {          // P operation — permit తీసుకో
    S = S - 1;
    if (S < 0) ఈ process ని block చేసి queue లో పెట్టు;
}
signal(S) {          // V operation — permit తిరిగి ఇవ్వు
    S = S + 1;
    if (S <= 0) queue నుండి ఒక waiting process ని wake చేయి;
}
```

రెండు రకాలు:

రకం	Counter max	ఏం చేస్తుంది	ఉదాహరణ
Binary semaphore	0 లేదా 1	mutex లాంటిది (mutual exclusion)	ఒక్క printer
Counting semaphore	0 నుండి N	ఏకకాలంలో **N** వరకు అనుమతి	5 DB connections, thread pool of 4

Real-life: Counting semaphore = restaurant లో 5 tables. Semaphore = 5 తో మొదలు. ప్రతి customer వచ్చినప్పుడు `wait()` (S: 5→4→...→0). S=0 అయ్యాక వచ్చినవారు బయట wait. Customer వెళ్ళినప్పుడు `signal()` (table ఖాళీ → waiting వారికి entry). **నీ MongoDB connection pool (default maxPoolSize) ఒక counting semaphore** — N connections మాత్రమే; అన్నీ busy అయితే queries queue లో wait.

Mutex vs Semaphore — ముఖ్య తేడా (interview favorite)

అంశం	Mutex	Semaphore
ఏంటి	locking mechanism (ownership)	signaling mechanism (counter)
Value	locked/unlocked (2 states)	integer (0..N)
ఎందరు లోపల	ఒక్కరే	N వరకు (counting)
Ownership	ఉంది (lock తీసుకున్నవాడే unlock)	లేదు (ఎవరైనా signal చేయవచ్చు)
ముఖ్య వాడకం	mutual exclusion (CS కాపాడటం)	resource counting + **signaling** (thread A, thread B కి "నేను అయిపోయా" అని చెప్పడం)

Crux: Mutex = "**నా** వస్తువు, నేనే తాళం తీస్తా" (protection). Semaphore = "N permits పంచడం" లేదా "event జరిగింది అని చెప్పడం" (coordination). Binary semaphore mutex లా కనిపిస్తుంది కానీ ownership లేదు — అందుకే mutex safer for pure locking.

Busy Waiting (Spinlock) vs Blocking

Lock దొరకనప్పుడు thread ఏం చేయాలి?

- **Spinlock (busy wait):** `while(locked);` — loop లో తిరుగుతూ మళ్ళీ మళ్ళీ check. **CPU waste,** కానీ lock చాలా త్వరగా వస్తుందని తెలిస్తే context switch cost తప్పుతుంది → వేగం. Multi-core లో చిన్న critical sections కి మంచిది.
- **Blocking:** thread ని sleep చేసి, lock available అయినప్పుడు OS wake చేస్తుంది. CPU waste లేదు, కానీ context switch cost. Long waits కి మంచిది.

Monitors — high-level, సురక్షితమైన wrapper

Semaphores powerful కానీ **ప్రమాదకరం** — `wait` / `signal` order తప్పితే deadlock/race (ఒక్క typo → గంటల debugging). **Monitor** = ఒక high-level construct — shared data + దాన్ని ముట్టుకునే methods ని ఒకే unit లో పెట్టి, **ఏ క్షణంలోనైనా monitor లోపల ఒక్క thread మాత్రమే** ఉండేలా compiler/language guarantee చేస్తుంది (automatic mutual exclusion).

```
monitor BankAccount {
    int balance;
    // ఈ methods automatically mutually exclusive
    procedure withdraw(amt) { ... }   // ఒకేసారి ఒక్క thread
```

```
    procedure deposit(amt)  { ... }
}
```

Java `synchronized` keyword = monitor. ప్రతి object కి built-in lock; `synchronized` method/block ఒకేసారి ఒక్క thread నే అనుమతిస్తుంది. Programmer manually lock/unlock రాయనవసరం లేదు — అందుకే safer.

Condition Variables — "నా షరతు తీరేదాకా నిద్రపో"

కొన్నిసార్లు mutual exclusion సరిపోదు — thread ఒక **condition తీరేదాకా wait** చేయాలి (ఉదా: producer buffer నిండేదాకా consumer wait). **Condition variable** రెండు ops:

- **`wait()`** — thread ని sleep చేసి, **lock ని తాత్కాలికంగా వదులుతుంది** (కీలకం! లేకపోతే deadlock). Signal వచ్చాక తిరిగి lock తీసుకుని కొనసాగుతుంది.
- **`signal()` / `notify()`** — wait చేస్తున్న ఒక thread ని wake చేస్తుంది. `broadcast()` / `notifyAll()` = అందరినీ wake.

```
// Consumer:
lock(m);
while (buffer ఖాళీ)          // ← 'if' కాదు, 'while' (spurious wakeup safety)
    cond.wait(m);             // sleep + lock వదులు; wake అయ్యాక తిరిగి lock
item = buffer.remove();
unlock(m);

// Producer (item పెట్టాక):
lock(m);
buffer.add(item);
cond.signal();                // waiting consumer ని wake చెయ్యి
unlock(m);
```

కీలక gotcha — `while` , `if` కాదు: wake అయ్యాక కూడా condition తిరిగి check చేయాలి — spurious wakeups (కారణం లేకుండా wake) మరియు మరో thread మధ్యలో condition మార్చడం వల్ల. `while` loop దీన్ని handle చేస్తుంది. **Node connection:** JS లో ఇదే idea Promises/async-await తో — "ఈ event జరిగేదాకా wait, జరిగాక resume." `EventEmitter` , `await` ఒక high-level condition-variable లాంటి coordination.

Key Points

- **Mutex = ownership ఉన్న lock; ఒక్కరే.** `lock()` / `unlock()` . తీసుకున్నవాడే వదలాలి.
- **Semaphore = counter + wait/signal.** Binary (0/1, mutex లా) vs Counting (0..N, N permits — connection pool!). Ownership లేదు; signaling కీ కూడా వాడతారు.
- **Spinlock (busy wait)** = CPU waste కానీ context-switch తప్పుతుంది (short waits); **blocking** = sleep, long waits కి.
- **Monitor = automatic mutual exclusion wrapper** (Java `synchronized`). Semaphore కంటే safer.
- **Condition variable** = condition తీరేదాకా wait; `wait()` **lock వదులుతుంది.** ఎప్పుడూ `while` loop లో check (spurious wakeups).

Interview దృష్టి

Q1: Mutex మరియు semaphore మధ్య తేడా?

> Mutex = ownership ఉన్న locking mechanism, ఒకేసారి ఒక్క thread నే అనుమతిస్తుంది, lock తీసుకున్నవాడే unlock చేయాలి — pure mutual exclusion కి. Semaphore = integer counter + wait/signal, N వరకు threads ని అనుమతించగలదు (counting), ownership లేదు, resource pool management మరియు signaling (thread-to-thread event) కి వాడతారు. Binary semaphore mutex లా కనిపిస్తుంది కానీ ownership లేదు.

Q2: Counting semaphore ఎక్కడ వాడతారు, ఒక real ఉదాహరణ?

> Fixed సంఖ్యలో identical resources ని manage చేయడానికి — semaphore ని N (resource count) తో initialize చేసి, ప్రతి use ముందు wait(), తర్వాత signal(). ఉదా: DB connection pool (maxPoolSize connections), thread pool, N licenses. నా MERN app లో MongoDB connection pool ఒక counting semaphore లాంటిది — అన్ని connections busy అయితే కొత్త queries wait చేస్తాయి.

Q3: Monitor semaphore కంటే ఎందుకు safer?

> Semaphore లో programmer manually wait/signal సరైన order లో రాయాలి — ఒక్క తప్పు (double signal, missing wait) → deadlock/race, కనిపెట్టడం కష్టం. Monitor mutual exclusion ని language/compiler automatically enforce చేస్తుంది (Java `synchronized`) — programmer lock/unlock రాయనవసరం లేదు, తప్పులు తగ్గుతాయి.

Q4: Condition variable లో `wait()` ఎందుకు lock ని వదులుతుంది?

> Thread condition తీరేదాకా sleep అవుతోంది. అది lock పట్టుకునే sleep అయితే, condition ని మార్చాల్సిన producer కూడా lock కోసం block అవుతుంది → deadlock (ఎవరూ కదలరు). అందుకే wait() atomic గా lock వదిలి sleep అవుతుంది; signal వచ్చి wake అయ్యాక తిరిగి lock తీసుకుంటుంది. Condition ని ఎప్పుడూ `while` loop లో re-check చేయాలి (spurious wakeups).

9. Classic Problems (Producer-Consumer, Readers-Writers, Dining Philosophers)

వివరణ

ఈ మూడు problems synchronization యొక్క "hello world." Interview లో guaranteed — వీటి solution structure గుర్తుంచుకో, కేవలం rote కాదు, **ఎందుకు ఏ semaphore అనేది** అర్థం చేసుకో.

Real-life Scenario

> ఈ మూడూ నిజ జీవితంలో ప్రతిచోటా ఉన్న coordination patterns — ఒక్కొక్కటి ఒక్క analogy తో కింద వివరంగా చూద్దాం:
>
> - **Producer-Consumer = బేకరీ** (baker shelf మీద పెడతాడు, customers తీస్తారు; shelf నిండితే/ఖాళీ అయితే ఆగుతారు) — నీ **message queue / stream backpressure.**
> - **Readers-Writers = Google Doc** (అందరూ ఒకేసారి చదవవచ్చు, ఒకరు edit చేస్తే మిగతా అందరూ ఆగాలి) — నీ **DB read/write locks.**
> - **Dining Philosophers = గుండ్రటి table + shared forks** (అందరూ ఎడమ fork తీస్తే deadlock) — **deadlock యొక్క textbook demo.**
>
> ప్రతిదానికీ కీలకం ఒకటే: **shared resource ని ఏ synchronization primitive ఎలా కాపాడుతుంది.**

9.1 Producer-Consumer (Bounded Buffer)

సమస్య: ఒక **producer** items తయారుచేసి **shared buffer** (fixed size N) లో పెడుతుంది; ఒక **consumer** వాటిని తీసి వాడుతుంది. షరతులు: buffer **నిండితే producer wait** చేయాలి; buffer **ఖాళీ అయితే consumer wait** చేయాలి; buffer ని ఇద్దరూ ఒకేసారి ముట్టుకోకూడదు.

> **Real-life:** బేకరీ. Baker (producer) shelf (buffer, 20 slots) మీద bread పెడతాడు; customers (consumer) తీసుకుంటారు. Shelf నిండితే baker ఆగుతాడు; shelf ఖాళీ అయితే customer wait చేస్తాడు. **నీ Node connection: message queue (RabbitMQ/Kafka), లేదా Node `stream` backpressure — producer వేగంగా push చేస్తే, buffer full అయ్యి slow అవుతుంది. ఇదే bounded-buffer.**

Solution — 3 semaphores:

```
semaphore mutex = 1;    // buffer ని కాపాడటానికి (mutual exclusion)
semaphore empty = N;    // ఖాళీ slots ఎన్ని (మొదట అన్నీ ఖాళీ)
semaphore full  = 0;    // నిండిన slots ఎన్ని (మొదట సున్నా)

Producer:                          Consumer:
while(true) {                      while(true) {
  item = produce();                  wait(full);     // item ఉందా?
  wait(empty);   // ఖాళీ slot ఉందా?  wait(mutex);    // buffer lock
  wait(mutex);   // buffer lock        item = remove();
  buffer.add(item);                    signal(mutex);  // unlock
  signal(mutex); // unlock             signal(empty);  // ఖాళీ slot పెరిగింది
  signal(full);  // నిండిన slot++     consume(item);
}                                  }
```

కీలక insight (order matters!): `wait(empty)` / `wait(full)` **ముందు,** `wait(mutex)` **తర్వాత.** ఒకవేళ mutex ముందు తీసుకుని, తర్వాత empty=0 అని block అయితే — mutex పట్టుకునే sleep → consumer buffer ముట్టుకోలేడు → **deadlock.** అందుకే counting semaphores ముందు, mutex చివర.

9.2 Readers-Writers

సమస్య: ఒక shared resource (ఉదా DB/file). **Readers** చదవడం మాత్రమే (ఒకరికొకరు గొడవ లేదు — ఎందరైనా ఒకేసారి చదవవచ్చు). **Writers** మార్చడం (ఒక writer ఉన్నప్పుడు ఇంకెవరూ — reader కానీ writer కానీ — ఉండకూడదు). లక్ష్యం: **readers concurrent గా అనుమతించు, writer కి exclusive access ఇవ్వు.**

> **Real-life:** Google Doc/shared whiteboard. చాలామంది ఒకేసారి **చదవవచ్చు** (view). కానీ ఒకరు **edit** చేస్తున్నప్పుడు — inconsistent state రాకుండా, ఆ క్షణం ఇంకెవరూ చదవకూడదు/రాయకూడదు. **నీ connection: database read/write locks; MongoDB WiredTiger document-level locks; `RWMutex` (Go). Read-heavy workloads కి ఇది huge win.**

Solution (reader-priority):

```
semaphore rw_mutex = 1;   // writer/resource కి exclusive lock
semaphore mutex = 1;      // read_count ని కాపాడటానికి
int read_count = 0;

Writer:                          Reader:
wait(rw_mutex);                  wait(mutex);
  // ... write ...               read_count++;
signal(rw_mutex);               if (read_count == 1)    // మొదటి reader
                                   wait(rw_mutex);       // writers ని block
                                signal(mutex);
                                  // ... read ... (concurrent!)
                                wait(mutex);
                                read_count--;
                                if (read_count == 0)     // చివరి reader
                                  signal(rw_mutex);       // writers ని విడుదల
                                signal(mutex);
```

Idea: మొదటి reader rw_mutex ని lock చేస్తాడు (writers రాకుండా); **చివరి reader** బయటికి వెళ్తూ దాన్ని విడుదల చేస్తాడు. మధ్యలో readers ఎందరైనా concurrent. **Trade-off — starvation:** ఈ reader-priority version లో readers వస్తూనే ఉంటే **writer starve** అవుతాడు. Writer-priority లేదా fair variants ఈ సమస్యను పరిష్కరిస్తాయి. Interview లో "ఏ variant, ఎవరు starve?" అని అడుగుతారు.

9.3 Dining Philosophers — deadlock యొక్క classic demo

సమస్య: 5 philosophers గుండ్రటి table చుట్టూ. ప్రతి ఇద్దరి మధ్య **ఒక్క fork** (మొత్తం 5 forks). ఒక philosopher తినాలంటే **రెండు forks (ఎడమ + కుడి)** కావాలి. ప్రతివాడు: ఆలోచించు → ఆకలి → ఎడమ fork తీసుకో → కుడి fork తీసుకో → తిను → రెండూ వదులు.

```
           P0
       🍴      🍴
  P4              P1
     🍴         🍴
       P3      P2
```

```
        🍴
  (ప్రతి P కి ఎడమ/కుడి forks; forks shared)
```

Deadlock ఎలా: అందరూ ఒకేసారి ఆకలేసి, **అందరూ ఎడమ fork తీసుకుంటే** — ఇప్పుడు ప్రతివాడూ కుడి fork కోసం wait చేస్తాడు, కానీ కుడి fork పక్కవాడి ఎడమ చేతిలో ఉంది → **ఎవరూ కుడి fork పొందరు, ఎవరూ తినరు, ఎవరూ వదలరు** → deadlock (circular wait). ఇది Topic 10 లో deadlock కి perfect demo.

Solutions (ఏదో ఒకటి circular wait ని break చేయాలి):

Solution	ఎలా పనిచేస్తుంది
Resource ordering	Forks కి number ఇచ్చి, ఎప్పుడూ **చిన్న number fork ముందు** తీసుకోవాలి. అప్పుడు circular wait అసాధ్యం (అందరూ ఒకే order లో తీస్తారు).
At most 4 at table	5 chairs కి బదులు 4 మందినే table కి అనుమతించు (counting semaphore = 4). కనీసం ఒకడికి రెండు forks దొరుకుతాయి.
Both-or-none (atomic)	రెండు forks ఒకేసారి (atomic గా) దొరికితేనే తీసుకో, లేకపోతే ఏది తీసుకోకు.
Asymmetric	సరి-number philosophers ఎడమ ముందు, బేసి-number కుడి ముందు — symmetry break.

Dining philosophers = deadlock ని ఎలా create/avoid చేయాలో నేర్పే textbook example. ఇది Topic 10 కి వంతెన.

Key Points

- **Producer-Consumer:** 3 semaphores — `mutex` (1), `empty` (N), `full` (0). **counting semaphores ముందు, mutex తర్వాత** wait చెయ్యి (లేకపోతే deadlock). = message queues, stream backpressure.
- **Readers-Writers:** readers concurrent, writer exclusive. మొదటి reader lock, చివరి reader unlock. Reader-priority → **writer starvation** (fair variant అవసరం). = DB read/write locks.
- **Dining Philosophers:** అందరూ ఎడమ fork తీస్తే **circular wait deadlock.** Fix: resource ordering / limit to 4 / atomic pickup / asymmetric.
- మూడింటిలోనూ కీలకం: **ఏ resource ని ఏ semaphore కాపాడుతుంది, wait/signal order ఏమిటి.**

Interview దృష్టి

Q1: Producer-Consumer లో empty/full semaphores ఎందుకు, mutex సరిపోదా?

> Mutex కేవలం buffer ని ఒకేసారి ఒక్కరే ముట్టుకునేలా చేస్తుంది (mutual exclusion). కానీ buffer **నిండినప్పుడు producer, ఖాళీ అయినప్పుడు consumer wait** చేయాలి — ఇది counting అవసరం. `empty` (ఖాళీ slots) మరియు `full` (నిండిన slots) counting semaphores ఈ resource availability ని track చేసి, తగిన party ని block చేస్తాయి. Mutex ఒక్కటే ఈ "wait for space/item" logic ఇవ్వదు.

Q2: Producer-Consumer లో wait() order ఎందుకు ముఖ్యం?

> Producer లో `wait(empty)` ముందు, `wait(mutex)` తర్వాత ఉండాలి. తారుమారు చేస్తే – producer mutex తీసుకుని, empty=0 అని block అయితే, mutex ని పట్టుకునే sleep అవుతుంది; consumer buffer ముట్టుకోలేక item తీయలేడు, empty పెరగదు → deadlock. అందుకే counting semaphore ముందు, protecting mutex చివర.

Q3: Readers-Writers లో writer starvation ఎందుకు, పరిష్కారం?

> Reader-priority solution లో readers వస్తూనే ఉంటే read_count ఎప్పుడూ > 0, rw_mutex ఎప్పటికీ విడుదల కాదు → writer అనంతంగా wait (starvation). పరిష్కారం: writer-priority (writer waiting ఉంటే కొత్త readers ని ఆపడం) లేదా fair/FIFO ordering – ఇవి readers, writers మధ్య balance ఇస్తాయి.

Q4: Dining Philosophers లో deadlock ఎలా వస్తుంది, ఎలా నివారిస్తారు?

> అందరూ ఒకేసారి ఎడమ fork తీసుకుని కుడి fork కోసం wait చేస్తే – ప్రతివాడి కుడి fork పక్కవాడి చేతిలో → circular wait → deadlock. నివారణ (circular wait break): forks కి global ordering ఇచ్చి ఎప్పుడూ చిన్న-number fork ముందు తీసుకోవడం; లేదా table కి గరిష్ఠంగా 4 మందిని అనుమతించడం; లేదా రెండు forks atomic గా తీసుకోవడం.

10. Deadlocks (Coffman conditions, prevention, avoidance/Banker's, detection, recovery)

వివరణ

Deadlock = రెండు (లేదా ఎక్కువ) processes, ఒక్కొక్కటి ఇంకోది పట్టుకున్న resource కోసం wait చేస్తూ – **ఎవరూ ముందుకు కదలని** పరిస్థితి. అందరూ శాశ్వతంగా ఆగిపోతారు (frozen). Dining philosophers (Topic 9) దీనికి perfect example.

Real-life Scenario

> **Real-life:** ఒక ఇరుకు వంతెన మీద రెండు cars ఎదురెదురుగా వచ్చాయి – రెండూ సగం వంతెన మీద ఆగాయి. Car A ముందుకు వెళ్ళాలంటే B వెనక్కి వెళ్ళాలి; B ముందుకు వెళ్ళాలంటే A వెనక్కి వెళ్ళాలి. ఇద్దరూ "నువ్వు ముందు కదులు" అని wait చేస్తారు – **ఎవరూ కదలరు, traffic frozen.** ఇదే deadlock. లేదా: నీ చేతిలో pen ఉంది, నాకు కావాలి; నా చేతిలో paper ఉంది, నీకు కావాలి; ఇద్దరం "ముందు నువ్వు ఇవ్వు" అంటే – ఇద్దరం stuck.

4 Coffman Conditions – deadlock రావాలంటే అన్నీ 4 ఏకకాలంలో ఉండాలి

ఇది deadlock యొక్క గుండె. ఏ ఒక్కటి లేకపోయినా deadlock రాదు.

#	Condition	అర్థం	వంతెన ఉదాహరణలో
1	**Mutual Exclusion**	resource ఒకేసారి ఒక్కరే వాడగలరు (shareable కాదు)	వంతెన lane లో ఒకేసారి ఒక్క car
2	**Hold and Wait**	ఒక resource పట్టుకునే, ఇంకోదాని కోసం wait	A సగం వంతెన పట్టుకునే, మిగతా సగం కోసం wait
3	**No Preemption**	resource ని బలవంతంగా తీసుకోలేం; owner తనే వదలాలి	A ని బలవంతంగా వెనక్కి తోయలేం
4	**Circular Wait**	processes ఒక చక్రంలో ఒకరికొకరు wait (P1→P2→...→Pn→P1)	A, B ఒకరికొకరు wait

గుర్తుంచుకో (mnemonic): "ME HoW No Circle" = Mutual Exclusion, Hold and Wait, No preemption, Circular wait. **ఏ ఒక్క condition ని break చేసినా deadlock impossible** — ఇదే prevention యొక్క ఆధారం.

Deadlock ని ఎదుర్కొనే 4 వ్యూహాలు

```
   1. Prevention  → 4 conditions లో ఒకదాన్ని design తోనే అసాధ్యం చెయ్యి
   2. Avoidance   → resource ఇచ్చేముందు "ఇది deadlock కి దారితీస్తుందా?" check చెయ్యి (Banker's)
   3. Detection   → deadlock రానివ్వు, కానీ కనిపెట్టి recover చెయ్యి
   4. Ignore      → "ostrich algorithm" — పట్టించుకోకు, అరుదుగా వస్తే reboot (Linux/Windows చాలావరకు
ఇదే!)
```

1. Deadlock Prevention — ఒక condition ని break చెయ్యి

Condition break	ఎలా	సమస్య
Mutual Exclusion తీసివేయి	resources ని shareable చెయ్యి (read-only)	చాలా resources inherently non-shareable (printer)
Hold & Wait తీసివేయి	ప్రారంభంలోనే **అన్ని** resources ఒకేసారి తీసుకో, లేదా ఏది పట్టుకోకుండా అడుగు	resource utilization తక్కువ, starvation
No Preemption తీసివేయి	కావలసిన resource దొరకకపోతే, పట్టుకున్నవన్నీ వదిలి మళ్ళీ మొదలుపెట్టు	rollback ఖరీదు, work waste
Circular Wait తీసివేయి	resources కి **global ordering**; ఎప్పుడూ **పెరుగుతున్న order** లోనే తీసుకో	ordering నిర్వహణ కష్టం

అత్యంత practical = Circular Wait break (resource ordering). ఇది dining philosophers లో "చిన్న-number fork ముందు" solution — code లో locks ని ఎప్పుడూ ఒకే order లో తీసుకోవడం. Real code deadlocks చాలావరకు ఇలాగే fix చేస్తారు.

2. Deadlock Avoidance — Banker's Algorithm

Idea: deadlock రానివ్వకుండా — ప్రతి resource request కి ముందు, "ఇది ఇస్తే system **safe state** లో ఉంటుందా?" అని ముందే check చేయి. Unsafe అయితే — process ని wait చేయించు (resource ఉన్నా ఇవ్వకు).

Safe state = అన్ని processes ని ఏదో *ఒక order* లో పూర్తి చేయగలిగే హామీ ఉన్న state. ఈ order = **safe sequence.** Safe state = deadlock లేదని guarantee. (Unsafe ≠ deadlock, కానీ deadlock కి దారితీయవచ్చు.)

పేరు ఎందుకు "Banker's"? Bank తన మొత్తం cash కంటే ఎక్కువ loans promise చేయదు — ప్రతి customer max అవసరాన్ని దృష్టిలో పెట్టుకుని, అందరినీ ఏదో ఒకరోజు satisfy చేయగలిగితేనే loan ఇస్తుంది.

Worked Example: 5 processes (P0-P4), 3 resource types (A=10, B=5, C=7 total instances).

Allocation (ఇప్పటికే ఇచ్చినవి), **Max** (గరిష్ఠ అవసరం):

Process	Allocation (A B C)	Max (A B C)	Need = Max−Alloc (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Total allocated A = 0+2+3+2+0 = 7; B = 1+0+0+1+0 = 2; C = 0+0+2+1+2 = 5. **Available = Total − Allocated = (10−7, 5−2, 7−5) = (3, 3, 2).**

Safety check: ఒక process యొక్క Need ≤ Available అయితే, అది run అయ్యి, తర్వాత తన Allocation ని తిరిగి ఇస్తుంది (Available పెరుగుతుంది). ఇలా అందరినీ finish చేయగలమా?

```
Available = (3,3,2)
─────────────────────────────────────────────
P1? Need(1,2,2) ≤ (3,3,2)? YES → run.
    Available += Alloc(2,0,0) → (5,3,4)       ✔ P1 done
P3? Need(0,1,1) ≤ (5,3,4)? YES → run.
    Available += (2,1,1) → (7,4,5)            ✔ P3 done
P4? Need(4,3,1) ≤ (7,4,5)? YES → run.
    Available += (0,0,2) → (7,4,7)            ✔ P4 done
P0? Need(7,4,3) ≤ (7,4,7)? YES → run.
    Available += (0,1,0) → (7,5,7)            ✔ P0 done
P2? Need(6,0,0) ≤ (7,5,7)? YES → run.
    Available += (3,0,2) → (10,5,7)           ✔ P2 done
─────────────────────────────────────────────
అందరూ finish! ⇒ SAFE STATE
Safe sequence: <P1, P3, P4, P0, P2>
```

System safe. ఇప్పుడు request వస్తే: ఉదా P1 additional (1,0,2) అడిగింది. (1,0,2) ≤ Need(1,2,2)? yes. (1,0,2) ≤ Available(3,3,2)? yes. **Pretend** ఇచ్చాం: Available=(2,3,0), P1 Alloc=(3,0,2), Need=(0,2,0). మళ్ళీ safety check run చేసి safe అయితే — grant; unsafe అయితే — P1 ని wait చేయించు.

- **లాభం:** deadlock ఎప్పటికీ రాదు (guarantee).
- **నష్టం:** ప్రతి process యొక్క **Max అవసరం ముందే తెలియాలి** (అరుదు); resources fixed సంఖ్యలో ఉండాలి; ప్రతి request కి $O(n^2 \times m)$ safety check — ఖరీదు. అందుకే **నిజ OS లు Banker's వాడవు** — ఇది concept/interview కి

ముఖ్యం.

3. Deadlock Detection & Recovery

Deadlock రానివ్వు, కానీ అప్పుడప్పుడు check చేసి, వస్తే recover చెయ్యి.

- **Detection: Resource Allocation Graph (RAG)** వేసి **cycle** వెతుకు. Single instance per resource type అయితే — cycle = deadlock. Multiple instances అయితే — cycle deadlock కావచ్చు/కాకపోవచ్చు; Banker's-లాంటి detection algorithm వాడాలి.

```
Resource Allocation Graph లో cycle:
P1 ──requests──► R2 ──held by──► P2 ──requests──► R1 ──held by──► P1
└─────────────────────── cycle ⇒ deadlock ──────────────────────────┘
```

- **Recovery** (deadlock కనిపెట్టాక):
  1. **Process termination** — deadlocked processes అన్నీ చంపు (drastic), లేదా cycle break అయ్యేదాకా ఒక్కొక్కటి చంపు (ఏది? least work lost, lowest priority — victim selection).
  2. **Resource preemption** — ఒక process నుండి resource లాక్కుని ఇంకోదానికి ఇవ్వు; ఆ process ని safe state కి **rollback** చెయ్యి. **Starvation ప్రమాదం** (అదే victim మళ్ళీ మళ్ళీ) — victim count పరిమితం చెయ్యాలి.

4. Ostrich Algorithm — పట్టించుకోకపోవడం

తలని ఇసుకలో పెట్టే ostrich లా — deadlock ని ignore చెయ్యి. Deadlocks అరుదు, handling ఖరీదు అయితే — వచ్చినప్పుడు reboot చేయడం చౌక. **నిజంగా Linux, Windows చాలావరకు ఇదే వాడతాయి** general resources కి (deadlock ని పూర్తిగా prevent చేయడం performance ని చంపుతుంది).

Livelock మరియు Starvation తేడా

- **Deadlock:** అందరూ **ఆగిపోయారు** (no motion).
- **Livelock:** processes **కదులుతున్నాయి కానీ ముందుకు వెళ్ళట్లేదు** — ఇద్దరు corridor లో ఎదురుపడి, ఇద్దరూ ఒకే వైపు తప్పుకుని, మళ్ళీ ఎదురుపడి... అనంతం. CPU వాడుతున్నారు కానీ progress లేదు.
- **Starvation:** ఒక process resource పొందకుండా అనంతంగా wait (కానీ మిగతా system పనిచేస్తోంది). Priority scheduling లో చూశాం; పరిష్కారం aging.

Key Points

- **Deadlock = circular waiting, ఎవరూ కదలరు.** రావాలంటే **4 Coffman conditions అన్నీ:** Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait (**"ME HoW No Circle"**).
- **4 వ్యూహాలు:** Prevention (condition break), Avoidance (Banker's), Detection+Recovery, Ignore (ostrich — నిజ OS లు).
- **అత్యంత practical prevention = circular wait break (resource ordering / lock ordering).**
- **Banker's = ప్రతి request ముందు "safe state?" check** (safe sequence ఉందా). Max ముందే తెలియాలి — అందుకే theoretical.

- **Detection = RAG లో cycle;** Recovery = process kill / resource preempt+rollback (victim selection, starvation జాగ్రత్త).
- **Deadlock (stuck) vs Livelock (కదులుతున్నా progress లేదు) vs Starvation (ఒకడు మాత్రం wait).**

Interview దృష్టి

Q1: Deadlock రావడానికి అవసరమైన 4 conditions?

> Mutual Exclusion (resource ఒకేసారి ఒక్కరే), Hold and Wait (ఒకటి పట్టుకునే ఇంకోది కోసం wait), No Preemption (బలవంతంగా తీసుకోలేం), Circular Wait (processes చక్రంలో ఒకరికొకరు wait). నాలుగూ ఏకకాలంలో ఉంటేనే deadlock; ఏ ఒక్కదాన్ని break చేసినా deadlock రాదు.

Q2: Deadlock prevention లో practical గా ఏ condition break చేస్తారు?

> చాలావరకు **circular wait** — resources/locks కి global ordering ఇచ్చి, ప్రతి process ఎప్పుడూ పెరుగుతున్న order లోనే వాటిని తీసుకోవడం. అప్పుడు circular dependency అసాధ్యం. Real code లో "ఎప్పుడూ locks ని ఒకే order లో acquire చెయ్యి" అనే rule ఇదే — deadlock bugs కి అత్యంత సాధారణ fix.

Q3: Banker's algorithm ఏం చేస్తుంది, safe state అంటే ఏమిటి?

> ఇది deadlock avoidance algorithm. ప్రతి resource request ముందు — ఆ request grant చేస్తే system safe state లో ఉంటుందా అని check చేస్తుంది; unsafe అయితే process ని wait చేయిస్తుంది. Safe state = అన్ని processes ని ఏదో ఒక order (safe sequence) లో పూర్తి చేయగలిగే హామీ ఉన్న state. దీనికి ప్రతి process యొక్క maximum resource అవసరం ముందే తెలియాలి — అందుకే practical కాదు.

Q4: Safe state అంటే deadlock లేదని guarantee — unsafe state అంటే deadlock ఖాయమా?

> కాదు. Safe state = deadlock లేదని guarantee. కానీ unsafe state = deadlock కావచ్చు, ఖాయం కాదు. Unsafe అంటే "OS ఇక అందరినీ finish చేయగలదని guarantee ఇవ్వలేని" state — అక్కడ నుండి deadlock వచ్చే అవకాశం ఉంది, కానీ processes అదృష్టవశాత్తు తక్కువ resources అడిగి తప్పించుకోవచ్చు.

Q5: నిజ OS లు deadlock ని ఎలా handle చేస్తాయి?

> చాలావరకు general resources కి "ignore" (ostrich algorithm) — deadlocks అరుదు, prevention/avoidance overhead ఎక్కువ, కాబట్టి వచ్చినప్పుడు process kill/reboot చౌక. కొన్ని critical subsystems లో lock ordering (prevention) వాడతారు. Banker's algorithm theoretical గా ముఖ్యం కానీ ఆచరణలో అరుదు (Max ముందే తెలియాలి, overhead ఎక్కువ).

Q6: Deadlock మరియు livelock తేడా?

> Deadlock లో processes పూర్తిగా ఆగిపోతాయి (no CPU activity, no progress). Livelock లో processes చురుగ్గా states మారుస్తూ ఉంటాయి (CPU వాడతాయి) కానీ ముందుకు వెళ్ళవు — ఇద్దరూ ఒకరికొకరు దారి ఇవ్వబోయి పదేపదే collide అయినట్టు. రెండింటిలోనూ useful progress లేదు, కానీ livelock busy, deadlock frozen.

Part 4 — Memory (RAM ని ఎలా పంచాలి)

> ప్రతి running process కి RAM కావాలి. కానీ RAM పరిమితం, processes చాలా, ఒకరి memory ఇంకొకరు చూడకూడదు. మరి OS ఈ చిన్న RAM ని ఎలా అందరికీ safe గా, అంతకంటే ఎక్కువ ఉన్నట్టు (virtual memory) పంచుతుంది? ఈ Part: logical vs physical addresses, MMU, fragmentation, paging (ఆధునిక systems గుండె), segmentation, మరియు virtual memory + page replacement — ఇవి memory management యొక్క పూర్తి చిత్రం.

---

11. Memory Management (logical vs physical, MMU, contiguous allocation, fragmentation)

వివరణ

Memory management = OS RAM ని processes కి ఎలా కేటాయిస్తుంది, track చేస్తుంది, విడుదల చేస్తుంది, మరియు ఒకరి memory ఇంకొకరు చూడకుండా ఎలా కాపాడుతుంది — అనే మొత్తం విభాగం.

మొదట అతి ముఖ్యమైన తేడా — **logical (virtual) vs physical address:**

- **Logical/Virtual address** — CPU (నీ program) generate చేసే address. నీ code లో pointer, variable address — ఇవన్నీ logical. ప్రతి process కి తను "0 నుండి మొదలయ్యే own continuous memory" ఉన్నట్టు కనిపిస్తుంది (illusion).
- **Physical address** — actual RAM chip లో నిజమైన address. Program దీన్ని నేరుగా చూడదు.

ఒక్క వాక్యంలో: **Logical = program అనుకునేది; Physical = నిజంగా RAM లో ఉన్నది. మధ్యలో translation అవసరం.**

Real-life Scenario

> **Logical vs Physical = apartment లో మీ flat number vs postal system.**
>
> నీ friend కి address ఇచ్చావు: "Flat 302, Sunrise Apartments" (logical address — నీకు తెలిసింది, simple). కానీ postman actual delivery చేయాలంటే GPS coordinates / physical location (physical address) కావాలి. మధ్యలో ఒక **map/directory (MMU)** ఈ "Flat 302" ని నిజమైన location కి translate చేస్తుంది.
>
> లాభం: building వాళ్ళు 302 ని physically వేరే block కి మార్చినా, నీకు address అదే ("302") — directory update అవుతుంది, నువ్వు మారవు. **అదే virtual memory power — program addresses stable, physical placement OS మారుస్తుంది.**

MMU — Memory Management Unit (అనువాదకుడు)

MMU = CPU లో ఉన్న **hardware** — ప్రతి memory access కి **logical → physical address translation** చేస్తుంది, real-time లో. Software చేస్తే చాలా slow; అందుకే hardware.

```
CPU generates          MMU (hardware)          Physical RAM
logical address  ───►  translate  ──────────►  physical address
   (e.g. 302)          (add base /              (e.g. 14302)
                       page table lookup)
```

Simplest scheme — Base + Limit registers:

- **Base register** = process యొక్క memory RAM లో ఎక్కడ మొదలవుతుంది.
- **Limit register** = ఆ process ఎంత memory వాడవచ్చు (size).
- Translation: `physical = base + logical` (logical < limit అయితేనే; లేకపోతే **segmentation fault!** — process తన హద్దు దాటింది).

కీలక insight: ఈ limit check hardware లో ప్రతి access కి జరుగుతుంది — అందుకే process A, process B memory ని చదవలేదు (**protection**). నీ program లో `nullptr` deref / out-of-bounds access "Segmentation fault" ఇచ్చేది ఇందుకే.

Contiguous Memory Allocation

పాత/simple scheme: ప్రతి process కి RAM లో **ఒకే continuous block** ఇవ్వడం. Process వచ్చినప్పుడు, సరిపడా big హోల్ (free block) వెతికి ఇవ్వాలి.

Free hole ఎంపిక strategies:

Strategy	ఎలా	సమస్య
First Fit	మొదట కనిపించే సరిపడా big హోల్	వేగం, కానీ ముందు భాగం fragment అవుతుంది
Best Fit	అతి చిన్న సరిపడే హోల్ (waste తక్కువ)	slow (అన్నీ search), చిన్న unusable holes మిగులుతాయి
Worst Fit	అతి పెద్ద హోల్	మిగిలిన భాగం usable గా ఉంటుందని ఆశ, కానీ practical గా worst

సాధారణంగా **First Fit / Best Fit** better. కానీ contiguous allocation లో పెద్ద problem — **fragmentation.**

Fragmentation — RAM waste అయ్యే రెండు రకాలు

ఇది interview లో guaranteed. రెండింటి తేడా స్పష్టంగా గుర్తుంచుకో:

రకం	ఏంటి	ఎక్కడ
External Fragmentation	Total free memory సరిపోతుంది, కానీ **చెల్లాచెదురుగా చిన్న holes** గా ఉంది — ఏ ఒక్క hole పెద్ద process కి సరిపోదు	contiguous allocation, segmentation
Internal Fragmentation	Process కి ఇచ్చిన block **అవసరం కంటే పెద్దది** — మిగిలిన space ఆ process లోపలే waste (ఇంకెవరూ వాడలేరు)	fixed-size blocks, paging

```
External:  [P1][free 2KB][P2][free 3KB][P3][free 2KB]
           7KB free total, కానీ 6KB process రాదు (continuous hole లేదు) ✗
```

```
Internal:  [P1 needs 3KB, got 4KB block → 1KB waste లోపలే]
           [====P1 data====|~~waste~~]
```

External కి పరిష్కారం:

- **Compaction** — అన్ని processes ని ఒక వైపు జరిపి, free space ని ఒక పెద్ద block గా చేయడం (defragmentation). ఖరీదు (అన్నీ కదపాలి, process పాజ్ అవ్వాలి).
- **Paging** (Topic 12) — memory ని fixed-size chunks గా చేసి, contiguous అవసరం తీసేయడం. **ఇదే అసలు పరిష్కారం** — నేటి systems paging వాడతాయి.

Internal కి పరిష్కారం: చిన్న block sizes (కానీ చాలా చిన్నగా చేస్తే page table పెద్దదవుతుంది — trade-off).

Node/V8 connection: V8 (Node యొక్క JS engine) heap ని fixed-size "pages" గా manage చేస్తుంది, GC ఆబ్జెక్ట్‌లను compact చేస్తుంది (mark-compact) — అంటే internal fragmentation ని తగ్గించడానికి external fragmentation compaction ఆలోచన ని వాడుతుంది. OS memory management ideas నీ runtime లోనే ఉన్నాయి.

Key Points

- **Logical/Virtual address = program అనుకునేది; Physical = RAM లో నిజమైనది.** MMU (hardware) ప్రతి access కి translate చేస్తుంది.
- **Base + Limit registers** = simplest translation + protection (limit దాటితే **segmentation fault**). ఇదే processes ని isolate చేస్తుంది.
- **Contiguous allocation** = process కి ఒక continuous block; hole ఎంపిక First/Best/Worst Fit.
- **External fragmentation** = free memory ఉంది కానీ చెల్లాచెదురు (contiguous hole లేదు); **Internal** = allocated block అవసరం కంటే పెద్దది, లోపల waste.
- External కి పరిష్కారం **compaction** లేదా **paging** (అసలు fix); internal కి చిన్న blocks.

Interview దృష్టి

Q1: Logical మరియు physical address తేడా, MMU పాత్ర?

> Logical (virtual) address = CPU/program generate చేసేది — program కి కనిపించే address. Physical address = actual RAM లో నిజమైన location. MMU (hardware unit) ప్రతి memory access కి logical → physical translation చేస్తుంది (base+limit లేదా page table ద్వారా), అలాగే bounds check చేసి protection ఇస్తుంది. దీనివల్ల program physical layout తెలియకుండానే continuous memory ఉన్నట్టు చూస్తుంది.

Q2: Internal మరియు external fragmentation తేడా?

> Internal = process కి ఇచ్చిన block అవసరం కంటే పెద్దది కావడం వల్ల మిగిలిన space ఆ block లోపలే waste (ఇంకెవరూ వాడలేరు) — fixed-size blocks/paging లో వస్తుంది. External = total free memory సరిపోతుంది కానీ చిన్న చిన్న scattered holes గా ఉండి, ఏ ఒక్క hole పెద్ద request కి సరిపోకపోవడం — contiguous allocation/segmentation లో వస్తుంది.

Q3: External fragmentation ని ఎలా పరిష్కరిస్తారు?

> రెండు మార్గాలు: (1) Compaction — processes ని జరిపి free holes ని ఒక పెద్ద continuous block గా చేయడం (ఖరీదు, processes pause చేయాలి). (2) Paging — memory ని fixed-size frames గా విభజించి, process కి non-contiguous frames ఇవ్వడం, తద్వారా continuous block అవసరమే తీసేయడం. నేటి systems paging వాడతాయి; అది external fragmentation ని పూర్తిగా తొలగిస్తుంది (కానీ చిన్న internal fragmentation తెస్తుంది).

Q4: Segmentation fault నిజంగా ఏమిటి?

> Process దానికి కేటాయించని memory ని access చేయబోతే — MMU యొక్క limit/bounds check fail అయ్యి, hardware exception raise చేసి, OS ఆ process ని terminate చేస్తుంది. C/C++ లో null/dangling pointer deref, array out-of-bounds — ఇవి ఈ hardware protection ని trigger చేస్తాయి. అంటే ఇది OS memory-protection mechanism నిన్ను "పక్క process/kernel memory చెడగొట్టకుండా" ఆపడం.

12. Paging (pages/frames, page table, TLB, multi-level)

వివరణ

Paging = memory management scheme, ఇక్కడ **process యొక్క memory ని fixed-size chunks (pages) గా, physical RAM ని అదే size chunks (frames) గా** విభజించి, process యొక్క ఏ page ని అయినా ఏ free frame కి అయినా map చేయడం. కీలకం: **page లు contiguous గా ఉండనవసరం లేదు** — process యొక్క page 0 frame 8 లో, page 1 frame 3 లో ఉండవచ్చు. దీంతో **external fragmentation పూర్తిగా పోతుంది** (ఏ free frame అయినా వాడవచ్చు).

- **Page** = logical (virtual) memory యొక్క fixed-size block (సాధారణంగా **4 KB**).
- **Frame** = physical RAM యొక్క అదే size block.
- **Page ↔ Frame mapping** ని **page table** ఉంచుతుంది.

ఒక్క వాక్యంలో: **Paging = process memory ని "పేజీలుగా" చించి, RAM యొక్క ఏ ఖాళీ అరలో అయినా పెట్టడం; ఏ page ఎక్కడ ఉందో page table చెబుతుంది.**

Real-life Scenario

> **Paging = ఒక పుస్తకంలోని పేజీలను వేర్వేరు lockers లో దాచడం.**
>
> నీ 100-పేజీల పుస్తకాన్ని (process) 100 lockers (frames) లో ఎక్కడపడితే అక్కడ దాచావు — page 1 → locker 47, page 2 → locker 12, ... క్రమం లేదు. ఇది గందరగోళంగా అనిపించవచ్చు, కానీ నీ దగ్గర ఒక **index card (page table)** ఉంది: "page 1 = locker 47, page 2 = locker 12..." ఏ page కావాలన్నా — index చూసి, సరైన locker తీస్తావు.
>
> లాభం: పుస్తకం మొత్తం పెట్టడానికి **100 continuous lockers అవసరం లేదు** — చెల్లాచెదురుగా ఉన్న ఏ 100 ఖాళీ lockers అయినా చాలు (external fragmentation పోయింది). ఇదే paging. ఖర్చు: index card (page table) maintain చేయాలి + ప్రతిసారి lookup.

Address Translation — logical address ఎలా translate అవుతుంది

Logical address ని రెండు భాగాలుగా విడగొడతారు:

```
Logical Address = [ Page Number (p) | Page Offset (d) ]
                         ↑                   ↑
                      ఏ page?            ఆ page లో ఎంత లోపల?

Translation:
┌─────────────────────────────────────────────────────┐
│  p ──► Page Table lookup ──► f (frame number)        │
│  Physical Address = [ f | d ]  (offset అలాగే ఉంటుంది)  │
└─────────────────────────────────────────────────────┘
```

ఉదాహరణ: page size = 4 KB (4096 bytes = 2^{12}). Logical address 16-bit లో — కింది 12 bits = offset (d), పైన bits = page number (p). Page number తో page table చూసి frame number తీసి, offset అలాగే జోడిస్తే physical address. **Offset ఎప్పుడూ మారదు** (page లోపల position అదే).

Page Table — page → frame directory

ప్రతి process కి **own page table** (దాని pages ఏ frames లో ఉన్నాయో). ఒక page table entry (PTE) లో:

- **Frame number** — ఈ page ఏ physical frame లో.
- **Valid/Invalid bit** — ఈ page currently RAM లో ఉందా (invalid → page fault, Topic 14).
- **Protection bits** — read/write/execute permissions.
- **Dirty bit** — ఈ page modify అయ్యిందా (write-back అవసరమా).
- **Reference bit** — ఇటీవల access అయ్యిందా (replacement algorithms కి).

సమస్య: page table RAM లో ఉంటుంది. అంటే ప్రతి memory access కి **రెండు RAM accesses** — ఒకటి page table చదవడానికి, ఒకటి actual data కి. **Memory access రెట్టింపు slow!** దీన్ని పరిష్కరించడానికి → TLB.

TLB — Translation Lookaside Buffer (page table యొక్క cache)

TLB = MMU లో ఉన్న చిన్న, **అతివేగ hardware cache** — ఇటీవల వాడిన page→frame mappings ని ఉంచుతుంది. ప్రతి translation కి ముందు TLB చూస్తారు:

```
CPU → logical addr → MMU
                      │
                 ┌────▼────┐
                 │  TLB?   │
     hit ──┤  (cache)  ├── miss
       │         └─────────┘      │
       ▼                          ▼
  frame వెంటనే             page table (RAM) చూసి,
  (వేగం!)                 mapping ని TLB లో ఉంచి, తర్వాత access
```

- **TLB hit** — mapping cache లో ఉంది → వెంటనే frame (extra RAM access లేదు).
- **TLB miss** — page table (RAM) లో చూసి, ఆ entry ని TLB లో పెట్టి (future కోసం), తర్వాత access.

TLB hit ratio చాలా ఎక్కువ (~99%) — ఎందుకు? **Locality of reference** — programs తరచూ అవే pages ని మళ్ళీ మళ్ళీ వాడతాయి (loops, arrays). అందుకే TLB చిన్నదైనా (64-1024 entries) చాలా effective. **Context switch లో TLB**

flush అవుతుంది (వేరే process = వేరే page table) — అందుకే process switch ఖరీదు (Topic 4 లో చెప్పిన TLB pollution ఇదే).

Multi-Level Page Table — పెద్ద page table సమస్య

సమస్య: 32-bit system, 4 KB pages → 2^{20} = ~10 lakh entries per process. ప్రతి entry 4 bytes → **4 MB page table per process!** వేలాది processes ఉంటే GB ల page tables. 64-bit లో ఇది అసాధ్యంగా పెద్దది. పైగా చాలా entries unused (process మొత్తం address space వాడదు).

పరిష్కారం — Multi-level (hierarchical) page table: page table ని కూడా pages గా విభజించి, "page table యొక్క page table" (outer table) పెట్టడం. అవసరమైన భాగాలు మాత్రమే RAM లో ఉంచడం (వాడని భాగాలకి table allocate చేయవద్దు).

```
Logical: [ p1 | p2 | offset ]
           |    |
           ▼    ▼
Outer table ➝ Inner table ➝ Frame
(level 1)     (level 2)

x86-64: 4-level (PML4 → PDPT → PD → PT → frame)
```

- **లాభం:** వాడని address ranges కి inner tables allocate చేయనవసరం లేదు → memory ఆదా.
- **నష్టం:** ప్రతి translation కి **ఎక్కువ lookups** (4-level → 4 RAM accesses). అందుకే TLB మరింత ముఖ్యం (miss ఖరీదు పెరిగింది).

ప్రత్యామ్నాయాలు: Inverted page table (frame → page, ఒక్క global table; memory ఆదా కానీ search slow), hashed page table.

Paging యొక్క fragmentation

- **External fragmentation = 0** (ఏ free frame అయినా వాడవచ్చు — ఇదే paging యొక్క గొప్పతనం).
- **Internal fragmentation = చిన్నది** — process size page size యొక్క multiple కాకపోతే, చివరి page లో కొంత waste (average అర page per process ≈ 2 KB). Acceptable.

Key Points

- **Paging = memory ని fixed-size pages (logical) + frames (physical) గా;** ఏ page ని ఏ frame కి అయినా map. **External fragmentation పోతుంది** (non-contiguous allowed).
- **Logical address = page number + offset;** page table page→frame ఇస్తుంది; **offset మారదు.**
- **Page table** RAM లో → ప్రతి access రెట్టింపు slow → **TLB** (hardware cache) దీన్ని fix చేస్తుంది (locality వల్ల hit ~99%).
- **Multi-level page table** = పెద్ద page tables ని hierarchical గా చేసి, వాడనివి allocate చేయకపోవడం (x86-64 = 4 levels). Trade-off: ఎక్కువ lookups.
- Paging: **external fragmentation 0, internal fragmentation చిన్నది** (చివరి page waste).
- **Context switch** → **TLB flush** (వేరే page table) → process switch ఖరీదుకి కారణం.

Interview దృష్టి

Q1: Paging అంటే ఏమిటి, ఏ problem పరిష్కరిస్తుంది?

> Paging = logical memory ని fixed-size pages గా, physical RAM ని అదే size frames గా విభజించి, process యొక్క pages ని ఏ free frames కి అయినా (non-contiguous) map చేయడం; ఆ mapping page table లో ఉంటుంది. ఇది contiguous allocation యొక్క **external fragmentation** ని పూర్తిగా తొలగిస్తుంది (ఏ ఖాళీ frame అయినా వాడవచ్చు) మరియు virtual memory (demand paging) ని సాధ్యం చేస్తుంది.

Q2: TLB అంటే ఏమిటి, ఎందుకు అవసరం?

> Page table RAM లో ఉంటుంది కాబట్టి ప్రతి memory access కి రెండు RAM accesses అవుతాయి (page table + actual data) — రెట్టింపు slow. TLB = MMU లోని చిన్న అతివేగ hardware cache; ఇటీవలి page→frame mappings ని ఉంచుతుంది. TLB hit అయితే page table చూడకుండానే frame దొరుకుతుంది. Locality of reference వల్ల hit ratio ~99% — అందుకే paging fast.

Q3: 64-bit system లో single-level page table ఎందుకు సాధ్యం కాదు, పరిష్కారం?

> 64-bit address space విపరీతంగా పెద్దది — single-level table కి అసాధ్యంగా ఎక్కువ entries (petabytes) అవసరం, పైగా చాలావరకు unused. పరిష్కారం multi-level (hierarchical) page table — table ని levels గా విభజించి, process actually వాడే address ranges కి మాత్రమే inner tables allocate చేయడం (x86-64 లో 4 levels). ఇది memory ఆదా చేస్తుంది కానీ translation కి ఎక్కువ lookups అవుతాయి (TLB అందుకే ముఖ్యం).

Q4: Paging లో ఏ రకం fragmentation ఉంటుంది?

> External fragmentation ఉండదు (ఏ free frame అయినా వాడవచ్చు — ఇదే paging యొక్క ప్రధాన లాభం). కానీ చిన్న internal fragmentation ఉంటుంది — process size page size యొక్క exact multiple కాకపోతే చివరి page లో కొంత space waste అవుతుంది (average process కి అర page ≈ 2KB). ఇది acceptable trade-off.

Q5: Page size పెద్దగా / చిన్నగా పెడితే trade-off?

> పెద్ద pages → చిన్న page table (తక్కువ entries), మెరుగైన TLB coverage, disk I/O efficient; కానీ ఎక్కువ internal fragmentation + demand paging లో అనవసర data load. చిన్న pages → తక్కువ internal fragmentation కానీ పెద్ద page table + ఎక్కువ TLB misses. అందుకే typical 4KB; పెద్ద workloads కి "huge pages" (2MB) వాడతారు.

13. Segmentation (vs paging)

వివరణ

Segmentation = memory ని **variable-size, logical units (segments)** గా విభజించడం — program యొక్క **అర్థవంతమైన భాగాల** ప్రకారం. Paging fixed-size mechanical chunks (4KB) గా చేస్తే, segmentation program యొక్క

logical structure ని అనుసరిస్తుంది: code segment, data segment, stack segment, heap segment.

అంటే — programmer/compiler దృష్టిలో program "functions, arrays, stack" అనే units గా ఉంటుంది; segmentation ఆ units నే memory segments గా ఉంచుతుంది. ప్రతి segment కి own **base** (RAM లో ఎక్కడ) + **limit** (ఎంత పెద్దది).

ఒక్క వాక్యంలో: **Paging = memory ని saman (equal) muklu గా చించడం (hardware view); Segmentation = memory ని artham (meaning) prakaram muklu గా చించడం (programmer view).**

Real-life Scenario

> **Segmentation = ఒక పుస్తకాన్ని అధ్యాయాల ప్రకారం (chapters) విభజించడం; Paging = అదే పుస్తకాన్ని ప్రతి 20 పేజీలకి ఒక bundle గా విభజించడం.**
>
> - **Segmentation (chapters):** "Introduction", "Methods", "Results" — ఒక్కో chapter వేరే length, కానీ ప్రతిదీ **అర్థవంతమైన unit.** నీకు "Results" కావాలంటే నేరుగా ఆ chapter కి వెళ్తావు. కానీ chapters వేర్వేరు sizes కాబట్టి shelf లో పెట్టేటప్పుడు ఖాళీలు (external fragmentation) వస్తాయి.
> - **Paging (20-page bundles):** ప్రతి bundle సరిగ్గా 20 పేజీలు — shelf లో ఏ ఖాళీలోనైనా సరిపోతుంది (no external fragmentation), కానీ "Results" ఒక bundle మధ్యలో మొదలవుతుంది — logical గా అర్థం లేదు.
>
> **అందుకే ఆధునిక systems రెండూ కలిపి వాడతాయి** (segmentation with paging) — logical meaning (segments) + efficient placement (pages).

Address Translation

Segmented address = **[segment number (s) | offset (d)]**. Segment table లో ప్రతి segment కి base + limit:

```
Logical: [ s | d ]
           |
           ▼
Segment Table[s] → (base, limit)
           |
if (d < limit)  →  physical = base + d
else            →  TRAP (segmentation fault — segment హద్దు దాటింది)
```

"Segmentation fault" అనే పేరు ఇక్కడి నుండే వచ్చింది — offset segment limit దాటితే hardware trap. (నేటి paging systems లోనూ ఈ పేరు మిగిలింది.)

Segmentation vs Paging — పూర్తి పోలిక (interview core)

అంశం	Paging	Segmentation
Block size	fixed (4 KB)	variable (segment size)
విభజన ఆధారం	mechanical (hardware)	logical (code/data/stack — meaning)
Programmer కి కనిపిస్తుందా	కాదు (invisible)	అవును (logical units)
Fragmentation	internal (చిన్నది)	**external** (variable sizes వల్ల)
Address	page number + offset	segment number + offset
Table	page table (frame numbers)	segment table (base + limit)
Protection	page-level (per page bits)	segment-level (సహజం — entire logical unit కి, ఉదా code = read-only)
Sharing	page-level	segment-level (సులభం — ఒక shared library segment ని పంచుకోవచ్చు)

కీలక తేడాలు:

1. **Paging → internal fragmentation; Segmentation → external fragmentation.** (సరిగ్గా వ్యతిరేకం — ఎందుకంటే paging fixed size, segmentation variable size.)
2. **Segmentation programmer view ని పోలుస్తుంది** (logical units), **paging హార్డ్‌వేర్ view** (equal chunks).
3. **Segmentation protection/sharing సహజం** — ఒక్క segment మొత్తం code కదా, దానికి "read-only" ఇస్తే చాలు.

Segmentation with Paging (ఆధునిక hybrid)

రెండింటి good parts: program ని **segments గా** విభజించి, ప్రతి segment ని **pages గా** విభజించడం. దీంతో logical structure (segments) + external fragmentation-free placement (pages). x86 ఈ hybrid ని support చేస్తుంది. కానీ ఆచరణలో **నేటి OS లు (Linux, Windows) దాదాపు pure paging వాడతాయి** — segmentation ని కనిష్ఠంగా (flat memory model) వాడతాయి, ఎందుకంటే paging simpler + external fragmentation లేదు.

నీ program memory layout (Topic 2) నిజంగా segments — text, data, heap, stack. అవి ఆధునిక systems లో paging మీద నిర్మించిన logical segments. అంటే నీ stack overflow / segfault అనే terms segmentation heritage నుండి.

Key Points

- **Segmentation = variable-size logical units** (code/data/stack/heap) — **programmer view;** paging = fixed-size mechanical chunks — hardware view.
- Address = **segment number + offset**; segment table లో base + limit; offset > limit → **segmentation fault** (ఈ పేరు మూలం).
- **Paging → internal fragmentation; Segmentation → external fragmentation** (వ్యతిరేకం).

- Segmentation **protection & sharing సహజం** (logical unit మొత్తానికి — code read-only, shared library).
- నేటి systems ఎక్కువగా **paging** (లేదా segmentation-with-paging hybrid); pure segmentation అరుదు.

Interview దృష్టి

Q1: Segmentation మరియు paging తేడా?

> Paging = fixed-size blocks (pages/frames), hardware-driven, programmer కి invisible, internal fragmentation ఇస్తుంది. Segmentation = variable-size logical units (code, data, stack), program structure ని పోలుస్తుంది, programmer కి visible, external fragmentation ఇస్తుంది. Paging placement efficient; segmentation logical protection & sharing సహజంగా ఇస్తుంది. ఆధునిక systems ఎక్కువగా paging (కొన్నిసార్లు segmentation-with-paging hybrid) వాడతాయి.

Q2: Paging internal, segmentation external fragmentation — ఎందుకు?

> Paging fixed-size blocks వాడుతుంది కాబట్టి process యొక్క చివరి page పూర్తిగా నిండకపోతే ఆ block లోపల space waste (internal). Segmentation variable-size segments వాడుతుంది కాబట్టి, segments వచ్చిపోతూ ఉంటే RAM లో వేర్వేరు size holes మిగిలి, ఏదీ కొత్త పెద్ద segment కి సరిపోకపోవడం (external). అంటే fixed→internal, variable→external.

Q3: Segmentation లో protection/sharing ఎందుకు సులభం?

> ప్రతి segment ఒక అర్థవంతమైన logical unit (మొత్తం code, మొత్తం data). కాబట్టి ఒక్క segment కి "read-only/execute-only" అని permission ఇస్తే మొత్తం code protect అవుతుంది; ఒక shared library segment ని అనేక processes వాడేలా ఒక్క entry share చేస్తే చాలు. Paging లో అదే protection/sharing ని అనేక individual pages మీద apply చేయాలి — segmentation లో ఇది natural.

Q4: "Segmentation fault" అనే పేరు ఎక్కడి నుండి వచ్చింది?

> Segmentation systems లో — process ఒక segment యొక్క offset ని దాని limit దాటి access చేస్తే, hardware "segment violation" trap raise చేస్తుంది; దానినే segmentation fault అన్నారు. ఆధునిక paging systems లో కూడా invalid memory access (null/dangling pointer, out-of-bounds) కి ఈ పేరు మిగిలిపోయింది — నిజంగా అది page-level protection fault అయినా.

14. Virtual Memory & Demand Paging (page faults, replacement, thrashing)

వివరణ

Virtual memory = OS గొప్ప illusion — **RAM కంటే ఎక్కువ memory ఉన్నట్టు** ప్రతి process కి కనిపింపజేయడం. ఎలా? process యొక్క అన్ని pages ని RAM లో ఉంచకుండా, **ప్రస్తుతం అవసరమైనవి మాత్రమే RAM లో, మిగతావి disk లో**

(swap space / page file) ఉంచడం. Program మొత్తం RAM లో ఉన్నట్టు అనుకుంటుంది; నిజానికి కొంత భాగం disk లో.

Demand Paging = దీన్ని అమలు చేసే విధానం — **page అవసరమైనప్పుడు మాత్రమే** (on demand) disk నుండి RAM కి తీసుకురావడం. Program start అయినప్పుడు ఏ page ని load చేయరు (lazy loading); access చేసినప్పుడే load.

ఒక్క వాక్యంలో: **Virtual memory = "RAM కంటే పెద్ద program run చేయవచ్చు" అనే illusion; demand paging = అవసరమైన పేజీలు మాత్రమే, అవసరమైనప్పుడే RAM కి తేవడం.**

Real-life Scenario

> **Virtual memory = నీ చిన్న study desk (RAM) + పెద్ద bookshelf (disk).**
>
> నీ desk (RAM) మీద 5 పుస్తకాలే పడతాయి. కానీ నీకు 50 పుస్తకాలు (program మొత్తం) కావాలి. ఏం చేస్తావు? అవసరమైన పుస్తకాన్ని shelf నుండి desk కి తెచ్చుకుంటావు (demand paging). Desk నిండితే — వాడని ఒక పుస్తకాన్ని shelf కి తిరిగి పెట్టి (page out), కొత్తది తెచ్చుకుంటావు (page in). నీకు "50 పుస్తకాలూ నా దగ్గరే ఉన్నాయి" అనిపిస్తుంది, నిజానికి desk మీద 5 మాత్రమే.
>
> కానీ ప్రతిసారి కావలసిన పుస్తకం shelf లో ఉంటే — లేచి వెళ్ళి తేవాలి (**page fault — నెమ్మది**). నువ్వు తెలివిగా, తరచూ వాడే పుస్తకాలు desk మీదే ఉంచుకుంటే — తక్కువ trips (locality). కానీ desk చాలా చిన్నదైతే — ప్రతి పుస్తకానికీ shelf trip → చదవడం కంటే నడవడంలోనే time (**thrashing!**).

Page Fault — heart of demand paging

Process ఒక page ని access చేసినప్పుడు — ఆ page RAM లో లేకపోతే (page table లో valid bit = 0), hardware ఒక **page fault** (interrupt/trap) raise చేస్తుంది. అప్పుడు OS:

```
1. Process page X access చేస్తుంది
2. Page table: valid bit = 0 (RAM లో లేదు)  ──► PAGE FAULT trap
3. OS: X నిజంగా valid page నా? (కాకపోతే segfault → kill)
4. Free frame వెతుకు (లేకపోతే victim page ని evict → page replacement)
5. Disk నుండి X ని free frame కి load (నెమ్మది — ms level I/O)
6. Page table update (valid bit = 1, frame number)
7. Process ని ఆగిన instruction దగ్గర resume
```

Page fault ఖరీదు: disk access RAM కంటే ~100,000× slow (nanoseconds vs milliseconds). అందుకే page fault rate తక్కువగా ఉండాలి. **Effective Access Time (EAT)** = `(1-p)×RAM_access + p×page_fault_time` — ఇక్కడ p = page fault probability. p చాలా చిన్నదైనా (0.001), page fault చాలా ఖరీదు కాబట్టి EAT గణనీయంగా పెరుగుతుంది.

Node connection: నీ Node process memory RAM కంటే పెరిగితే — OS swapping మొదలుపెడుతుంది → latency spikes (GC pauses తో కలిసి). Production లో "container memory limit దాటి swap → app slow" అనేది exact ఇదే. అందుకే memory limits + monitoring ముఖ్యం.

Page Replacement Algorithms — victim ఎవరు?

RAM (frames) నిండి, కొత్త page కావాలంటే — ఏదో ఒక పాత page ని disk కి పంపి (evict), ఖాళీ చేయాలి. **ఏ page ని evict చేయాలి?** ఇది page replacement algorithm నిర్ణయిస్తుంది. లక్ష్యం: **page fault rate minimize.**

అన్నిటికీ ఒకే **reference string** వాడి compare చేద్దాం: `7 0 1 2 0 3 0 4 2 3 0 3 2` తో **3 frames.**

FIFO (First-In First-Out): అతి పాతది (ముందు వచ్చినది) ని evict.

```
Ref:  7  0  1  2  0  3  0  4  2  3  0  3  2
      ─────────────────────────────────────
F1:   7  7  7  2  2  2  2  4  4  4  0  0  0
F2:      0  0  0  0  3  3  3  2  2  2  2  2
F3:         1  1  1  1  0  0  0  3  3  3  3
      ─────────────────────────────────────
Fault:F  F  F  F  .  F  F  F  F  F  F  .  .
```

Page faults = **10.** Simple కానీ పేలవం — పాతదైనా తరచూ వాడే page ని కూడా evict చేస్తుంది. **Belady's anomaly కి గురవుతుంది.**

Optimal (OPT / MIN): భవిష్యత్తులో **అత్యంత దూరంగా (లేదా అస్సలు)** వాడే page ని evict.

```
Ref:  7  0  1  2  0  3  0  4  2  3  0  3  2
F1:   7  7  7  2  2  2  2  2  2  2  2  2  2
F2:      0  0  0  0  0  0  4  4  4  0  0  0
F3:         1  1  1  3  3  3  3  3  3  3  3
Fault:F  F  F  F  .  F  .  F  .  .  F  .  .
```

Page faults = **6** (minimum possible). **సమస్య: భవిష్యత్తు తెలియాలి → practical గా అసాధ్యం.** కానీ ఇది "best possible" benchmark — మిగతా algorithms ని దీనితో పోల్చుతారు.

LRU (Least Recently Used): అత్యంత చాలాకాలంగా వాడని page ని evict (గతం ఆధారంగా — "గతంలో వాడనిది భవిష్యత్తులోనూ వాడకపోవచ్చు").

```
Ref:  7  0  1  2  0  3  0  4  2  3  0  3  2
F1:   7  7  7  2  2  2  2  4  4  4  0  0  0
F2:      0  0  0  0  0  0  0  0  3  3  3  3
F3:         1  1  1  3  3  3  2  2  2  2  2
Fault:F  F  F  F  .  F  .  F  F  F  F  .  .
```

Page faults = **8.** Optimal (6) కి దగ్గర, practical. **సమస్య:** ప్రతి access కి "ఎప్పుడు వాడారు" track చేయాలి (timestamp/stack) → hardware overhead ఎక్కువ. అందుకే approximations (Clock) వాడతారు.

Clock (Second Chance) — LRU యొక్క practical approximation: pages ని circular list (clock) లో పెట్టి, ప్రతిదానికి **reference bit** (వాడితే 1). Evict చేయాల్సినప్పుడు clock hand తిరుగుతూ:

- reference bit = 0 → evict (ఈ victim).
- reference bit = 1 → **second chance** ఇచ్చి, bit ని 0 చేసి, hand ముందుకు.

```
        [A:1]
      ↗       ↘
   [D:0]       [B:1]      hand తిరుగుతూ 1 → 0 చేస్తూ,
```

```
     ↖          ↙                 మొదటి 0 దగ్గర ఆగి evict
       [C:0]
```

LRU అంత ఖచ్చితం కాదు కానీ **చాలా చౌక** (ఒక్క bit) — అందుకే **నిజ OS లు Clock/variants వాడతాయి,** pure LRU కాదు.

సారాంశం (అదే reference string, 3 frames):

Algorithm	Page Faults	ఆలోచన	Practical?
Optimal	6	భవిష్యత్తులో దూరంగా వాడేది evict	కాదు (benchmark)
LRU	8	గతంలో చాలాకాలం వాడనిది evict	ఖరీదు కానీ మంచిది
Clock	~LRU	reference bit తో second chance	**అవును (నిజ OS)**
FIFO	10	అతి పాతది evict	simple కానీ పేలవం

Belady's Anomaly

సాధారణ intuition: **frames ఎక్కువ ఇస్తే page faults తగ్గాలి.** కానీ **FIFO లో కొన్నిసార్లు frames పెంచితే page faults పెరుగుతాయి!** — ఇదే **Belady's anomaly.** ఇది counter-intuitive, అందుకే interview favorite.

ఉదాహరణ: reference string `1 2 3 4 1 2 5 1 2 3 4 5`:

- **3 frames → 9 page faults.**
- **4 frames → 10 page faults!** (ఎక్కువ frames, ఎక్కువ faults — anomaly).

ఎందుకు? FIFO memory యొక్క usage pattern ని పట్టించుకోదు (కేవలం age). **LRU, Optimal ఈ anomaly కి గురికావు** — అవి **stack algorithms** (n frames లో ఉన్న pages ఎప్పుడూ n+1 frames లో ఉన్న pages యొక్క subset; అందుకే frames పెంచితే faults తగ్గుతాయి లేదా same, ఎప్పుడూ పెరగవు).

Thrashing — virtual memory యొక్క nightmare

Thrashing = process లకి కావలసినంత frames లేకపోవడం వల్ల — CPU **actual work కంటే page swapping (page in/out) లోనే ఎక్కువ సమయం** గడపడం. Page fault → evict → వెంటనే ఆ evicted page మళ్ళీ కావాలి → మళ్ళీ fault → ... అనంత swapping. **CPU utilization కుప్పకూలుతుంది** (CPU disk I/O కోసం ఎదురుచూస్తూ ఖాళీ).

```
   CPU
   util
    │    /‾‾‾‾‾‾\              ← degree of multiprogramming పెంచితే
    │   /        \               మొదట utilization పెరుగుతుంది
    │  /          \______        ← ఒక point తర్వాత thrashing → కుప్పకూలుతుంది
    │ /
    └──────────────────────► processes సంఖ్య
                 ↑ ఇక్కడ నుండి thrashing
```

Thrashing కి కారణం: ఎక్కువ processes ని ఏకకాలంలో run చేయడం (over-committed memory) → ఒక్కొక్కరికీ frames చాలవు. **పరిష్కారం: working set model.**

Working Set Model

Working set = ఒక process ఇటీవలి time window (Δ) లో access చేసిన **unique pages సమూహం** — అంటే "ఇప్పుడు actively వాడుతున్న pages." **Locality of reference** వల్ల ఒక process ఏ క్షణంలోనైనా కొన్ని pages మాత్రమే (working set) వాడుతుంది.

Idea: ప్రతి process యొక్క **working set మొత్తం RAM లో ఉంచగలిగితే** — page faults తక్కువ, thrashing లేదు. OS అన్ని processes యొక్క working sets మొత్తం available frames కంటే ఎక్కువైతే — ఒక process ని suspend (swap out completely) చేసి, మిగతా వాటికి తగినంత frames ఇస్తుంది. దీంతో thrashing ఆగుతుంది.

Page Fault Frequency (PFF) = ఇంకో approach — page fault rate ని monitor చేసి, ఎక్కువైతే ఆ process కి frames పెంచు, తక్కువైతే తగ్గించు.

Node/production connection: container కి RAM తక్కువ ఇచ్చి, memory-heavy Node app run చేస్తే — swap thrashing → requests అన్నీ నెమ్మది. "Right-size your memory limits" అనే ops advice యొక్క OS కారణం ఇదే.

Key Points

- **Virtual memory = RAM కంటే పెద్ద program run చేసే illusion** (కొన్ని pages disk లో). **Demand paging = అవసరమైన page ని అవసరమైనప్పుడే load** (lazy).
- **Page fault** = access చేసిన page RAM లో లేదు → OS disk నుండి load (చాలా ఖరీదు, ~100,000× slow). EAT page fault rate కి చాలా sensitive.
- **Replacement:** Optimal (best, impractical) > LRU (గతం ఆధారంగా, ఖరీదు) ≈ Clock (reference bit, **నిజ OS**) > FIFO (simple, పేలవం).
- **Belady's anomaly:** FIFO లో frames పెంచితే faults పెరగవచ్చు. **LRU/Optimal (stack algorithms) దీనికి గురికావు.**
- **Thrashing** = work కంటే swapping ఎక్కువ → CPU util కుప్పకూలుతుంది. కారణం: too many processes, memory over-commit.
- **Working set** = ఇటీవల వాడిన pages; ప్రతి process working set ని RAM లో ఉంచితే thrashing లేదు.

Interview దృష్టి

Q1: Virtual memory మరియు demand paging అంటే ఏమిటి?

> Virtual memory = process కి physical RAM కంటే ఎక్కువ memory ఉన్నట్టు కనిపింపజేసే abstraction — కొన్ని pages RAM లో, మిగతావి disk (swap) లో. Demand paging = దీన్ని అమలు చేసే lazy strategy: pages ని ముందే load చేయకుండా, process access చేసినప్పుడే disk నుండి RAM కి తెస్తుంది. ఇది తక్కువ RAM తో పెద్ద programs run చేయనిస్తుంది + startup వేగం.

Q2: Page fault అంటే ఏమిటి, ఎందుకు ఖరీదు?

> Process access చేసిన page RAM లో లేనప్పుడు (page table valid bit = 0) hardware raise చేసే trap. OS disk నుండి ఆ page ని free frame కి load చేసి (అవసరమైతే victim page evict), page table update చేసి, process ని resume చేస్తుంది. Disk access RAM కంటే ~100,000× నెమ్మది కాబట్టి చాలా ఖరీదు; అందుకే page fault rate తక్కువగా ఉంచడం కీలకం.

Q3: LRU vs FIFO vs Optimal — ఏది ఉత్తమం, నిజ OS ఏది వాడుతుంది?

> Optimal (భవిష్యత్తులో దూరంగా వాడే page ని evict) కనిష్ఠ faults ఇస్తుంది కానీ భవిష్యత్తు తెలియాలి — కేవలం benchmark. LRU (గతంలో చాలాకాలం వాడని page) practical గా మంచిది కానీ ప్రతి access track చేయాలి (ఖరీదు). FIFO simple కానీ పేలవం + Belady's anomaly. నిజ OS లు LRU యొక్క చౌక approximation — **Clock (second-chance)** — reference bit తో వాడతాయి.

Q4: Belady's anomaly అంటే ఏమిటి?

> సాధారణంగా frames పెంచితే page faults తగ్గాలి. కానీ FIFO లో కొన్ని reference strings కి frames పెంచితే page faults **పెరుగుతాయి** — ఇదే Belady's anomaly (counter-intuitive). FIFO usage pattern ని పట్టించుకోకపోవడం వల్ల. LRU, Optimal వంటి stack algorithms (n frames pages ⊆ n+1 frames pages) దీనికి గురికావు.

Q5: Thrashing అంటే ఏమిటి, ఎలా పరిష్కరిస్తారు?

> Processes కి కావలసిన frames లేకపోవడం వల్ల CPU actual work కంటే page in/out (swapping) లోనే ఎక్కువ సమయం గడపడం — page faults పేలి CPU utilization కుప్పకూలుతుంది. కారణం memory over-commit (too many processes). పరిష్కారం: working set model (ప్రతి process working set ని RAM లో ఉంచడం, లేకపోతే కొన్ని processes suspend), లేదా page-fault-frequency ఆధారంగా frames సర్దుబాటు, లేదా RAM పెంచడం.

Q6: Working set అంటే ఏమిటి?

> ఒక process ఇటీవలి time window (Δ) లో access చేసిన unique pages సమూహం — అంటే ప్రస్తుతం అది actively వాడుతున్న "hot" pages. Locality of reference వల్ల ఇది process మొత్తం pages కంటే చాలా చిన్నది. OS ప్రతి process యొక్క working set ని RAM లో ఉంచగలిగితే page faults తక్కువ, thrashing ఉండదు; working sets మొత్తం RAM దాటితే కొన్ని processes ని suspend చేస్తుంది.

Part 5 — Storage & I/O

> CPU, memory అయ్యాయి. ఇప్పుడు data శాశ్వతంగా ఎక్కడ ఉంటుంది (disk/files), ఆ slow disk ని OS ఎలా efficient గా వాడుతుంది (disk scheduling), మరియు keyboard/network/disk లాంటి devices తో CPU ఎలా మాట్లాడుతుంది (I/O, interrupts, DMA) — ఈ మూడు ఈ Part లో. నీ MongoDB data చివరికి disk మీద files గా ఉంటుంది; ఆ files ని OS ఎలా manage చేస్తుందో ఇక్కడ.

---

15. File Systems (files, directories, allocation, inodes)

వివరణ

File system = OS disk (శాశ్వత storage) మీద data ని **files మరియు directories** గా organize చేసి, store/retrieve చేసే విధానం. Disk నిజానికి కేవలం లక్షల **blocks** (sectors) — 0, 1, 2, ... అనే numbered slots. File system ఈ raw blocks మీద "file", "folder", "name", "permissions" అనే abstractions నిర్మిస్తుంది.

- **File** = related data యొక్క named collection (నీ `server.js` , ఒక image, MongoDB యొక్క data file). OS దృష్టిలో file = bytes యొక్క sequence + metadata (size, owner, dates, permissions).
- **Directory (folder)** = files మరియు ఇతర directories ని organize చేసే container — నిజానికి "name → file location" mappings ఉన్న ఒక special file.

ఒక్క వాక్యంలో: **File system = disk అనే గోడౌన్ (numbered shelves) ని ఒక చక్కటి filing cabinet (folders + labeled files) గా మార్చే software.**

Real-life Scenario

> **File system = ఒక పెద్ద library.**
>
> Library లో వేలాది పుస్తకాలు (data blocks) ఉన్నాయి. కానీ నీకు కావలసిన పుస్తకం వెంటనే దొరకాలంటే — ఒక **catalog system** (file system) కావాలి:
>
> - **Books** = files (actual content).
> - **Shelves with numbers** = disk blocks.
> - **Catalog card** (పుస్తకం title, author, ఏ shelf లో ఉంది, ఎన్ని పేజీలు) = **inode** (file యొక్క metadata + block locations).
> - **Section directory** ("Fiction → Row 5") = directory (name → inode mapping).
>
> నువ్వు "Harry Potter" అడిగితే — catalog (directory) చూసి catalog card (inode) తీసి, అది "shelf 47, 52, 89" అని చెబితే (block pointers), ఆ shelves నుండి పుస్తకం భాగాలు తెస్తావు. **పుస్తకం పేరు (filename) vs దాని physical location (blocks) — file system ఈ రెంటిని వేరు చేసి connect చేస్తుంది.**

File Attributes & Operations

Attributes (metadata): name, unique identifier (inode number), type, size, location (blocks), protection (permissions rwx), owner, timestamps (created/modified/accessed).

Operations (system calls): create, open, read, write, seek (position మార్పు), close, delete. నీ Node `fs.open/read/write/close` ఇవే system calls wrap చేస్తాయి. **File descriptor (fd)** = open file కి OS ఇచ్చే integer handle (0=stdin, 1=stdout, 2=stderr; తర్వాతివి నీ files/sockets). ప్రతి process కి own fd table (PCB లో).

Directory Structure

```
/                    ← root
├── home/
│   └── yaswanth/
│       ├── server.js
│       └── data/
│           └── db.json
└── usr/
    └── bin/
        └── node
```

Tree-structured directory (నేటి standard) — root (`/`) నుండి మొదలై, folders లోపల folders. **Path** = root నుండి file కి route (`/home/yaswanth/server.js` = absolute; `data/db.json` = relative to current dir). Unix లో అంతా ఒకే tree; Windows లో ప్రతి drive (C:, D:) కి వేరే tree.

File Allocation Methods — disk blocks ని file కి ఎలా కేటాయించాలి

ఒక file చాలా blocks తీసుకుంటుంది. ఆ blocks ని ఎలా track చేయాలి? మూడు methods:

Method	ఎలా	లాభం	నష్టం
Contiguous	file యొక్క blocks అన్నీ **వరుసగా** (block 10,11,12...)	వేగం (sequential + random access easy), simple	**external fragmentation**, file పెరగడం కష్టం (పక్క block occupied అయితే)
Linked	ప్రతి block లో **తర్వాతి block pointer** (linked list)	fragmentation లేదు, file పెరగవచ్చు	**random access slow** (nth block కి n blocks traverse), pointer corrupt → data loss
Indexed	ప్రతి file కి ఒక **index block** — అందులో file యొక్క అన్ని block pointers	random access fast, fragmentation లేదు	index block overhead (చిన్న files కి కూడా)

Indexed allocation = నేటి Unix/Linux వాడేది (inode ద్వారా). అందుకే దాన్ని deep గా చూద్దాం.

inode — Unix/Linux యొక్క గుండె (interview favorite)

inode (index node) = ఒక file యొక్క **మొత్తం metadata + data block pointers** ఉంచే on-disk structure. **కీలకం: inode లో filename ఉండదు!** inode లో ఉండేవి:

```
┌─────────────── inode ───────────────┐
│ File type (regular/dir/link)         │
│ Permissions (rwxr-xr-x)              │
│ Owner (UID), Group (GID)             │
│ Size, Timestamps (atime/mtime/ctime) │
│ Link count (ఎన్ని names దీన్ని చూపుతున్నాయి)│
│ ─── Data Block Pointers: ───         │
│  Direct pointers  → data blocks (12) │
│  Single indirect  → block of pointers│
│  Double indirect  → block of blocks  │
│  Triple indirect  → ...              │  ← పెద్ద files కి scale
└──────────────────────────────────────┘
```

Filename ఎక్కడ? — directory లో. Directory నిజానికి "name → inode number" entries ఉన్న file. అంటే: **directory: "server.js" → inode 12345; inode 12345: metadata + "data blocks 88, 91, 102".**

ఈ separation యొక్క గొప్పతనం — hard links: ఒకే inode ని అనేక names (directory entries) చూపవచ్చు. `ln original.txt backup.txt` → రెండు names, ఒకటే inode (ఒకటే data). **Link count** ఎన్ని names ఉన్నాయో లెక్కిస్తుంది; అన్ని names delete అయ్యాక (count=0) మాత్రమే నిజంగా data free అవుతుంది. **Soft link (symlink)** = వేరు — అది ఒక path ని (filename ని) store చేసే separate file (target delete అయితే symlink broken).

Direct + indirect pointers ఎందుకు? చిన్న files (చాలావరకు) 12 direct pointers తో పూర్తవుతాయి (వేగం). పెద్ద files కి indirect blocks (pointers of pointers) scale అవుతాయి. అంటే చిన్నవాటికి fast, పెద్దవాటికి capable — ఒకే design లో రెండూ.

Node connection: నీ `fs.stat(file)` return చేసే `size`, `mtime`, `mode`, `ino` (inode number), `nlink` — ఇవన్నీ నేరుగా inode నుండి. `fs.link()` = hard link, `fs.symlink()` = soft link. MongoDB, PostgreSQL data files కూడా ఇలాంటి inodes ద్వారా managed.

Free Space Management & Journaling

- **Free space tracking:** ఏ blocks ఖాళీగా ఉన్నాయో track చేయడానికి — **bitmap** (ప్రతి block కి 1 bit: 0=free, 1=used) లేదా free list. కొత్త file కి free blocks ఇక్కడ నుండి తీస్తారు.
- **Journaling (crash safety):** file operation మధ్యలో power పోతే — file system corrupt కాకూడదు. **Journaling file systems** (ext4, NTFS) మార్పులని ముందు ఒక "journal" (log) లో రాసి, తర్వాత actual చేస్తాయి. Crash అయితే journal replay చేసి consistent state కి తెస్తాయి. (ఇది DB write-ahead-log ఆలోచనే — నీ MongoDB journal కూడా ఇదే principle.)

Key Points

- **File system = raw disk blocks మీద files/directories abstraction.** File = named bytes + metadata; directory = "name → inode" mappings ఉన్న special file.
- **File descriptor (fd)** = open file కి integer handle (0/1/2 = stdin/out/err); per-process fd table.

- **Allocation:** Contiguous (fast, external fragmentation), Linked (no fragmentation, slow random access), **Indexed/inode (నేటి Unix — fast random access)**.
- **inode = file metadata + data block pointers; filename inode లో ఉండదు** (directory లో). Direct + indirect pointers → చిన్న files fast, పెద్దవి scale.
- **Hard link = ఒకే inode కి అనేక names** (link count); **soft link = target path ని ఉంచే separate file.**
- **Journaling** = crash safety (మార్పులని log లో ముందు రాసి, replay చేయగలగడం) — DB WAL ఆలోచనే.

Interview దృష్టి

Q1: inode అంటే ఏమిటి, అందులో ఏం ఉంటుంది (filename ఉంటుందా)?

> inode = ఒక file యొక్క metadata + data block pointers ఉంచే on-disk structure — file type, permissions, owner, size, timestamps, link count, మరియు direct/indirect data block pointers. **Filename inode లో ఉండదు** — అది directory లో "name → inode number" mapping గా ఉంటుంది. ఈ separation వల్ల hard links (ఒకే inode కి అనేక names) సాధ్యం.

Q2: Hard link మరియు soft (symbolic) link తేడా?

> Hard link = అదే inode కి ఇంకో directory entry (name) — రెండు names ఒకే data ని చూపుతాయి, link count పెరుగుతుంది; ఒక name delete అయినా data ఉంటుంది (count>0 దాకా). Soft link = target యొక్క path ని store చేసే separate file (own inode); target delete అయితే symlink "broken" అవుతుంది. Hard links same filesystem లోనే + directories కి కుదరవు; symlinks filesystems దాటవచ్చు.

Q3: File allocation methods మూడింటి trade-offs?

> Contiguous = blocks వరుసగా; sequential & random access fast, simple; కానీ external fragmentation + file grow కష్టం. Linked = ప్రతి block లో next pointer; fragmentation లేదు, grow easy; కానీ random access slow (traverse), pointer corruption ప్రమాదం. Indexed (inode) = ప్రతి file కి index block తో అన్ని pointers; random access fast, fragmentation లేదు; చిన్న index overhead. నేటి Unix indexed/inode వాడుతుంది.

Q4: inode లో direct + indirect pointers ఎందుకు?

> చిన్న files (చాలావరకు) కొన్ని direct pointers (ఉదా 12) తోనే పూర్తవుతాయి — వేగం, ఒక్క inode read సరిపోతుంది. పెద్ద files కి single/double/triple indirect pointers (pointers-of-pointers blocks) scale అవుతాయి — ఒక్క inode తో GB ల files address చేయవచ్చు. ఇది "చిన్నవాటికి fast, పెద్దవాటికి capable" అనే balance ఒకే structure లో ఇస్తుంది.

Q5: File descriptor అంటే ఏమిటి?

> Process ఒక file/socket ని open చేసినప్పుడు OS ఇచ్చే non-negative integer handle. అది process యొక్క fd table లో ఒక entry ని (అది system-wide open-file table → inode కి) index చేస్తుంది. 0/1/2 = stdin/stdout/stderr; తర్వాతివి నీ files/sockets. Node లో `fs.open` ఇచ్చే fd ఇదే — read/write కి దీన్ని పాస్ చేస్తావు. fd లు per-process, exhausted అయితే "too many open files" error (production లో socket/file leak లక్షణం).

16. Disk Scheduling (FCFS / SSTF / SCAN / C-SCAN)

వివరణ

Disk scheduling = disk కి వచ్చిన అనేక read/write requests ని **ఏ order లో serve చేయాలో** OS నిర్ణయించడం — **disk head కదలికను (seek time) minimize** చేయడం లక్ష్యం.

ముందు ఒక HDD (spinning disk) ఎలా పనిచేస్తుందో అర్థం చేసుకో:

- Disk లో గుండ్రటి **platters** తిరుగుతూ ఉంటాయి; ప్రతి platter మీద concentric circles = **tracks**; tracks లో **sectors** (blocks).
- ఒక **read/write head** ఒక arm మీద ఉండి, కావలసిన track కి కదులుతుంది.
- **Seek time** = head ని కావలసిన track కి కదిలించడానికి పట్టే సమయం (**అత్యంత ఖరీదైన భాగం** — mechanical movement).
- **Rotational latency** = కావలసిన sector head కింద తిరిగి రావడానికి పట్టే సమయం.

అంటే **seek time = head కదలిక.** Requests ని తెలివిగా order చేస్తే — head తక్కువ దూరం కదిలి, అన్నీ వేగంగా అవుతాయి. **ఇది HDD కి critical; SSD కి seek time దాదాపు లేదు** (no moving parts) కాబట్టి disk scheduling SSD లకి తక్కువ ముఖ్యం.

Real-life Scenario

> **Disk scheduling = ఒక lift (elevator) 20 అంతస్తుల building లో.**
>
> Lift (disk head) ప్రస్తుతం 5వ అంతస్తులో ఉంది. వేర్వేరు అంతస్తుల నుండి requests వచ్చాయి: 15, 2, 18, 8, 11. Lift ఏ order లో వెళ్తే — తక్కువ కదలికతో అందరినీ చేరవేస్తుంది?
>
> - **వచ్చిన order లో** (FCFS): 15→2→18→8→11 — పైకి కిందికి వెర్రిగా, చాలా కదలిక (అలసట).
> - **దగ్గరిదానికి** (SSTF): ఎప్పుడూ దగ్గరి అంతస్తు — తక్కువ కదలిక కానీ దూరపు requests wait చేస్తాయి.
> - **ఒకే దిశలో చివరిదాకా, తర్వాత తిరిగి** (SCAN/elevator): నిజ lift ఇలాగే పనిచేస్తుంది — పైకి వెళ్తూ దారిలో ఉన్నవన్నీ, తర్వాత కిందికి.
>
> **అందుకే SCAN ని "elevator algorithm" అంటారు — నిజ lift యొక్క logic.**

అన్ని ఉదాహరణలకి: **head start = 53**, request queue = **98, 183, 37, 122, 14, 124, 65, 67** (disk range 0-199).

FCFS — First Come First Serve

వచ్చిన order లోనే serve. Fair, simple, కానీ head అటూఇటూ వెర్రిగా కదులుతుంది.

```
Order: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
Moves: |98−53| + |183−98| + |37−183| + |122−37| + |14−122| + |124−14| + |65−124| + |67−65|
     =  45   +   85    +  146    +   85    +  108    +  110    +   59    +  2
Total head movement = 640
```

640 cylinders. ఎక్కువ (37 తర్వాత మళ్ళీ 122 కి వెళ్ళి, మళ్ళీ 14 కి రావడం — waste).

SSTF — Shortest Seek Time First

ఎప్పుడూ **ప్రస్తుత head position కి అతి దగ్గరి request** ని ముందు serve. Greedy.

```
From 53: దగ్గరిది 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
Moves: |65−53|+|67−65|+|37−67|+|14−37|+|98−14|+|122−98|+|124−122|+|183−124|
     =  12 + 2 + 30 + 23 + 84 + 24 + 2 + 59
Total head movement = 236
```

236 cylinders — FCFS (640) కంటే చాలా మెరుగు. **సమస్య: starvation** — head ఒక ప్రాంతంలో busy గా ఉంటే, దూరపు requests (ఉదా 183) ఎప్పటికీ turn రాకపోవచ్చు (కొత్త దగ్గరి requests వస్తూనే ఉంటే). SJF scheduling లాంటిదే — optimal-ish కానీ starvation.

SCAN (Elevator Algorithm)

Head **ఒక దిశలో చివరిదాకా** (disk చివర) వెళ్తూ దారిలో వచ్చే అన్ని requests serve చేసి, **చివర చేరాక తిరిగి** వ్యతిరేక దిశలో. Lift లాగే.

```
From 53, moving toward 0 first (ఎడమ):
53 → 37 → 14 → 0 → (తిరిగి) → 65 → 67 → 98 → 122 → 124 → 183
Moves: (53→0) = 53, then (0→183) = 183
Total head movement = 53 + 183 = 236
```

236 cylinders. లాభం: starvation లేదు (ప్రతి request ఒక sweep లో serve అవుతుంది), uniform. **సమస్య:** head చివరికి వెళ్ళి తిరిగే ముందు, ఇప్పుడే దాటిన ప్రాంతంలో కొత్త request వస్తే అది ఒక పూర్తి sweep wait చేయాలి → edges దగ్గర wait అసమానం.

C-SCAN (Circular SCAN)

SCAN లాగే ఒక దిశలో వెళ్తుంది, కానీ చివర చేరాక — **serve చేయకుండా నేరుగా మొదటికి jump** చేసి, మళ్ళీ అదే దిశలో మొదలుపెడుతుంది (circular). దీంతో **అన్ని requests కి uniform wait time.**

```
From 53, moving toward 199 (కుడి):
53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → (jump to 0) → 14 → 37
Moves: (53→199) = 146, jump (199→0) = 199, then (0→37) = 37
Total head movement = 146 + 199 + 37 = 382 (jump తో)
```

SCAN edges ని రెండుసార్లు serve చేస్తే, C-SCAN ఎప్పుడూ ఒకే దిశలో — **fairness (uniform wait) మెరుగు,** కానీ return jump overhead. **LOOK/C-LOOK** = optimization: disk చివరిదాకా కాకుండా, **చివరి request దాకా** మాత్రమే వెళ్ళి తిరగడం (అనవసర edge travel తప్పుతుంది).

పోలిక

Algorithm	ఆలోచన	Head movement (ఈ ఉదా)	Starvation?	ఎప్పుడు
FCFS	వచ్చిన order	640 (worst)	లేదు	simple, low load
SSTF	దగ్గరిది ముందు	236	**అవును**	greedy, but unfair
SCAN	చివరిదాకా, తిరిగి	236	లేదు	uniform, elevator
C-SCAN	ఒకే దిశ, jump back	382 (jump తో)	లేదు	**most uniform wait**

SSD gotcha: SSD లకి seek time లేదు (electronic, no moving head) → disk scheduling algorithms అంత relevant కావు; SSD లు internal parallelism + wear-leveling కి optimize చేస్తాయి. Cloud లో నీ DB SSD మీద ఉంటే — ఈ classic algorithms తక్కువ matter, కానీ interview కి HDD context లో అడుగుతారు.

Key Points

- **Disk scheduling లక్ష్యం = seek time (head movement) minimize.** Seek time = head ని track కి కదిలించే mechanical time (అత్యంత ఖరీదు).
- **FCFS** = fair కానీ ఎక్కువ movement. **SSTF** = దగ్గరిది ముందు, తక్కువ movement కానీ **starvation.**
- **SCAN (elevator)** = ఒక దిశలో చివరిదాకా, తిరిగి — starvation లేదు, uniform. **C-SCAN** = ఒకే దిశ + jump back → **most uniform wait.**
- **LOOK/C-LOOK** = disk చివర కాకుండా చివరి request దాకా (optimization).
- **SSD లకి seek time లేదు** → ఈ algorithms తక్కువ relevant.

Interview దృష్టి

Q1: Disk scheduling ఎందుకు అవసరం, ఏం minimize చేస్తుంది?

> HDD లో read/write head ని కావలసిన track కి కదిలించడం (seek) mechanical + అత్యంత నెమ్మది. అనేక requests వచ్చినప్పుడు వాటిని తెలివైన order లో serve చేస్తే head total movement (seek time) తగ్గి throughput పెరుగుతుంది. Disk scheduling ఈ ordering నిర్ణయిస్తుంది; seek time (head movement) minimize చేయడం ప్రధాన లక్ష్యం.

Q2: SSTF యొక్క ప్రధాన సమస్య?

> Starvation. SSTF ఎప్పుడూ దగ్గరి request ని ఎంచుకుంటుంది కాబట్టి — head ఒక ప్రాంతంలో దట్టమైన requests మధ్య ఉంటే, దూరపు requests ఎప్పటికీ turn రాకపోవచ్చు (కొత్త దగ్గరివి వస్తూనే ఉంటే). ఇది CPU scheduling లోని SJF starvation కి సమానం.

Q3: SCAN ని ఎందుకు elevator algorithm అంటారు?

> Head ఒక దిశలో చివరిదాకా వెళ్తూ దారిలోని అన్ని requests serve చేసి, చివర చేరాక తిరిగి వ్యతిరేక దిశలో వస్తుంది — సరిగ్గా ఒక lift/elevator పైకి వెళ్తూ దారిలోని floors అన్నీ ఆపి, పైన చేరాక కిందికి వచ్చినట్లు. అందుకే SCAN = elevator algorithm.

Q4: SCAN మరియు C-SCAN తేడా?

> SCAN చివర చేరాక తిరిగి వ్యతిరేక దిశలో requests serve చేస్తూ వస్తుంది (రెండు దిశలా serve). C-SCAN చివర చేరాక serve చేయకుండా నేరుగా మొదటికి jump చేసి, మళ్ళీ అదే దిశలో మొదలుపెడుతుంది — ఎప్పుడూ ఒకే దిశలో serve. C-SCAN అన్ని requests కి uniform waiting time ఇస్తుంది (SCAN లో middle requests కంటే edge requests కి తేడా); ఖర్చు: return jump.

Q5: SSD లకి disk scheduling relevant ఆ?

> చాలా తక్కువ. SSD లకి moving head/platters లేవు — data access electronic, seek time దాదాపు 0. కాబట్టి head movement minimize చేసే classic algorithms (SSTF/SCAN) SSD కి పెద్ద ప్రయోజనం ఇవ్వవు. SSD లు internal parallelism, wear-leveling, garbage collection కి optimize చేస్తాయి. ఈ algorithms ముఖ్యంగా HDD context.

17. I/O & Interrupts (polling vs interrupt, DMA)

వివరణ

CPU చాలా వేగం; devices (keyboard, disk, network card) చాలా నెమ్మది. మరి CPU ఈ slow devices తో ఎలా coordinate అవుతుంది — వాటి కోసం ఖాళీగా wait చేయకుండా? ఇదే I/O management యొక్క ప్రధాన సవాలు. మూడు విధానాలు: **polling, interrupts, DMA.**

Real-life Scenario

> **CPU vs slow device = నువ్వు (fast) పొయ్యి మీద నీళ్ళు (slow) మరిగించడం.**
>
> - **Polling:** ప్రతి 5 సెకన్లకి పొయ్యి దగ్గరకి వెళ్ళి "మరిగిందా? మరిగిందా?" అని చూస్తూనే ఉంటావు. నీ time మొత్తం checking లోనే waste — వేరే పని చేయలేవు.
> - **Interrupt:** ఒక **whistle kettle (సిటీ కొట్టే kettle)** పెట్టావు. నువ్వు హాయిగా వేరే పని (TV, ఫోన్) చేస్తావు. నీళ్ళు మరిగినప్పుడు kettle **"కూ...!" అని అరుస్తుంది (interrupt)** → అప్పుడే నువ్వు వెళ్తావు. Efficient — checking లో time waste లేదు.
> - **DMA:** ఒక assistant ని పెట్టావు — "నీళ్ళు మరిగాక, వాటిని కప్పుల్లో పోసి, అయ్యాక నాకు చెప్పు." నువ్వు అస్సలు involve అవ్వవు; assistant (DMA controller) మొత్తం transfer చేసి, చివర్లో ఒక్కసారి "అయ్యింది" అంటాడు.
>
> **Polling = నువ్వు పదేపదే check; Interrupt = device నిన్ను పిలుస్తుంది; DMA = assistant మొత్తం చేసి చివర్లో చెబుతాడు.**

Polling (Programmed I/O) — CPU పదేపదే adigithundi

CPU device యొక్క status register ని **loop లో పదేపదే check** చేస్తుంది — "ready అయ్యావా? ready అయ్యావా?" — ready అయ్యేదాకా.

```
while (device.status != READY)
    ;              // busy-wait — CPU ఇక్కడే గిర్రున తిరుగుతోంది (waste!)
transfer data;
```

- **సమస్య:** CPU device కోసం ఖాళీగా spin చేస్తూ **cycles waste** చేస్తుంది. Slow device (keyboard — నువ్వు type చేసేదాకా) కోసం lakhs of cycles waste.
- **ఎప్పుడు OK:** device చాలా వేగంగా ready అయ్యేటప్పుడు, లేదా చాలా simple systems లో. (Topic 8 spinlock ఇదే idea.)

Interrupts — device CPU ni pilusthundi

Polling తారుమారు: CPU device ని check చేయదు; device పని పూర్తయ్యాక **CPU కి signal (interrupt) పంపుతుంది.** అప్పటిదాకా CPU వేరే పని చేసుకుంటుంది.

```
1. CPU device కి "ఈ పని చెయ్యి" అని చెప్పి, వేరే process run చేస్తుంది
2. Device (background లో slowly) పని చేస్తుంది
3. పని అయ్యాక device → CPU కి INTERRUPT signal
4. CPU ప్రస్తుత పని ఆపి (state save), Interrupt Service Routine (ISR)
   / interrupt handler run చేస్తుంది
5. ISR అయ్యాక, CPU ఆగిన పనికి తిరిగి వెళ్తుంది
```

- **Interrupt Service Routine (ISR) / handler** = ఆ interrupt కి OS run చేసే code (ఉదా: "network packet వచ్చింది → దాన్ని buffer లో పెట్టు").
- **Interrupt Vector Table** = ప్రతి interrupt type కి ఏ handler అనేది map చేసే table.
- **లాభం:** CPU busy-wait చేయదు → efficient. **Modern systems ఇదే వాడతాయి.**

Interrupt types: hardware interrupts (device signals — keyboard, timer, network), software interrupts/traps (system calls, exceptions — divide-by-zero, page fault). **Timer interrupt** = scheduler ని క్రమం తప్పకుండా run చేసేది (preemptive scheduling కి foundation — Topic 5).

Node connection — ఇది huge: Node event loop యొక్క non-blocking model interrupt-driven I/O మీద నిర్మించబడింది. నువ్వు `socket.on('data', cb)` రాస్తే — CPU ఖాళీగా wait చేయదు; OS network interrupt వచ్చినప్పుడు data ready అని Node కి తెలుస్తుంది (epoll/kqueue), అప్పుడు callback queue అవుతుంది. **"CPU polls or gets interrupted" అనే OS idea = నీ event loop యొక్క పునాది.** Polling model అయితే Node ఇంత scalable అయ్యేది కాదు.

DMA — Direct Memory Access (CPU ni motham thappisthundi)

Interrupt మంచిదే, కానీ ఒక సమస్య: పెద్ద data transfer (ఉదా disk నుండి 1MB) లో — ప్రతి byte/word ని device నుండి memory కి **CPU తానే copy చేయాలి.** అంటే CPU ఇంకా transfer లో busy. **DMA దీన్ని పరిష్కరిస్తుంది.**

DMA controller = ఒక ప్రత్యేక hardware chip — CPU జోక్యం లేకుండా **device ↔ memory మధ్య నేరుగా data transfer** చేస్తుంది.

```
Without DMA:  Device → CPU → Memory   (CPU ప్రతి byte handle చేస్తుంది)
With DMA:     Device → DMA → Memory   (CPU free; DMA మొత్తం చేస్తుంది)

1. CPU DMA controller కి చెబుతుంది: "disk నుండి 1MB ని memory address X కి తే"
2. CPU వేరే పని చేసుకుంటుంది (free!)
3. DMA controller device ↔ memory మధ్య మొత్తం transfer చేస్తుంది
4. Transfer పూర్తయ్యాక DMA → CPU కి ఒక్క INTERRUPT ("అయ్యింది")
```

- **లాభం:** CPU పెద్ద transfers లో involve అవ్వదు → useful work కి free. ప్రతి byte కి interrupt కాకుండా, మొత్తం block కి ఒక్క interrupt.
- **ఎక్కడ:** disk I/O, network cards, GPU — అన్ని high-throughput devices DMA వాడతాయి.

మూడింటి పోలిక

విధానం	CPU involvement	Efficiency	ఎప్పుడు
Polling	CPU పదేపదే check (busy-wait)	పేలవం (cycles waste)	చాలా వేగవంతమైన/simple devices
Interrupt	Device signal, CPU free దాకా	మంచిది	సాధారణ I/O (నేటి standard)
DMA	CPU almost zero (setup + final interrupt)	ఉత్తమం	పెద్ద data transfers (disk/network)

నిజంగా అవి కలిసి పనిచేస్తాయి: CPU DMA ని setup చేస్తుంది → DMA transfer చేస్తుంది → పూర్తయ్యాక interrupt తో CPU కి చెబుతుంది. అంటే **interrupt + DMA కలిపి** ఆధునిక I/O.

Key Points

- **Polling** = CPU device status ని loop లో పదేపదే check (busy-wait) → CPU cycles waste. Simple కానీ inefficient.
- **Interrupt** = device పని అయ్యాక CPU కి signal పంపుతుంది; అప్పటిదాకా CPU వేరే పని. ISR/handler run అవుతుంది. **నేటి standard.**
- **Timer interrupt** = preemptive scheduling కి foundation.
- **DMA** = ప్రత్యేక controller device ↔ memory మధ్య CPU లేకుండా transfer; పూర్తయ్యాక ఒక్క interrupt. పెద్ద transfers కి ఉత్తమం.
- Node event loop = **interrupt-driven, non-blocking I/O** (epoll/kqueue) మీద నిర్మితం — polling model కాదు; అందుకే scalable.

Interview దృష్టి

Q1: Polling మరియు interrupt-driven I/O తేడా?

> Polling లో CPU device status register ని loop లో పదేపదే check చేస్తుంది — device ready అయ్యేదాకా busy-wait, CPU cycles waste. Interrupt-driven లో CPU device కి పని అప్పగించి వేరే పని చేసుకుంటుంది; device పూర్తయ్యాక CPU కి interrupt signal పంపుతుంది, అప్పుడు CPU handler (ISR) run చేస్తుంది. Interrupt చాలా efficient (busy-wait లేదు) — అందుకే నేటి systems దీన్ని వాడతాయి.

Q2: DMA అంటే ఏమిటి, ఏం సాధిస్తుంది?

> Direct Memory Access — ఒక ప్రత్యేక hardware controller device మరియు memory మధ్య data ని CPU జోక్యం లేకుండా నేరుగా transfer చేయడం. CPU DMA ని setup చేసి (source, dest, size), వేరే పని చేసుకుంటుంది; DMA మొత్తం transfer చేసి, పూర్తయ్యాక ఒక్క interrupt ఇస్తుంది. దీంతో పెద్ద data transfers (disk, network) లో CPU ప్రతి byte handle చేయనవసరం లేదు → CPU useful work కి free, throughput పెరుగుతుంది.

Q3: Interrupt వచ్చినప్పుడు CPU ఏం చేస్తుంది?

> ప్రస్తుత instruction పూర్తి చేసి, ప్రస్తుత process యొక్క state (PC, registers) ని save చేసి, interrupt vector table నుండి తగిన interrupt handler (ISR) address తీసి, kernel mode లో ఆ handler ని run చేస్తుంది (ఉదా packet ని buffer లో పెట్టడం). Handler పూర్తయ్యాక state restore చేసి, ఆగిన పనికి (లేదా scheduler నిర్ణయిస్తే వేరే process కి) తిరిగి వెళ్తుంది.

Q4: Timer interrupt scheduling కి ఎలా సంబంధం?

> Preemptive scheduling కి timer interrupt పునాది. Hardware timer క్రమం తప్పకుండా (ఉదా ప్రతి 10ms) interrupt పంపుతుంది; అది CPU control ని OS scheduler కి ఇస్తుంది. Scheduler "ఈ process quantum అయిపోయిందా, వేరేదానికి switch చేయాలా" అని నిర్ణయిస్తుంది. Timer interrupt లేకపోతే ఒక process CPU ని వదలకపోతే OS దాన్ని preempt చేయలేదు (cooperative scheduling అవుతుంది).

Q5: Node.js non-blocking I/O ఇందులో ఎక్కడ?

> Node network I/O కోసం CPU ని block/poll చేయదు — OS యొక్క interrupt-driven, event-notification mechanisms (Linux epoll, macOS kqueue) వాడుతుంది. Socket మీద data వచ్చినప్పుడు OS event loop కి తెలియజేస్తుంది, అప్పుడు Node సంబంధిత callback ని queue చేస్తుంది. అంటే CPU idle wait చేయకుండా వేలాది connections handle చేస్తుంది — ఇది interrupt/DMA-driven efficient I/O యొక్క application-level ఫలితం.

Part 6 — Reference (interview కి చివరి మెరుగు)

> చివరి Part — రెండు topics. మొదటిది IPC (processes ఒకదానితో ఒకటి ఎలా మాట్లాడతాయి — నీ microservices, worker processes కి ప్రత్యక్షంగా సంబంధించినది). రెండోది ఒక పెద్ద interview Q&A bank + గుర్తుంచుకోవడానికి memory tips table + అందరూ చేసే common mistakes. Interview ముందు రోజు ఈ Part revise చెయ్యి.

---

18. IPC — Inter-Process Communication (pipes, shared memory, message passing, sockets)

వివరణ

Processes ఒకదాని memory ఇంకొకటి చూడలేవు (isolation — Topic 2). మరి రెండు processes **data ఎలా పంచుకుంటాయి / coordinate అవుతాయి?** ఇదే **IPC (Inter-Process Communication)** — processes మధ్య communication కి OS ఇచ్చే mechanisms.

ఎందుకు అవసరం? నీ Node `cluster` workers ఒకరితో ఒకరు; నీ microservices ఒకదానితో ఒకటి; shell లో `cat file | grep foo | wc -l` (pipe); database server ↔ నీ app — ఇవన్నీ IPC. **Isolation మంచిదే (safety), కానీ కొన్నిసార్లు processes కలిసి పని చేయాలి — IPC ఆ వంతెన.**

రెండు ప్రధాన నమూనాలు:

1. **Shared memory** — రెండు processes ఒకే memory region ని share చేస్తాయి; నేరుగా read/write.
2. **Message passing** — processes OS ద్వారా messages పంపుకుంటాయి (memory share చేయవు).

Real-life Scenario

> **IPC = రెండు వేర్వేరు ఇళ్ళలో (isolated processes) ఉన్న స్నేహితులు మాట్లాడుకోవడం.**
>
> - **Shared memory = ఒక common notice board** రెండిళ్ళ మధ్య గోడ మీద. ఒకరు రాస్తారు, ఇంకొకరు చదువుతారు — వేగం (నేరుగా), కానీ ఇద్దరూ ఒకేసారి రాస్తే గందరగోళం (synchronization కావాలి!).
> - **Message passing = ఉత్తరాలు/postman.** ఒకరు letter రాసి post చేస్తారు (send), ఇంకొకరు అందుకుంటారు (receive). నెమ్మది (postman ద్వారా = OS ద్వారా copy), కానీ safe (ఎవరి ఇల్లు వాళ్ళదే, గందరగోళం లేదు).
> - **Pipe = రెండిళ్ళ మధ్య ఒక గొట్టం** — ఒక చివర పోస్తే, ఇంకో చివర వస్తుంది (ఒకే దిశ).
> - **Socket = phone line** — దూరపు ఇళ్ళ మధ్య కూడా (వేరే machines కూడా) మాట్లాడవచ్చు.

IPC Mechanisms

1. Pipes

Pipe = ఒక process యొక్క output ని ఇంకో process యొక్క input కి కలిపే ఒక **unidirectional (ఒకే దిశ) byte stream** (in-memory FIFO buffer). Producer ఒక చివర రాస్తుంది, consumer ఇంకో చివర చదువుతుంది.

- **Anonymous pipe: related processes** (parent-child) మధ్య మాత్రమే. Shell లో `ls | grep txt` — `ls` output ఒక pipe ద్వారా `grep` input కి. (`|` = pipe!)
- **Named pipe (FIFO):** filesystem లో ఒక పేరు ఉంటుంది → **unrelated processes** కూడా వాడవచ్చు.
- Node: `child.stdout.pipe(...)` , streams — ఇవి pipe abstraction. Backpressure (Topic 9 bounded buffer) ఇక్కడ వర్తిస్తుంది.

2. Shared Memory (అత్యంత వేగం)

రెండు processes ఒకే physical memory region ని తమ address spaces లో map చేసుకుంటాయి. ఒకరు రాసింది ఇంకొకరు వెంటనే చూస్తారు — **OS ద్వారా copy లేదు కాబట్టి fastest IPC.**

```
  Process A             Shared Region (RAM)         Process B
 ┌────────┐            ┌─────────────────┐         ┌────────┐
 │ writes ├───────────►│   shared data   │◄────────┤ reads  │
 └────────┘            └─────────────────┘         └────────┘
                 (both map the same physical pages)
```

- **లాభం:** fastest (direct memory, no kernel copy per access).
- **నష్టం: synchronization మీ బాధ్యత** — ఇద్దరూ ఒకేసారి రాస్తే race condition (Topic 7)! Semaphores/mutexes తప్పనిసరి. అందుకే fast కానీ error-prone.
- Node: `SharedArrayBuffer + worker_threads` — threads మధ్య shared memory; `Atomics` తో synchronize. Node `cluster` workers (processes) మధ్య OS shared memory అరుదు — message passing ఎక్కువ.

3. Message Passing

Processes memory share చేయకుండా, OS ద్వారా **send(message)** / **receive(message)** తో communicate. OS message ని ఒక process నుండి ఇంకోదానికి copy చేస్తుంది.

- **లాభం:** safe (no shared state → races తక్కువ), వేరే machines కి కూడా విస్తరించవచ్చు (distributed).
- **నష్టం:** shared memory కంటే slow (OS copy overhead).
- **Sync vs async:** blocking send/receive (అవతలివాడు అందుకునేదాకా/పంపేదాకా wait) vs non-blocking.
- **Direct vs indirect:** నేరుగా process కి vs mailbox/message queue ద్వారా.
- **Node connection:** `worker.postMessage()` , `process.send()` (cluster), `child.send()` — ఇవన్నీ message passing. Data serialize/copy అవుతుంది (structured clone) — అందుకే పెద్ద objects పంపడం ఖరీదు. **Node processes మధ్య default = message passing (safe), shared memory కాదు.**

4. Sockets

Socket = network communication endpoint — **వేరే machines మధ్య** (లేదా అదే machine లో) processes కమ్యూనికేట్ చేసే mechanism. IP address + port తో identify. TCP (reliable, ordered — నీ HTTP/Express) లేదా UDP (fast, unreliable).

- **లాభం:** machines దాటవచ్చు (distributed systems, microservices, client-server).
- **Unix domain socket** = అదే machine లో processes మధ్య (network stack లేకుండా, faster). నీ MongoDB/Redis local connection తరచూ Unix socket.
- **Node connection:** నీ మొత్తం MERN world sockets మీద — Express server (TCP socket), `net` module, WebSocket, MongoDB driver (TCP/Unix socket). **నువ్వు రోజూ వాడేది ఇదే IPC.**

అన్నిటి పోలిక

Mechanism	వేగం	పరిధి	Synchronization	ఉదాహరణ
Pipe	మధ్యస్థం	same machine, related	OS handles buffer	`ls | grep`, streams
Shared memory	**fastest**	same machine	**మీ బాధ్యత** (semaphore)	`SharedArrayBuffer`
Message passing	slow	same machine (extendable)	built-in (copy)	`worker.postMessage`
Socket	slow(er)	**వేరే machines కూడా**	protocol (TCP)	HTTP, DB connection

Trade-off crux: Shared memory = fastest కానీ synchronization మీ తలనొప్పి; Message passing/sockets = slower కానీ safe + distributed. "Share memory to communicate" vs "Communicate to share" — Go/Erlang "communicate to share" (message passing) ని ఇష్టపడతాయి, safety కోసం.

Key Points

- **IPC = isolated processes మధ్య communication.** రెండు నమూనాలు: **shared memory** (share to communicate) vs **message passing** (communicate to share).
- **Pipe** = unidirectional byte stream (`ls | grep` , Node streams); anonymous (related) vs named FIFO (unrelated).
- **Shared memory** = fastest (direct, no copy) కానీ **synchronization మీ బాధ్యత** (race conditions).
- **Message passing** = safe (no shared state, OS copy) కానీ slow; `worker.postMessage` , `process.send` .
- **Socket** = వేరే machines మధ్య కూడా (IP+port, TCP/UDP); నీ మొత్తం web stack ఇదే. Unix domain socket = same machine, faster.
- **Trade-off:** shared memory వేగం vs safety; message passing/sockets safety + distributability.

Interview దృష్టి

Q1: IPC ఎందుకు అవసరం, ప్రధాన mechanisms ఏమిటి?

> Processes memory-isolated కాబట్టి (ఒకరి memory ఇంకొకరు చూడలేరు) — data పంచుకోవడానికి/coordinate అవ్వడానికి OS-provided mechanisms అవసరం. ప్రధానంగా: pipes (byte stream, related processes), shared memory (fastest, direct), message passing (send/receive, safe), sockets (network — వేరే machines కూడా). రెండు నమూనాలు: shared memory vs message passing.

Q2: Shared memory మరియు message passing తేడా, trade-off?

> Shared memory = రెండు processes ఒకే memory region access చేస్తాయి; direct read/write కాబట్టి fastest, కానీ synchronization (mutex/semaphore) programmer బాధ్యత — లేకపోతే race conditions. Message passing = OS ద్వారా data copy చేసి send/receive; slower కానీ safe (shared state లేదు) మరియు వేరే machines కి extend అవుతుంది. వేగం కావాలంటే shared memory, safety/distribution కావాలంటే message passing.

Q3: Pipe అంటే ఏమిటి, shell లో `\|` ఎలా పనిచేస్తుంది?

> Pipe = ఒక unidirectional in-memory byte stream (FIFO buffer) — ఒక process రాస్తుంది, ఇంకోది చదువుతుంది. Shell లో `ls | grep txt` — OS ఒక pipe create చేసి, `ls` యొక్క stdout ని pipe write-end కి, `grep` యొక్క stdin ని pipe read-end కి కలుపుతుంది. `ls` output నేరుగా `grep` కి stream అవుతుంది, intermediate file అవసరం లేదు.

Q4: Node.js cluster/worker_threads లో ఏ IPC వాడతారు?

> `cluster` (multiple processes) — message passing: `process.send()` / `worker.send()` , data serialize (copy) అవుతుంది కాబట్టి safe. `worker_threads` (threads) — message passing (`postMessage` , structured clone) డిఫాల్ట్; అదనంగా `SharedArrayBuffer` + `Atomics` తో నిజమైన shared memory కూడా (fastest కానీ synchronization జాగ్రత్త). Isolation/safety కోసం Node message passing ని ఇష్టపడుతుంది.

Q5: Socket-based IPC ఎప్పుడు వాడతారు?

> Processes వేరే machines లో ఉన్నప్పుడు (distributed systems, microservices, client-server) — sockets IP+port తో network ద్వారా communicate చేస్తాయి (TCP reliable/ordered, UDP fast/unreliable). అదే machine లో అయితే Unix domain socket (network stack bypass, faster). నీ Express server, MongoDB connection, WebSocket — అన్నీ socket IPC. అంటే web development లో అత్యంత సాధారణ IPC ఇదే.

19. Interview Q&A + Memory Tips + Common Mistakes

వివరణ

ఇది revision Part — interview ముందు రోజు చదవడానికి. మూడు భాగాలు: (A) rapid-fire Q&A bank (అన్ని topics కవర్), (B) memory tips table (గుర్తుంచుకోవడానికి mnemonics), (C) common mistakes (అందరూ చేసేవి — నువ్వు చేయకు).

A. Rapid-Fire Interview Q&A Bank

Part 1 — Basics

Q: Process vs Thread ఒక్క వాక్యంలో?

> Process = own memory ఉన్న independent program instance; thread = process లోపల execution path, memory share చేస్తుంది కానీ own stack/registers. Process isolated+heavy, thread lightweight+shared.

Q: User mode vs kernel mode ఎందుకు?

> Protection. User mode privileged instructions block చేస్తుంది; buggy app OS ని కూల్చలేదు. Privileged పని కావాలంటే system call ద్వారా kernel ని అడగాలి.

Q: System call function call కి ఎలా వేరు?

> System call = user→kernel mode switch (trap), OS privileged పని చేస్తుంది, ఖరీదు ఎక్కువ. Function call = అదే process/mode లో jump.

Q: fork() ఎన్నిసార్లు return అవుతుంది?

> ఒకసారి పిలిస్తే రెండుసార్లు — parent లో child PID, child లో 0.

Q: Node.js single-threaded అయితే concurrency ఎలా?

> JS execution single-threaded (event loop); network I/O ని OS non-blocking (epoll), file/crypto/DNS ని libuv thread pool (4) కి offload చేస్తుంది. CPU-heavy పని event loop ని block చేస్తుంది → worker_threads/cluster.

Q: Zombie vs orphan process?

> Zombie = child పూర్తయ్యింది కానీ parent exit status చదవలేదు (PCB మిగిలింది). Orphan = parent ముందే చనిపోయింది; init/systemd దత్తత తీసుకుంటుంది.

Q: Context switch ఖరీదు ఎందుకు?

> State save/load (direct) + cache/TLB pollution (indirect — కొత్త process cold cache). Thread switch < process switch (TLB flush అవసరం లేదు).

Part 2 — Scheduling

Q: TAT, WT formulas?

> TAT = CT − AT; WT = TAT − BT.

Q: ఏ algorithm min avg waiting time, ఎందుకు impractical?

> SJF/SRTF; burst time ముందే తెలియదు (అంచనా వేయాలి) + starvation.

Q: Preemptive vs non-preemptive?

> Non-preemptive = process తనే వదిలేదాకా CPU తీసుకోరు (convoy effect). Preemptive = OS ఏ క్షణంలోనైనా తీసుకోగలదు (responsive, ఎక్కువ switches).

Q: Starvation పరిష్కారం?

> Aging — ఎక్కువసేపు wait చేసిన process priority ని క్రమంగా పెంచడం.

Q: Round Robin quantum trade-off?

> చిన్నది → ఎక్కువ switches/overhead; పెద్దది → FCFS లా అవుతుంది. ~80% bursts quantum లో పూర్తవ్వాలి.

Q: నిజ OS ఏ scheduler?

> MLFQ variants (behavior-based), Linux CFS (red-black tree, fair).

Part 3 — Concurrency

Q: Race condition అంటే?

> Concurrent shared-data access, result execution order మీద ఆధారపడటం. ఉదా: `count++` (load-add-store) interleave → lost update.

Q: Critical section 3 అవసరాలు?

> Mutual Exclusion + Progress + Bounded Waiting.

Q: Mutex vs semaphore?

> Mutex = ownership ఉన్న lock, ఒక్కరే. Semaphore = counter (0..N), ownership లేదు, resource counting + signaling. Binary semaphore ≈ mutex కానీ ownership లేదు.

Q: Producer-consumer semaphores?

> mutex(1), empty(N), full(0). counting semaphore ముందు, mutex తర్వాత wait (లేకపోతే deadlock).

Q: Deadlock 4 conditions?

> Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait (అన్నీ కావాలి). "ME HoW No Circle."

Q: Banker's algorithm ఏం చేస్తుంది?

> ప్రతి request ముందు "safe state?" check; unsafe అయితే wait. Safe state = అందరినీ finish చేయగల safe sequence ఉంది. Max ముందే తెలియాలి → theoretical.

Q: Deadlock prevention లో practical fix?

> Circular wait break — locks ని ఎప్పుడూ ఒకే global order లో acquire చెయ్యి.

Q: Deadlock vs livelock vs starvation?

> Deadlock = frozen (no motion). Livelock = కదులుతున్నా progress లేదు. Starvation = ఒకడు మాత్రం అనంతంగా wait.

Part 4 — Memory

Q: Logical vs physical address, MMU?

> Logical = program view; physical = RAM లో నిజం. MMU (hardware) ప్రతి access కి translate + bounds check.

Q: Internal vs external fragmentation?

> Internal = allocated block అవసరం కంటే పెద్దది (లోపల waste — paging). External = free memory scattered, contiguous hole లేదు (segmentation/contiguous).

Q: Paging ఏం fix చేస్తుంది?

> External fragmentation (non-contiguous frames). Logical addr = page# + offset; page table page→frame.

Q: TLB ఎందుకు?

> Page table RAM లో → ప్రతి access రెట్టింపు slow; TLB (hardware cache) recent mappings ఉంచి fix చేస్తుంది (locality → hit ~99%).

Q: Page fault అంటే?

> Access చేసిన page RAM లో లేదు → OS disk నుండి load (చాలా ఖరీదు, ~100,000× slow).

Q: Belady's anomaly?

> FIFO లో frames పెంచితే faults పెరగవచ్చు. LRU/Optimal (stack algorithms) దీనికి గురికావు.

Q: Thrashing?

> Work కంటే page swapping ఎక్కువ → CPU util కుప్పకూలుతుంది. Fix: working set model.

Q: నిజ OS ఏ replacement?

> Clock (second-chance) — LRU యొక్క చౌక approximation (reference bit).

Part 5 — Storage & I/O

Q: inode లో ఏం ఉంటుంది, filename ఉంటుందా?

> Metadata (perms, size, owner, timestamps, link count) + data block pointers. Filename ఉండదు (directory లో "name→inode").

Q: Hard link vs soft link?

> Hard = అదే inode కి ఇంకో name (link count). Soft = target path ని ఉంచే separate file (broken అవుతుంది target పోతే).

Q: Disk scheduling లక్ష్యం?

> Seek time (head movement) minimize. SSTF = starvation; SCAN = elevator; C-SCAN = uniform wait.

Q: Polling vs interrupt vs DMA?

> Polling = CPU పదేపదే check (waste). Interrupt = device CPU ని పిలుస్తుంది (efficient). DMA = controller device↔memory transfer, CPU free, చివర్లో ఒక్క interrupt.

Part 6 — IPC

Q: Shared memory vs message passing?

> Shared memory = fastest, direct, synchronization మీ బాధ్యత. Message passing = safe, OS copy, slower, distributable.

Q: Socket IPC ఎక్కడ?

> వేరే machines మధ్య (IP+port, TCP/UDP). నీ Express/MongoDB/WebSocket అన్నీ sockets.

B. Memory Tips Table (గుర్తుంచుకోవడానికి)

Concept	Trick / Mnemonic	ఎలా గుర్తుంచుకోవాలి
Deadlock 4 conditions	**"ME HoW No Circle"**	**M**utual **E**xclusion, **Ho**ld & **W**ait, **No** preemption, **Circle** (circular wait)
Critical section 3 needs	**"ME Pro Bo"**	**M**utual **E**xclusion, **Pro**gress, **Bo**unded waiting
TAT & WT	"**T**otal time (CT−AT), **W**ait = Total − Burst"	TAT = CT−AT; WT = TAT−BT
Ready vs Waiting	"**Ready** = CPU కావాలి; **Waiting** = I/O కావాలి"	Ready CPU ఇస్తే run; Waiting I/O పూర్తవ్వాలి
Internal vs External frag	"**In**ternal = **in**side block waste (paging); **Ex**ternal = scattered **ex**ternal holes (segmentation)"	Fixed size → internal; variable size → external
Paging vs Segmentation	"Pa**g**ing = **g**leam equal chunks (hardware); Se**g**ment = **g**roup by meaning (programmer)"	Equal→paging; meaningful→segmentation
Page replacement quality	"**O**ptimal > **L**RU ≈ **C**lock > **F**IFO" (OLCF)	Optimal best (impractical), FIFO worst (Belady)
Mutex vs Semaphore	"**Mut**ex = **owner** తో tal**k** (lock); **Sema**phore = **count**er signal"	Mutex ownership; semaphore counter
Producer-consumer wait order	"**Count first, lock last**"	wait(empty/full) ముందు, wait(mutex) తర్వాత
SCAN	"**Elevator**"	Lift లా ఒక దిశ చివరిదాకా, తిరిగి
Polling/Interrupt/DMA	"**Check / Call / Courier**"	Polling=check; Interrupt=device calls; DMA=courier transfers
Starvation fix	"**Aging**" (అన్నివోట్లూ)	Priority scheduling, disk SSTF — waited వాటి priority పెంచు
fork returns	"**Two returns, one call**"	Parent: child PID; Child: 0
System call cost	"**Trap = expensive door**"	Mode switch ఖరీదు; batch చెయ్యి

C. Common Mistakes (అందరూ చేసేవి — నువ్వు చేయకు)

❌ తప్పు అభిప్రాయం	✅ నిజం
"Process మరియు thread ఒకటే"	Process own memory (isolated); thread memory share చేస్తుంది (own stack మాత్రమే).
"Node.js కి threads లేవు, పూర్తిగా single-threaded"	JS execution single-threaded; libuv కి thread pool (4) ఉంది; worker_threads నిజ OS threads.
"Ready state = Running state"	Ready = CPU కోసం wait (run చేయడానికి సిద్ధం); Running = actually CPU మీద.
"Waiting = Ready"	Waiting = I/O కోసం (CPU ఇచ్చినా వాడలేదు); Ready = CPU కోసం.
"SJF ఎప్పుడూ వాడవచ్చు"	Burst time ముందే తెలియదు → అంచనా; + starvation. అందుకే practical కాదు.
"Preemptive ఎప్పుడూ better"	ఎక్కువ context switches = overhead; short tasks కి non-preemptive సరిపోవచ్చు.
"Mutex = binary semaphore"	Binary semaphore ≈ mutex కానీ **ownership లేదు;** mutex తీసుకున్నవాడే వదలాలి.
"Semaphore అంటే ఎప్పుడూ mutual exclusion"	Counting semaphore = N resources; signaling కి కూడా (event notification).
"Deadlock = ఏదో ఒక condition చాలు"	**నాలుగు** conditions అన్నీ ఏకకాలంలో కావాలి; ఒకటి break చేస్తే deadlock రాదు.
"Banker's algorithm ని నిజ OS లు వాడతాయి"	Max ముందే తెలియాలి + overhead → theoretical; నిజ OS లు ఎక్కువగా ignore (ostrich).
"frames పెంచితే ఎప్పుడూ page faults తగ్గుతాయి"	FIFO లో పెరగవచ్చు (**Belady's anomaly**); LRU/Optimal మాత్రమే guarantee.
"Virtual memory = RAM లేని extra hardware"	Virtual memory = abstraction (కొన్ని pages disk లో); hardware కాదు, technique.
"Paging internal fragmentation ని ఇవ్వదు"	ఇస్తుంది (చివరి page waste), కానీ **external** fragmentation ని పోగొడుతుంది.
"Segmentation fault = code bug మాత్రమే"	అది OS memory-protection mechanism (unallocated memory access → hardware trap).
"inode లో filename ఉంటుంది"	ఉండదు; filename directory లో ("name→inode number"). అందుకే hard links సాధ్యం.
"Disk scheduling SSD కి ముఖ్యం"	SSD కి seek time లేదు → classic algorithms తక్కువ relevant.
"Polling interrupt కంటే simple కాబట్టి better"	Polling CPU cycles waste చేస్తుంది; interrupt-driven నేటి standard (efficient).
"Shared memory ఎప్పుడూ best IPC (fastest కదా)"	Fastest అవును, కానీ synchronization మీ బాధ్యత (races); safety కి message passing.
"`count++` atomic operation"	కాదు — load/add/store (3 steps); race కి గురవుతుంది.

❌ తప్పు అభిప్రాయం	✅ నిజం
"Node లో race conditions అస్సలు ఉండవు"	Variables మీద ఉండవు, కానీ `await` **boundaries దగ్గర logical races** ఉంటాయి.

D. చివరి మాట — OS ని ఒక్క చిత్రంలో

```
┌─────────────────────────────────────────────────────────────┐
│  నీ MERN app (user mode) ── system calls ──► OS Kernel       │
│                                                              │
│  OS 4 పెద్ద పనులు:                                           │
│   1. Process/Thread management  → ఎవరు run అవ్వాలి          │
│      (scheduling, context switch)                            │
│   2. Concurrency control        → safe గా share              │
│      (mutex, semaphore, deadlock నివారణ)                     │
│   3. Memory management           → RAM ని పంచడం              │
│      (paging, virtual memory, TLB)                           │
│   4. Storage & I/O               → data + devices            │
│      (file system, disk sched, interrupts, DMA)              │
│                                                              │
│  అన్నిటి వెనుక ఒకటే idea: పరిమిత hardware ని,               │
│  అనేక programs కి, safe గా, fair గా, efficient గా పంచడం.    │
└─────────────────────────────────────────────────────────────┘
```

నీ web knowledge తో OS ముడి (గుర్తుంచుకో):

- Node event loop ← OS interrupt-driven non-blocking I/O (epoll/kqueue)
- libuv thread pool, worker_threads ← OS threads
- cluster / child_process ← fork + exec, IPC (message passing)
- MongoDB connection pool ← counting semaphore
- `await` boundary races ← critical section problem
- Container OOM / swap slowness ← thrashing, working set
- `fs.stat` fields (size, mtime, ino, nlink) ← inode
- Express/DB connections ← sockets (IPC)
- "Segmentation fault", "stack overflow" ← memory layout + protection

Interview దృష్టి

Q: OS ఒక్క వాక్యంలో ఏం చేస్తుంది?

> పరిమిత hardware resources (CPU, RAM, disk, devices) ని అనేక programs మధ్య safe గా (isolation/protection), fair గా (scheduling), efficient గా (virtual memory, caching) పంచే resource manager + hardware ని దాచే abstraction layer.

Q: ఈ guide లో ఏ 5 concepts అత్యంత తరచుగా SSE interviews లో వస్తాయి?

> (1) Process vs thread + Node.js single-thread model, (2) Deadlock 4 conditions + prevention, (3) Mutex vs semaphore + race conditions, (4) Paging + virtual memory + page replacement (Belady/thrashing), (5) Scheduling algorithms (Gantt + TAT/WT calculation). ఈ ఐదింటిని deep గా, examples తో ready గా ఉంచుకో.

Q: OS fundamentals నా daily web development లో ఎక్కడ కనిపిస్తాయి?

> Node event loop (non-blocking I/O = interrupt-driven), connection pools (counting semaphore), async races at await (critical section), container memory limits (thrashing/working set), worker_threads/cluster (threads/processes + IPC), file operations (inodes/fd), network calls (sockets). OS ప్రతి abstraction వెనుక నిశ్శబ్దంగా పనిచేస్తోంది — అది తెలిస్తే bugs త్వరగా అర్థమవుతాయి, systems బాగా design చేస్తావు.

> **అభినందనలు!** నువ్వు OS ని ZERO నుండి SSE interview స్థాయి వరకు పూర్తి చేశావు — 19 topics, ప్రతిదానికి analogy + diagram + interview angle. ఇప్పుడు ఈ guide ని ఒకసారి పూర్తిగా చదివి, తర్వాత ప్రతి topic చివరి **Interview దృష్టి** ప్రశ్నలని బిగ్గరగా (loud గా) సమాధానం చెప్పి practice చెయ్యి. Gantt charts, page-replacement traces, Banker's algorithm — వీటిని కాగితం మీద చేతితో వేసి practice చెయ్యి. **"ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" — ఇప్పుడు అది నీది.** All the best! 🚀